Design Patterns for Searching in C#

By Fred Mellender

Copyright © 2008 by Fred Mellender. All rights reserved.

Contact the author at fredm73@hotmail.com.

You can download the source code and the SEL library at: http://www.lulu.com/content/2008403

Source code may be freely used, copied, and modified. However, no warranty is given as to its suitability or accuracy.

The source code was compiled under Microsoft's Visual Studio 2005, Standard Edition (C#), and uses generics, iterators, and anonymous methods.

ISBN: 978-1-4357-2301-6

Contents

Preface.....vii
1 Permutations.....11
Design Patterns.....12
Permutations.....13
Lexigraphical Order and “Cut”.....16
Summary of the Permutation Design Pattern.....19
2 Combinations and Cartesian Product.....21
Combinations.....22
Summary of the Combination Design Pattern.....28
3 Depth First Search.....31
Depth First Search Classes.....34
DFS Class Collaboration.....38
Some Graph Theory.....39
Chains in DFS.....40
Application Analysis.....45
DFS Debugging Tips.....59
How DFS Works.....60
Depth Bound.....60
Summary of the Depth First Search pattern.....61
4 Variations on Depth First Search.....63
Divide and Conquer (D&C).....63
Performance of D&C.....70
Recursion vs. DFS.....70
Summary of the Divide and Conquer Pattern.....70
Branch and Bound (B&B).....71

Heuristics....77
Summary of the Branch and Bound Pattern....80
5 Dynamic Programming....83
Using DFS in Dynamic Programming....84
Branch and Bound Revisited....94
Summary of the Dynamic Programming Pattern....105
6 Breadth First Search....107
Best-First....110
Greedy Search....112
Beam Search....119
A Storage Optimization....126
Summary of the Breadth First Search Design Pattern....126
7 A*....129
Heuristics....129
Summary of the A* Design Pattern....143
8 Game Trees....145
Preliminary notions....145
Minimax....148
Alpha/Beta Pruning....160
Summary of the Game Tree Design Pattern....163
Iterative Deepening and Move Ordering....164
9 Simulated Annealing....167
The SA Algorithm....168
Summary of the Simulated Annealing Design Pattern....178
Envoi....179
Bibliography....181

Index of Problems and Applications

Traveling Salesman Problem (TSP)....14
Machine Sequencing....15
8-Queens....15
Quadratic Assignment Problem....18
Obtaining all sublists of a list....22
Creating a round-robin tournament....23
Nested parentheses....23
Combinations of Combinations....25
8-Queens Revisited....31
Searching a Maze....40
A Parser....48
Parenthesizing a list....64
Knapsack....72
Knapsack Problem (Revisited)....83
Lot Sizing Problem....96
TSP, Version 1, Greedy Search....113
TSP, Version 2, Beam Search....120
TSP, Version 3, Beam Search....122
Knapsack via A*....132
15-Puzzle....135
Reversi....150
TSP solved via SA....168

Preface

DESIGNING OBJECT ORIENTED software is difficult, but there are reoccurring patterns that have been documented [1]. These patterns specify how interacting classes and objects can combine to solve very general problems. It is up to the designer to recognize when a pattern can be employed and then to implement the domain specific classes and objects that follow the pattern, in order to serve the application functionality.

This book takes off from two design patterns mentioned in the literature, *Iterator* and *Template Method*. We devise sub-patterns that are specific for *enumeration* (constructing collections of objects and then making them available one at a time), and *searching* (ranging over an object space to find objects that satisfy certain criteria).

Readers will require some object-oriented experience. You should be comfortable with the notions of objects, classes, inheritance, interfaces, methods, and the like. No knowledge of design patterns is required.

We will present some of the classic search algorithms in a new setting. You need not be familiar with these already. The book does not give extensive mathematical analysis of the algorithms used. Hints are given when there are particular inefficiencies or when obvious improvements can be made. To maintain focus it was necessary that the examples lack detail and complexity. However, it is intended that readers will make practical use of the design patterns in real projects.

Our book contains examples in CSharp (C#), version 2. This language was chosen because of its implementation of "generics" and "iterators", and because it has a useful library of collection classes. We could have used Java, Smalltalk, or C++ instead, but C# is especially concise and the example code therefore relatively uncluttered. Certainly the patterns themselves are not language specific: you can probably translate the code into the object-oriented language of your choice. However, we will not discuss C# in much detail, so a prior knowledge of that language will be helpful.

Typically, design patterns are too abstract to be reduced to code but must be implemented every time they're used. With the advent of generic classes and iterators in C# it is possible to separate the part of the patterns that require application specific classes from the part that controls the searching/enumeration logic. The latter piece we put in a small class library (called the Searching and Enumeration Library, or *SEL*)*. By doing this we not only provide code for your reuse, but we can devote most of our discussion to the concepts that require the designer's imagination in applying the pattern.

The sample applications in the book necessarily lack complexity so that they can be described briefly. Furthermore, issues of error detection and efficiency have been largely ignored. Occasionally, some of the source code has been omitted from the text (but is available along with the SEL). However, all of the applications are complete enough to be executed and include a simple user interface. At least two of the examples, a parser and the game of Reversi, are rich enough to be used as a framework for similar applications.

It is best to read this book from start to finish. The early design patterns are simple and the discussion rather verbose. Subsequent patterns become more

* The SEL and C# source code for this book can be found at: http://www.lulu.com/content/2008403

complex and the discussion a bit more terse. Some vocabulary introduced earlier is reused, as are some examples.

1 *Permutations*

EVEN THE SIMPLEST of programs is likely to involve a search of some sort. The one every programmer is familiar with is searching sequentially through an array, as in:

```
int[] someInts = new int[10];

for (int i = 0; i < someInts.Length; i++)
{
    //do something with someInts[i]
}
```

C# has given us a way to enumerate the members of the array with a `for` loop. Our sequential search must examine each element in the entire array.

C# has also provided a way to enumerate an arbitrary collection with an *iterator*, as in:

```
List<MyClass> myList;
foreach (MyClass myObj in myList)
{
    //do something with myObj
}
```

Here we are using "generics" which let us define the type of elements in `myList` (they are of type `MyClass`). Our iterator (introduced with the `foreach` keyword) lets us enumerate all the elements in the collection sequentially.

A more complicated kind of search is the binary search, which also has support in C#:

```
int hit = myList.BinarySearch(someObject, aComparer);
```

The `BinarySearch` expects the list to be in order, and that order is specified by an object that inherits from `IComparer`. The returned integer (`hit`) will specify either the index that contains the match, or be a negative

integer whose bitwise complement can be used to insert the search object (`someObject`) into the list in the proper spot.

In this book we will explore techniques for enumeration and searching. You will find these methods useful not only for doing searches and enumerations within an application, but also for using the search patterns as the major architectural scaffolding for the application.

Design Patterns

A design pattern has been defined as a description of "communicating objects and classes that are customized to solve a general design problem in a particular context" [1, p.3]. The two general design problems we will examine are

1. Constructing collections of objects and then enumerating them.

2. Searching through an object space to find objects that satisfy certain criteria.

These are very general problems and have little meaning without some application context. The design patterns we will devise must be customized to be useful. We will provide examples and motivation for their use.

Two of the design patterns identified in [1] are the *Iterator* and the *Template Method*. An Iterator provides sequential access to a collection without revealing the structure or the control logic. As mentioned above, C# provides language constructs for creating and using iterators. Programmers can write their own iterators:

```
foreach (Node node in graph.depthFirst())
{
        //do something to node
}
```

The object `graph` implements an iterator method, called `depthFirst`. To the user of the iterator, the method looks like a collection. The `graph` "underneath the covers" can be doing complicated control logic and, perhaps, creating objects on the fly as they are needed. As the above code hints, most of our searching patterns will have iterators that application classes can employ.

The Template Method design pattern separates the control aspects of an algorithm from the application specifics of the data and subroutines that refine the algorithm. Typically, an abstract class will be written that makes calls to abstract methods in a certain order, with certain control logic. It is up to a designer's subclass to implement the abstract methods (in a concrete subclass) in an application context.

In this book, we will use generic classes to implement the control aspects of certain search routines. These classes were placed in a class library (called the Search and Enumeration Library, or **SEL**). Instead of subclasses, the designer will invent application specific classes that inherit from some interface, also defined in the **SEL**.

As described above, the designer's use of the **SEL** would seem to be just a context-specific implementation of the Template Method Pattern. However, our intent is to show that a wide variety of applications can be designed around search patterns. We will challenge you to think about application design based on the patterns we will describe subsequently.

This chapter and the next will describe two of the simplest enumeration tasks. We start with permutations.

Permutations

If we have a list of items, say integers **[1, 2, 3, 4, 5]**, a permutation of that list is just a rearrangement of the items in the list. Thus **[1, 3, 2, 4, 5]** is a permutation of the first list. The problem of determining all permutations of a list occurs often enough to deserve a generic solution.

```
1               List<int> ints = new List<int>(5);
2               for (int i = 1; i <= 5; i++)
3                   ints.Add(i);
4               Permute<int> permute = new Permute<int>(ints);
5               foreach (List<int> ans in permute.permutations())
6               {
7                    if (ans[0] == 5 || ans[ans.Count-1] == 5)
8                       aFiver(ans);
9               }
```

Lines 1-3 in the code above will build a list of integers, [1, 2, 3, 4, 5]. All permutations of that list are enumerated in lines 4-5. The class `Permute` is supplied in the **SEL**. It is a generic class, and as such must be supplied with

a type. At line 4, we construct an object, `permute`, of type `Permute<int>` and initialize it with our list, `ints`.

The `foreach` statement makes use of the iterator, `permute.permutations`. This will return all permutations, one at a time, in the variable `ans`, which can be examined in the body of the iterator loop. In this example, we see if `ans` begins or ends with a five.

Now this is so simple it hardly needs be dignified as a "pattern". However, it does have some interesting aspects we will find in search patterns introduced later:

1.We have an "object space", (the set of all permutations of a list), that we wish to range over.

2.We want to select out and examine either all of the objects in the space, or a subset that satisfy some criteria.

3.We separated the control logic (the generation of the permutations) from the application logic. The former could be coded and reused across applications. The latter must be redesigned for each application, once the "permutation pattern" is recognized as a solution.

Thus the **SEL** provides a toolkit for finding the permutations of a list of any type. Note that if an object appears twice in the original list, it will appear twice in each permutation.

Here are some sample problems that can be solved by the "permutation pattern".

TRAVELING SALESMAN PROBLEM (TSP)

Suppose we have a list of cities, one marked as the start. We want to construct all "tours" that begin at the start city, visit each of the other cities once, and returns to the start city. We might also be interested in the shortest tour and its total distance.

We could solve that problem by removing the start city from the list. We obtain each permutation of the resulting `List<City>` as in the sample code. The body of the iterator loop could "reattach" the start city at the head and tail of the list to see if the tour is the shortest one discovered so far.

Many papers have been written about the TSP, and no efficient technique is known for finding the very best tour when the number of cities becomes large. However, we will see a way to obtain a "pretty good" solution when we discuss Simulated Annealing in a later chapter.

MACHINE SEQUENCING

Suppose we have a factory with one machine and a list of jobs that must run on the machine. If adjacent jobs are compatible (in some way) the machine need not be setup between jobs. If not, the setup time will depend on the particular aspects of the adjacent jobs. We want to find a schedule (a `List<Job>`) that will minimize the total time taken to process all the jobs.

We could solve that problem by getting all the permutations of the job list. As each is returned by the iterator, we can calculate its runtime and save the shortest.

Surely, that is an inefficient way to solve the problem. Instead, we can group the compatible jobs together. Then we can do the permutations on the groups (instead of the jobs) to find the best sequence for the groups. Perhaps further details of the problem will suggest that after the groups are scheduled, permutations or sorts within groups might need to be examined to refine the schedule.

8-QUEENS

We have a standard chessboard (8 rows, 8 columns) and wish to place 8 queens on it so that no two queens attack each other. I.E. no two queens are on the same row, or on the same column, or on the same diagonal.

To solve that problem, consider the permutations of the list **[1, 2, 3, 4, 5, 6, 7, 8].** The position in the list will represent the row a queen is on, the value of the element will represent the column the queen is in. Since we have 8 queens we know that exactly one must appear in each row, and exactly one must appear in each column. The particular list we gave means there is a queen in row **x**, column **x**, where **x** takes on the values 1-8. All of these queens are on the main diagonal (from the top left-hand corner (row 1, column 1) to the bottom right-hand corner (row 8, column 8)). Hence it does not represent a solution.

The list **[2, 1, 3, 4, 5, 6, 7, 8]** has the same configuration, except that row 1 now has a queen in column 2 whereas row 2 has a queen in column 1.

If you study the situation, you will see that a list of permutations of the original list will contain (somewhere) all valid solutions to the 8-Queens problem. No solution can be devised that is not somewhere in that list. Furthermore, the problem *representation* has solved part of the problem for us before we start. This is because it is incapable of showing two queens on the same row or in the same column.

To work out the problem, all we need do is code up a permutation iterator on **[1, 2, 3, 4, 5, 6, 7, 8]** and examine the lists returned, checking each one to see if it is a valid placement of non-attacking queens.

Lexigraphical Order and "Cut"

The number of permutations of a list of 8 objects is 8 factorial (written *8!*). That equates to 8*7*6*5*4*3*2*1, or 40,320. Most of these will not be valid 8-Queens solutions (in fact, there are only 92 solutions). It would be nice if we could easily eliminate ones that are obviously invalid. For example, any permutation that begins **[1, 2…]** is clearly invalid since the queen on row 1 attacks the one on row 2 (along the diagonal). No placement of queens on subsequent rows can fix this situation.

We can tell the iterator to skip over permutations that have the same prefix, starting at element 0 through element x, by invoking `Permute.cut(x)`. Our code would look like this:

```
foreach (List<int> ans in permute.permutations())
{
    int cutAt = valid(ans);
    if (cutAt >= 0)
        permute.cut(cutAt);
}
```

The method `valid` would return a negative integer if the configuration were valid. It would examine the rows in order, checking for a queen that attacked a queen on a previous row. If it found one, it would return the last row number it examined. The queen on that row represents an invalid configuration, and no placement on subsequent rows need be examined; all will fail.

To be concrete, if the last permutation returned was **[1, 3, 4,…]** and the queen on row 3 (it is in column 4) was the first attacking queen discovered, we would call `cut(2)` (we are 0 based when indexing a List). Then the next permutation returned by the iterator would begin **[1, 3, 5, ….]**.

This will reduce the number of permutations examined in 8-Queens from 40,320 to 2056. The cut changes our enumeration (presentation of all members of a set) into a search.

The `Permute` class generates permutations in "lexigraphical" order. If we had a list of 5 objects and numbered them based on their index in the list, the permutations returned by Permute would be in the following order:

0 1 2 3 4

0 1 2 4 3

0 1 3 2 4

0 1 3 4 2

0 1 4 2 3

0 1 4 3 2

0 2 1 3 4

0 2 1 4 3

0 2 3 1 4

0 2 3 4 1

0 2 4 1 3

0 2 4 3 1

0 3 1 2 4

0 3 1 4 2

0 3 2 1 4

0 3 2 4 1

…….

This is called lexigraphical order because if we sorted the list as if the members were strings, they would be in "dictionary" order. The ordering is

based on the index of an element in the original list. You will see that the original index of the last element of the list of permutations varies the fastest, the first element the slowest. It is because the permutations are generated in this order that we can apply `cut(x)` to eliminate all permutations with the same prefix.

This is a natural order for most permutations problems, and you will find the `cut` method useful to reduce the search space.

QUADRATIC ASSIGNMENT PROBLEM

The 8-queens problem has a non-numeric criterion for success. Often, as in TSP, our permutation problem will attach a number to each configuration, and we seek an optimization over all permutations. For example, suppose we had 8 locations and 8 factories with material flows between the factories. The cost of moving shipments between factories at locations **i** and **j** would depend on the distance between the locations and the number of shipments per month. It could be given by a method in class `Factory`,

```
int cost(Factory otherFactory, int i, int j)
```

Where `i` and `j` are the locations of *this* `Factory`, and `otherFactory` respectively. The cost method takes into account the shipments per month between the two factories and the distance between the locations.

Our problem representation is very like 8-queens. The list `List<Factory>`, [f,g,…] would mean that `factory` **f** (the first member of the list) is placed at location 0, factory **g** at location 1, etc. Here is the code to search out the best locations for our factories:

```
            Permute<Factory> permute = new Permute<Factory>(Factory.allFactories);

            int bestCost = -1;
            List<Factory> bestList = null;

            foreach (List<Factory> factoryList in permute.permutations())
            {
                int totCost = 0;
                for (int i = 0; i < factoryList.Count-1; i++)
                    for (int j = i + 1; j < factoryList.Count; j++)
                    {
                        totCost += factoryList[i].cost(
                            factoryList[j], i, j);
```

```
                    }
                if (bestList == null || bestCost > totCost)
                {
                    bestList = factoryList;
                    bestCost = totCost;
                }
            }
```

A minor optimization is available by jumping out of the double loop as soon as `totCost` exceeds `bestCost`.

This problem is called a *Quadratic Assignment Problem* because the "objective" function, `cost`, is quadratic (it involves the product of the distance between factories and the number of shipments between them). The QAP is a hard problem to solve exactly and many papers have been written about it. Note that (unlike 8-queens) the cut does not seem to be of any use in trimming the search space.

Summary of the Permutation Design Pattern

If you can visualize your application as a search through all orderings of a list you can, perhaps, use the Permutation design pattern. You will need to make up the initial list and write code (in the `Permute.permutations` iterator loop) that examines each permutation for validity or optimality.

The `Permute` class does not hold a reference to the individual permutations. If the application does not keep a reference to each of them, space (i.e. memory) should not be a problem. However, unless the `cut` method can be used, time (i.e. processor time) could be a problem. As the list increases in size, the number of permutations generated goes up faster than exponentially (10! ~= exp(15); 20! ~= exp(42)). Therefore, the Permutation design pattern is not likely to be useful for large problems. For example, it is a non-starter for TSP with more than a few cities or for QAP with more than a few factories.

You will find more useful solutions for what appear to be permutation problems in subsequent chapters. However, for small problems this pattern is very straight-forward and works well.

2 *Combinations and Cartesian Product*

COMBINATIONS AND CARTESIAN products are usually defined in terms of sets and subsets. Neither the C# language nor its library contains direct support for sets. However, the class `List<>` is very useful and we will use it instead. Remember that in a list (unlike in a set), the elements have an index (their zero-based position in the list), and that the same element can appear more than once in a list.

A *combination* drawn from a list is a sublist whose elements are in the same order as in the list. For example, **[1,3]** is a sublist of **[1,2,a,m,3].**

An *n-combination* (*n* is an integer) of list X is simply a sublist of X that contains n elements. We allow a 0-combination of any list; this is a list whose Count is zero (the "empty list"). We also allow the entire list of X to be a combination of X. The number of k-combinations taken from a list of n elements is n!/[(n-k)!*k!]. Because elements can be repeated in a list (but not in a set), there is some confusion as to when two combinations are the same and should be counted only once. We will assume that an element is implicitly tagged with its original index, making all elements distinct and thus allowing the above formula to work for combinations drawn from a list.

The *Cartesian product* of a list of lists, M, is another list of lists, N, where each list in N contains one element from each of the lists in M. The order of the lists in N is derived from the order of those in M. For example, if we have **[a,b],[1,2],[A,B,C]** *some* of the elements in the Cartesian product of these 3 lists are **[a,1,A], [a,1,B], [a,1,C]…** All of the elements in the Cartesian product have 3 objects (because we are taking the Cartesian product of 3 lists). The number of lists in the Cartesian product is found by

taking the product of the Counts of the original lists. So in the above example, the number of lists in the Cartesian product is 2 * 2* 3, or 12.

Combinations

If we have a list, `ints`, of items, say integers **[1, 2, 3, 4, 5]**, we can obtain all of the 3-combinations from `ints`, one at a time, with the following code:

```
Combine<int> combine = new Combine<int>(ints, 3);
foreach (List<int> ans in combine.combinations())
{
        //do something with the list ans
}
```

The lists returned in the `foreach` loop are:

1 2 3

1 2 4

1 2 5

1 3 4

1 3 5

1 4 5

2 3 4

2 3 5

2 4 5

3 4 5

You can see that support in SEL for combinations is very similar to that for permutations. We need only construct the list from which we wish to draw the combinations, make a `Combine` object (the second parameter to the constructor is the size of the combinations), and invoke its iterator.

We turn now to some applications of combinations.

OBTAINING ALL SUBLISTS OF A LIST

Since we can obtain all sublists of length n, it is quite easy to obtain all sublists of a list.

```
List<List<int>> sublists = new List<List<int>>(32);
for (int i = 0; i <= ints.Count; i++)
{
    Combine<int> combine = new Combine<int>(ints, i);
    foreach (List<int> ans in combine.combinations())
    {
                sublists.Add(ans);
    }
}
```

There will be 2^n sublists if the original list has `ints.Count` = n.

CREATING A ROUND-ROBIN TOURNAMENT

Suppose we have 10 players and wish to schedule two-player matches, so that every player plays the others once. This is just a list of 2-combinations:

```
List<Player> players = Player.allPlayers();
Combine<Player> combine = new Combine<Player>(players,
2);
foreach (List<Player> c in combine.combinations())
{
    //do something with the combination c
}
```

NESTED PARENTHESES

Suppose we wish to find all ways to place k right parentheses and k left parentheses in a list so that they are balanced. The list is balanced if, as we scan the list from left to right, we never have more right parentheses than left ones. For example, "(()())" is balanced whereas "(()))(" is not, even though both have 3 left parentheses and 3 right ones. A clever, but not very efficient, way of generating the lists is to find all ways to place the left parentheses, fill the empty spaces with right ones, and then test to see if the list is balanced. In the code below, `sizeResult` is the desired length of our list (and hence must be an even number).

```
if (sizeResult % 2 == 1)
    sizeResult++;

List<int> subs = new List<int>(sizeResult);
```

```
5               for (int i = 0; i < sizeResult; i++)
6                   subs.Add(i);
7
8               int qty = 0;
9
10              List<List<char>> valid = new List<List<char>>(10);
11
12              Combine<int> combine = new Combine<int>(subs,
    sizeResult/2);
13
14              foreach (List<int> ans in combine.combinations())
15              {
16                  qty++;
17                  List<Char> trial = new List<Char>(sizeResult);
18                  for (int i = 0; i < sizeResult; i++)
19                      trial.Add(')');
20
21                  foreach (int j in ans)
22                  {
23                      trial[j] = '(';
24                  }
25
26                  int balance = 0;
27                  bool noGood = false;
28
29                  for (int i = 0; i < trial.Count; i++)
30                  {
31                      if (trial[i] == '(')
32                          balance ++;
33                      else balance --;
34                      if (balance < 0)
35                      {
36                          noGood = true;
37                          break;
38                      }
39                  }
40                  if (noGood)
41                      continue;
42
43                  valid.Add(trial);
44              }
```

Lines 1-2 insure that we have an even number of elements in our list of parentheses. In lines 4-6 we get a list of subscripts. At line 12 we set up to get combinations of these so as to obtain half the available subscripts in each combination. For each combination, we fill in our list (`trial`) with right parentheses (lines 18-19), and then use the combination to replace half of these with left ones (lines 21-24). We then test to see if the list is balanced, and if so, we add it to the list of valid parentheses (line 43).

If `sizeResult` is 6, we get 5 lists in `valid`:

((( )))

(( ) ( ))

(( )) ( )

( ) (( ))

( ) ( ) ( )

There were 20 total combinations generated. So, there were many useless combinations generated. In fact, only 1/(n+1) of the lists generated will be valid, where n is the number of left (or right) parentheses. We will see another way of generating balanced parentheses when we discuss Divide and Conquer in a later chapter.

COMBINATIONS OF COMBINATIONS

The "configuration problem" occurs when we have a product with different options, and we wish to list all of the possible variations of the product. If there is only one alternative to be drawn from each option set, we can solve the problem with the Cartesian product of the option sets. If we can select a combination of alternatives for each option we need to list a combination of combinations.

For example, suppose our company makes a line of sweaters. For each sweater, the customer can pick 2 of 3 colors from **[red, green, blue]**, 1 of 3 patterns from **[check, plaid, stripe]**, and a blend of 2 of 3 yarns from **[wool, poly, cotton]**. Hence one permissible sweater configuration is: **[red, blue, plaid, poly, cotton]**, which means this sweater's colors are red and blue, its pattern is plaid, and it is made from a blend of poly and cotton.

A C# solution to list the possible sweaters is:

```
1               List<List<string>> selections = new
    List<List<string>>(10);
2
3               Combine<string> combineColor = new
    Combine<string>(colors, 2);
4
5               foreach (List<string> col in
    combineColor.combinations())
6               {
```

```
7                   Combine<string> combinePattern = new
    Combine<string>(patterns, 1);
8                   foreach (List<string> pat in
    combinePattern.combinations())
9                   {
10                      Combine<string> combineMaterial = new
11                                      Combine<string>(materials, 2);
12                      foreach (List<string> mat in
    combineMaterial.combinations())
13                      {
14
15                          List<String> sel = new List<String>(3);
16                          sel.AddRange(col);
17                          sel.AddRange(pat);
18                          sel.AddRange(mat);
19                          selections.Add(sel);
20                      }
21                  }
22              }
```

The first few configurations generated from the above code are:

red green check wool poly

red green check wool cotton

red green check poly cotton

red green plaid wool poly

red green plaid wool cotton

red green plaid poly cotton

red green stripe wool poly

red green stripe wool cotton

red green stripe poly cotton

red blue check wool poly

.....

The colors, patterns, and materials are assumed to be gathered (as strings) in the corresponding lists, `colors`, `patterns`, `materials`. We have 3 nested loops, enumerating the combinations of each of the options. In the inner loop, at lines 15-18, we "flatten" the lists so as to put the strings

representing a single configuration into a list of strings (`sel`), which we put in the list of all configurations (`selections`) at line 19.

This solution is straight-forward, but is not very general since the number of options is "hard coded". If we add a fourth option, we have to nest another loop to range through its combinations.

A more general solution obtains the combinations for each option, puts them in a list, and then takes the Cartesian product of the combinations to get each configuration. The following code does that:

```
1               List<List<string>> allLists = new
    List<List<string>>(3);
2               allLists.Add(colors);
3               allLists.Add(patterns);
4               allLists.Add(materials);
5
6               List<int> atATime = new List<int>(3);
7               atATime.Add(2);
8               atATime.Add(1);
9               atATime.Add(2);
10
11              selections.Clear();
12
13              List<List<List<string>>> allCombs = new
    List<List<List<string>>>(3);
14
15              for (int i = 0; i < atATime.Count; i++ )
16              {
17                  List<List<String>> collectComb = new
    List<List<String>>(3);
18                  Combine<string> combine = new Combine<string>(
19                      allLists[i], atATime[i]);
20                  foreach (List<string> c in combine.combinations())
21                  {
22                      collectComb.Add(c);
23                  }
24
25                  allCombs.Add(collectComb);
26              }
27
28              Cartesian<List<string>>sweaters =
29                      new Cartesian<List<string>>(allCombs);
30
31              foreach (List<List<string>> sweat in
    sweaters.cartesian())
32              {
33                  List<string> flatten = new List<string>(3);
34                  foreach (List<string> config in sweat)
35                  {
```

```
36                      flatten.AddRange(config);
37                  }
38
39                  selections.Add(flatten);
40              }
41          }
```

At lines 1-4 we gather all of the options into a `List<List<string>>, allLists`. The number of elements we can select from each option is gathered in `atATime`, and will be used to form the combinations of the options (lines 6-9).

The rest of the code is completely independent of the number of options in the configuration. We accumulate the permissible combinations of each option in lines 13-26. The combinations for each option is a `List<List<string>>`, and so the collection of these, gathered in `allCombs`, must be a `List<List<List<string>>>`. This list looks like:

[[[red, green], [red, blue], …], [[check], [plaid], [stripe]], [[wool, poly], [wool, cotton], …]]]

In lines 28-31, we take the Cartesian product of that list, in order to select one alternative for each option. One of those would look like:

[[red, green], [check], [wool, poly]]

Lines 33-38 "flatten" that list of lists, so that we get a list of strings, which we can put into the collection of all configurations, at line 39.

This code is a little obscure, but it shows how combinations of combinations can be handled in a general way, with the help of Cartesian product.

Summary of the Combination Design Pattern

Like the permutation pattern, the Combination design pattern searches through an object space of lists. If your problem is to find alternative combinations drawn from a list, you can use this pattern. If you need to make selections from multiple lists you can consider using a Cartesian product of the lists.

As with the permutation pattern, the individual permutations are not held in memory by the SEL so, unless the application holds references to them, memory use is minimal.

As the size of the list increases, the number of combinations can increase very rapidly. Thus this pattern is suitable for small problems only.

3 *Depth First Search*

IN THIS CHAPTER we will investigate a technique for exploring a graph, called Depth First Search. This is a very useful design pattern that has an astonishing range of applications. We will present some of these applications in this chapter, and in subsequent chapters.

8-QUEENS REVISITED

We saw in Chapter 1 how the Permutations pattern could solve the 8-Queens problem. However, that solution depended on a clever representation and was not likely to be the first attack one would make on the problem. Let's develop a more natural solution by listening in to a designer as she wrestles with the problem.

Let's see. I have to put 8 queens on this 8 by 8 board so that no two are on the same row, column, or diagonal. I guess I will start by putting a queen on row one, then another on row two that does not attack the first, then another on row three that is still a valid partial solution, and proceed until I get to the last row.

row 1: I can put a queen in any column here, but I might just as well start with column 1. That takes care of row 1. On to row 2.

row 2: I cannot put a queen in column 1 since there is already a queen in that column in row 1. Column 2 is no good because the queen in row 1, column 1, could attack it along the diagonal. Column 3 looks good. Let's put a queen there. On to row 3.

row 3: I have a partial solution with queens on rows 1 and 2, and want to extend it to row 3. I'll check columns until I find a spot where this queen cannot be attacked.

……

Suppose she proceeds that way until she gets to row 8 with the set-up in Figure 3.1.

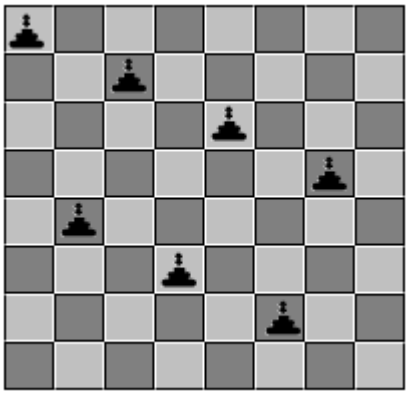

Figure 3.1: 8-Queens partial solution

row 8: Hmm, I cannot put a queen on row 8 that would not be attacked. The only column that does not have a queen in it is column 8, but a queen in that column on row 8 would be attacked diagonally by the queen on row 1. Some prior queen is going to have to be placed elsewhere. Which one? In order to save the work I have already done, I will backtrack to the last valid partial solution and see if I can move the last queen I placed.

row 7: No need to check columns 1-5 since I looked at them before I placed the queen in column 6 and they were no good. The queen I placed in column 6 cannot be moved to column 7 because that column has a queen in it on row 4. It could be moved from column 6 to column 8, but that would be attacked diagonally by the queen on row 2. So, row 7 cannot be fixed. I'll remove the queen on row 7 and try and fix row 6 with a different alternative.

row 6: I've tried columns 1-4 already (when I placed the queen on row 6 for the first time). I see that columns 5, 6, 7, and 8 are all attacked by some queen on rows 1-5. I'll have to give up on row 6 (remove its queen), and examine row 5 for another alternative.

The process our designer is employing is "depth-first search with backtracking". It turns out that many problems can be structured this way. If we analyze what is going on, we see two principle kinds of "moves": an extension of the partial solution to the next row, and, if that is not possible, backtracking to an alternative of the previous move. If we consider the moves to be nodes in a graph, the first type we might call "firstChild". The

name is meant to suggest that this node extends the partial solution for the first time, by linking the previous node (the parent) to this one. The second kind of move we will call "nextSibling", since we are seeking a node with the same parent as the last one that succeeded. It represents not an extension of the current solution, but an alternative to a node previously accepted. The graph in figure 3.2 will make these notions clear.

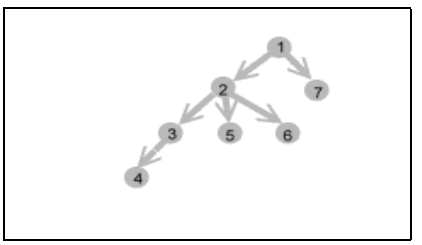

Figure 3-2: Depth First Graph

This is a piece of a depth-first graph showing the order in which the nodes are visited by DFS. We can take node 1 to be a queen on the first row. Next, we place a node on the second row, node 2. We call this the "firstChild" of node 1. Similarly, we create "firstChild" nodes as nodes 3 and 4. The arrows point to child nodes (those that extend the partial solution). If we find that we cannot extend the partial solution represented by nodes 1-2-3-4, we ask node 4 for its nextSibling. It does not have one (there are no other nodes that have node 3 as a parent), so we backtrack and ask node 3 for its "nextSibling". This is a node with the same parent as node 3 (i.e. node 2), and represents an alternative move for node 3. This is node 5 and is visited next. It is a queen on row 3 (because the "parent chain" above it represents a queen on row 2, and then a queen on row 1). The graph indicates that the partial solution 1-2-5 cannot be extended, so we ask node 5 for its "nextSibling", which is node 6, and we visit that next. The partial solution 1-2-6 cannot be extended and node 6 has no nextSibling, so we backtrack and ask node 2 for its nextSibling, which is node 7. At this point we have partial solution 1-7.

Depth First Search Classes

Let's implement 8-queens with some C# classes. Our nodes will be implemented with class `Queen`. It will hold the row and column the queen is on, along with a reference to the parent node (the queen on the previous row in our partial solution). We will also implement the methods `nextSibling` and `firstChild`.

```
public class Queen: IGNode<Queen>
    {
        public static int max = 8; //number of rows/cols in the chessboard
                                        // 8, for 8-Queens problem
        public int row = 1;  //that this queen is on
        public int col = 1;  //that this queen is on
        Queen theParent = null;  //queen on row just above
                                 //this one in the solution so far

        public Queen(int r, int c, Queen p)
        {
            row = r;
            col = c;
            parent = p;
        }

        public Queen nextSibling()
        {
            //make a valid queen on the same row, but in next col.

            Queen q = new Queen(row, 1, parent);
            for (q.col = this.col + 1; q.col <= max; q.col++)
            {
                if (q.isValid())
                    return q;
            }
            return null;
        }

        public Queen parent
        {
            get
            {
                return theParent;
            }
             set
            {
                theParent = value;
            }
        }
```

```

        public Queen firstChild()
        {
            if (row >= max)
                return null;

            //make a valid queen on the next row
            Queen q = new Queen(row + 1, 1, this);

            for (q.col = 1; q.col <= max; q.col++)
            {
                if (q.isValid())
                    return q;
            }
            return null;
        }
```

We see that the method `firstChild` extends the search by placing a queen on the next row, or returns null if that is not possible. The method `nextSibling` finds an alternative queen on the same row. Note that `nextSibling` remembers that previous columns have been tried, so tries the next column (i.e. earlier columns are alternatives that have already been tried). Remember that we used the cut in our Permutations solution to 8-Queens in Chapter 1? We accomplish the same thing by returning null in `nextSibling` and `firstChild` when we detect that a partial solution must be abandoned because its extension cannot possibly lead to a full solution.

Both `nextSibling` and `firstChild` make use of `isValid` to check to see that there is no queen on the same diagonal or column as this one. The `isValid` method does this by chasing the parent references to get the queens in the partial solution developed so far. That method looks like this:

```
public bool isValid()
        {
            Queen par = parent;

            //see if any queen on a row above this one could
            //capture this queen.
            while (par != null)
            {
                if (par.col == this.col)
                    return false;
```

```
                if (Math.Abs(par.row - row) == Math.Abs(par.col - col))
                    return false;  //diag capture
                par = par.parent;
            }
            return true;
        }
```

The class `Queen` implements the generic interface `IGNode<T>`, which wants an implementation of the `firstChild`, `nextSibling`, and `parent` methods. The interface definition is supplied in the SEL.

How does `Queen` solve the DFS problem for us? Here is a method (which we would put in another class, say: `SolveQueens`):

```
public void solveDFS()
        {
            Graph<Queen> queenGraph = new Graph<Queen>(new Queen(1, 1, null));

            //solve via depthFirst search
            foreach (Queen q in queenGraph.depthFirst())
            {
                nodesSearched++;

                if (q.row == Queen.max)
                {
                    makeSolution(q);
                }
            }
        }
```

The method makes use of the generic class `Graph<T>`, which has an iterator that will return the nodes visited in the depth first search. The queen node passed to the constructor of `Graph<Queen>` is a root, or "start" node. It is the first queen placed in the first partial solution we will examine. The generic class `Graph<T>`, including its iterator, is supplied by the SEL.

All the application need do in the body of the iterator loop is to detect a terminal node that represents a solution. We know that if a queen is placed in row 8 (`Queen.max`) we have a complete solution.

It is the responsibility of the `Graph<Queen>` iterator, `depthFirst`, to make calls to the appropriate nodes, asking for `firstChild`, `nextSibling`, and `parent` to accomplish the depth first search. When a method in the `IGNode<Queen>` returns `null`, the graph's iterator will backtrack appropriately. Each (non-null) node visited during the search is

returned by the iterator. We keep track of the total nodes returned (in `nodesSearched`) out of curiosity.

Figure 3.3 shows the action of the DFS during the 8-Queens program.

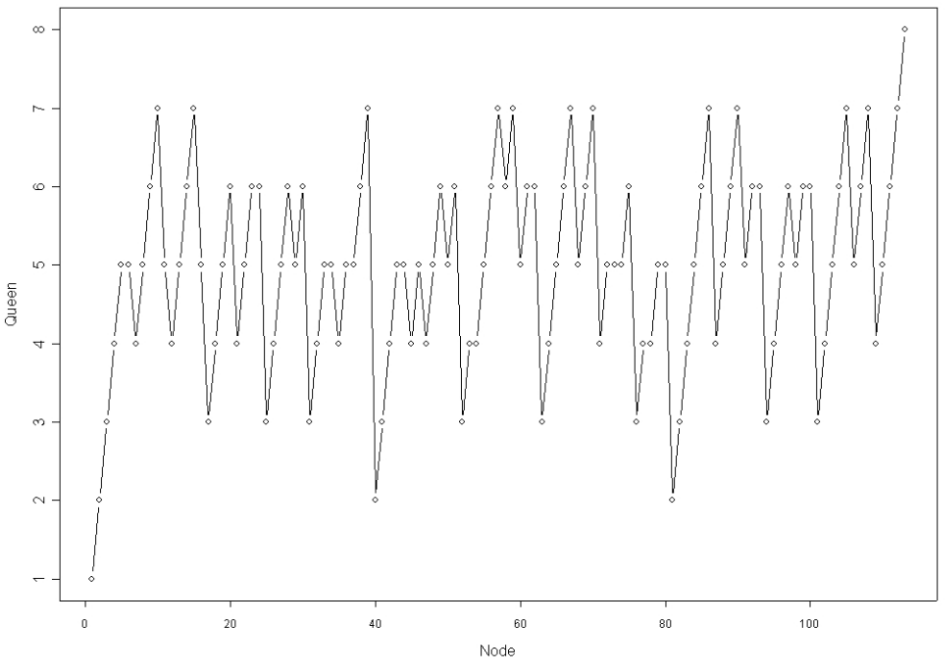

Figure 3-3: DFS during 8-Queens

The y-axis of figure 3.3 is the row the queen is on, the x-axis is the `nodeSearched` number maintained in the iterator loop. We can see that the search makes progress, placing a new queen on each row until it gets to row 5. The `firstChild` call fails and the search can go no deeper. It does a `nextSibling` call at that point, and finds a different placement for the queen on row 5, but at that point it can go no deeper; there are no more siblings, and it backtracks to place a different queen on row 4. You can see that at about node 40 (and also at node 80) it has to backtrack all the way to replace the queen on row 2. Finally, at node 109 it takes off from row 4 and gets to the last row at node 113, via `firstChild` nodes.

DFS Class Collaboration

Notice how the responsibilities are divided between the application and the generic `Graph<T>` class. The application designer writes the node class T, which inherits from `IGNode<T>`. Here is where all the application specific logic and data representation is kept. The `Graph<T>` does the DFS logic and bookkeeping; it is supplied by the class library SEL. Figure 3.4 shows the collaboration.

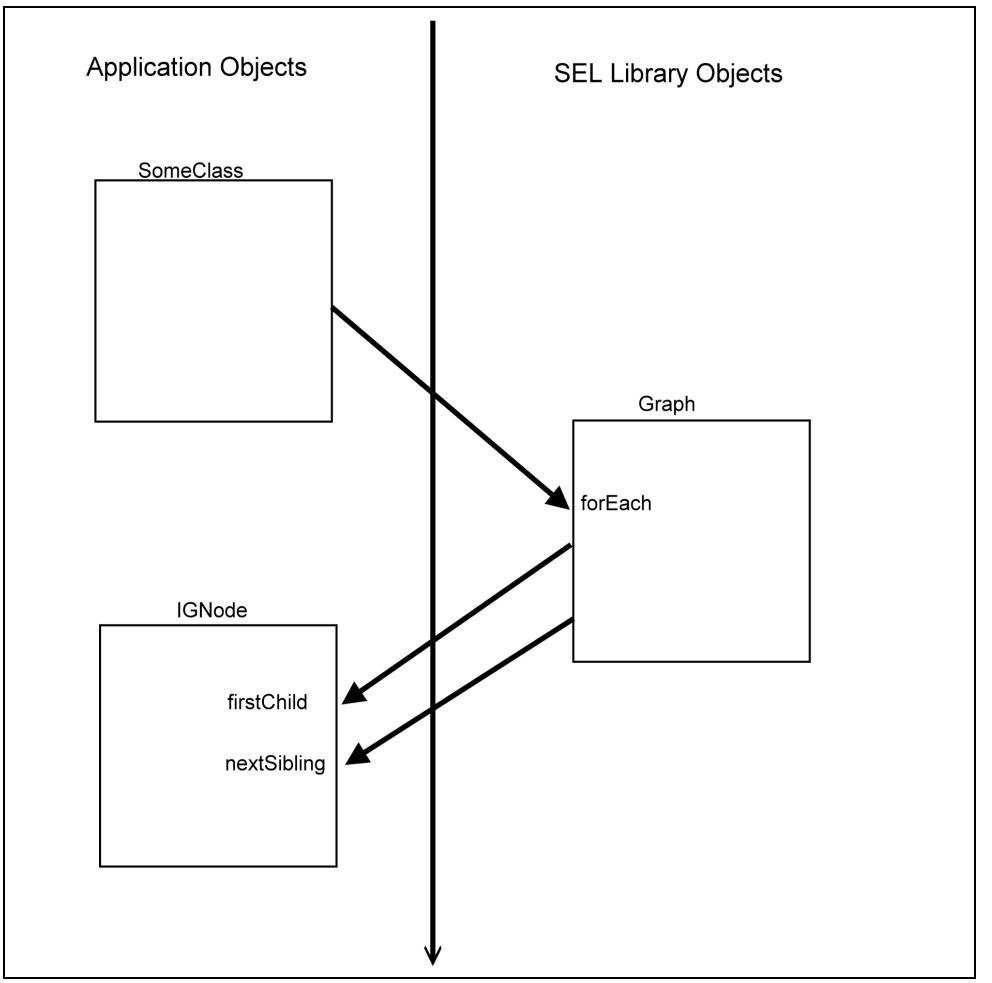

Figure 3-4: Collaboration between application and SEL

Some Graph Theory

Depth first search is a topic in a large field called "graph theory". We will not get into this too heavily, but we do need some definitions. Much of this will be intuitive and useful subsequently.

A *graph* consists of *nodes* and *edges*, represented by circles and lines respectively. An edge is a line drawn between two nodes. We will assume that two given nodes are never connected by more than one edge. An edge connecting a node to itself is also outlawed. A *path* is just a sequence of nodes, as in [a, b, c, d], where the graph contains an edge between the adjacent nodes. A graph is said to be *connected* if there is a path between any two nodes.

To continue our graph terminology in the context of our `Graph` class: when `Graph` calls `firstChild` or `nextSibling`, the node returned by either of these functions is said to be *visited* by the DFS. When `x.firstChild` returns a node, `x` is said to be *expanded*. Note that this call to `firstChild` establishes the parent-child relationship between `x` and the node the call returned. In theory, a different search could reverse the relationship. The rest of the children of a node are established by the DFS calling `nextSibling` against the `firstChild` node, and then continuing to call it against the siblings returned. If all the children of a node have been visited, that node is said to be *explored*, or (*fully expanded*). The children of a node are also called *successors* of the node. A node with no parent is a *root* node; one with no children is a *leaf* node.

Here's an example of the above terminology, commenting some C# code :

```
1           List<MyNodeClass> allSuccessors(MyNodeClass parent)
2           {
3               List<MyNodeClass> successors = new
    List<MyNodeClass>(3);
4               MyNodeClass node = parent.firstChild();   //visit the
    firstChild,
5                                                   //and expand parent
6               while (node != null)
7               {
8                   successors.Add(node);
9                   node = node.nextSibling();  //visit another sibling
10              }
11
12              return successors;  //we have fully expanded (explored)
    parent
```

```
        }
```

In the above code, `MyNodeClass` is assumed to inherit from `IGNode<MyNodeClass>`.

Chains in DFS

If we were to call `nextSibling` against a node, and then against the node returned, and so forth, we would obtain a list of nodes we will call the *nextSibling* chain.

Similarly, if we were to call firstChild against a node, and continue against the node returned, we would have a list of nodes forming the *firstChild* chain.

If we reverse the firstChild chain, going from a terminal node to the root of the graph, we would have a *parent* chain. This is obtainable by calling `parent` against a node, and the node returned by the call, etc.

We will refer to these chains in subsequent discussions. It is important to realize that only the parent chain is really manifested (via the parent reference in each node). The other chains are conceptual; we do *not* keep the references that would be needed to support them, in either the SEL or the application.

SEARCHING A MAZE

All this is pretty abstract. Graphs do not have much use until we give meaning to the nodes, and interpret the edges as relationships between nodes.

Hence, the difficult part of applying the DFS pattern is analyzing the problem and recognizing that DFS applies. To help you with that, we will work another problem. This one looks more like a graph at the outset, but it has a wrinkle that we glossed over in the 8-queens puzzle.

Let's represent a maze as a system of caves and passages, and associate a graph with it. We will use this to figure out how to write a program that can let us escape from the cave. Figure 3.5 shows our maze of caves.

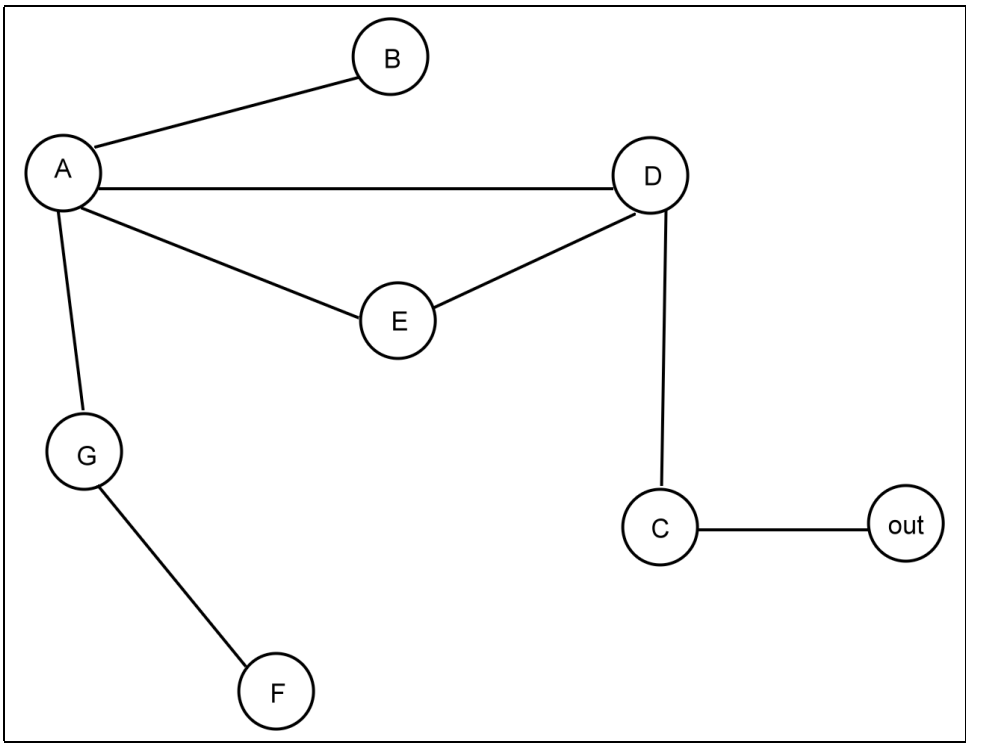

Figure 3-5. Maze of Caves

Figure 3.5 is a graph. The nodes are caves; the edges are tunnels between caves. We are in cave A and we seek a path that will get us out (the only way out is through cave C). This graph has a *cycle* (a path that connects a node to itself), [A, D, E, A]. A connected graph with no cycles is said to be a *tree*.

In general, edges do not have an implied direction. If our application assigns a direction to an edge, the edge is said to be *directed*, and we will indicate that with an arrow instead of just a line.

DFS works on any connected graph to construct a directed tree. That tree *spans* the original graph (includes all the nodes in the original graph), and contains a subset of the edges. In our maze, DFS will eliminate either edge AE, edge AD, or edge ED to make the spanning tree. If we number the nodes in the tree as they are visited by DFS, the direction of an edge is from the node with the smaller number to the one with the larger number.

The wrinkle we glossed over in 8-queens is that it is the *application's* responsibility to eliminate cycles (our 8-queens solution did that implicitly, see below). It will usually do this in the `IGNode<T>` implementation. Let's execute DFS on our caves and show how to eliminate cycles.

```
public class Cave: IGNode<Cave>
    {
        public string name;
        public bool visited = false;
        public Cave aParent = null;
        public List<Cave> adjCaves = new List<Cave>(10);
        public bool way_out = false;

        public Cave(string nme)
        {
            name = nme;
        }

        public Cave nextSibling()
        {
            if (parent == null)
                return null;

            //look at caves with the same parent as this one
            foreach (Cave cave in parent.adjCaves)
            {
                if (!cave.visited)
                {
                    cave.parent = this.parent;
                    cave.visited = true;
                    return cave;
                }
            }
            return null;
        }

        public Cave parent
        {
            get
            {
                return aParent;
            }
            set
            {
                aParent = value;
            }
        }

        public Cave firstChild()
        {
```

```
                //look at caves adjacent to this one
            foreach (Cave cave in adjCaves)
            {
                if (!cave.visited)
                {
                    cave.parent = this;
                    cave.visited = true;
                    return cave;
                }
            }
            return null;
        }
    }
```

Each `Cave` contains a list of adjacent caves. The boolean, `way_out`, will be set to `true` if the cave is an exit. The `name` corresponds to that in the figure. The boolean, `visited`, is set when we deliver the node to the calling `Graph` during the DFS. We will assume that the caves and the adjacency lists have been set up by some external application according to the graph given in the figure. Note that if caves A and B are adjacent (connected by a tunnel), then each will have the other in its adjacency list.

The method `nextSibling` gets the `parent` and looks at the adjacent nodes under it, returning the first cave not yet visited. The method `firstChild` is similar, except it examines the adjacent caves of the current cave. Both methods set the `parent` field of the cave returned. Note that of the adjacent caves of a node, one will be returned as a `firstChild` — the rest as `nextSibling`.

To do the DFS, we execute the following code (in some application class):

```
public void solveDFS(Cave start)
        {
            start.visited = true;
            Graph<Cave> caveGraph = new Graph<Cave>(start);
                //solve via depthFirst search
            foreach (Cave c in caveGraph.depthFirst())
            {
                nodesSearched++;
                if (c.way_out)
                {
                    makeSolution(c);
                }
            }
        }
```

This method is called with the starting cave, which is passed to a `Graph<Cave>`. We have seen the operation of the DFS iterator in 8-Queens and its operation is the same here. We detect a solution node by seeing if it is a `way_out`. The path to the way out is given by chasing the `parent` references from the `way_out` node.

When we run the application on the graph above, it visits the nodes in the following order: A, B, D, C, Out, E, G, F. Note that it was a matter of luck that we found the shortest path out. If we had arranged the adjacent nodes in the collection for node A differently, we would have found the route A, E, D, C, Out. The spanning tree created by DFS on the Caves graph is in figure 3.6.

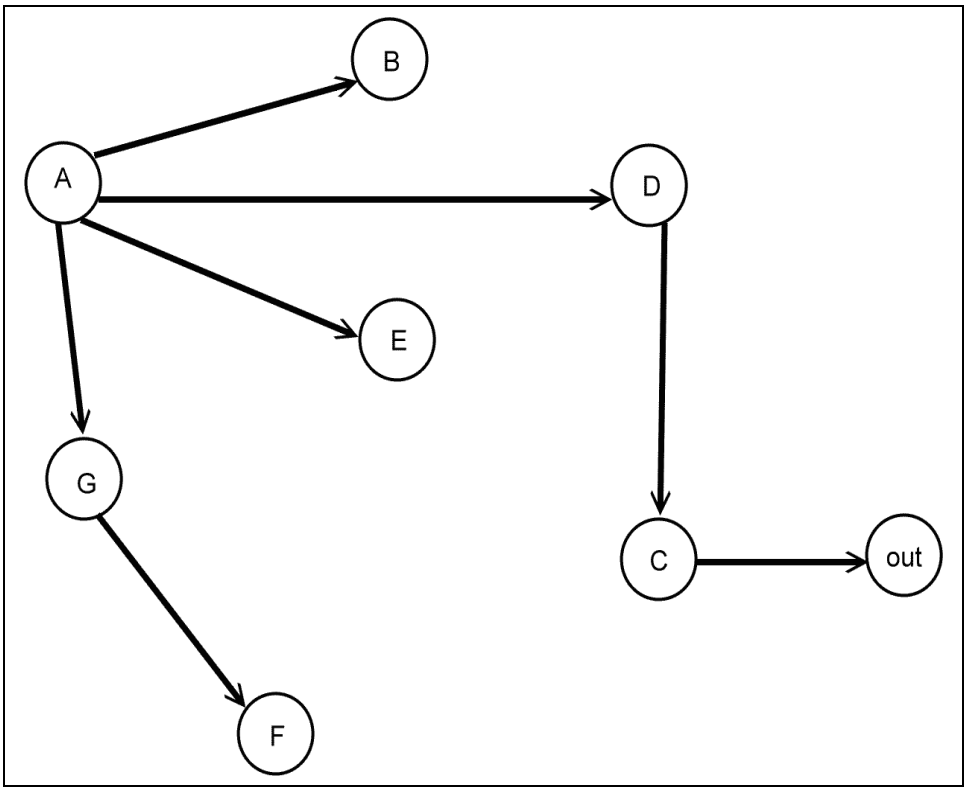

Figure 3-6. Spanning Tree Created by DFS on Maze of Caves

Application Analysis

The 8-Queens problem could be construed as a graph. Each node is a Queen with a column and row position. There are 64 nodes (one for each square on the board). There are edges from every queen on row 1 to every queen on row 2. There are edges from every queen on row n to every queen on row n +1. A complete solution is represented by a path with an edge from some queen (node) on row 1, to one on row 2, an edge from that queen to one on

row 3, etc, until we get to a queen on row 8. A partial solution is represented by a path starting on row 1 and continuing to some row short of row 8.

In Caves, the graph was explicit, consisting of the caves and the passages that connect them. A solution is a path that begins from the starting cave and finds its way to the exit. A partial solution is a path that begins at the starting cave, but stops short of the exit.

In 8-Queens we generated the nodes as they were needed. As we explored the solution space, the same node might be reconstructed, possibly with a different parent. The C# garbage collector reclaimed storage for nodes that were no longer needed. The maximum number of nodes that had to be held in memory was 8 (one for a queen on each row in the current partial solution). Interestingly, the `nextSibling` method did not have to access data in the `parent` to construct a node; all information needed was in the current node.

In Caves, the entire graph was in memory at the start. None were removed. In order to figure out the `nextSibling` we did have to access the `parent`, since it held the other children of the `parent` of the current node.

In 8-Queens all queens on row 1 had no parent. In that sense they were all roots. However, we can still call `nextSibling` against a queen on row 1; it was expected to return the queen in the next column of row 1 (you can think of them as having `null` as the common parent). The DFS graph that results is a *forest*, because the trees that are rooted in row one do not have a common root.

In Caves, the DFS generated a single spanning tree, starting from the single root (Cave A).

The most important difference between 8-Queens and Caves is the way they avoided cycles (looping) in the DFS. As mentioned above, it is the responsibility of the methods implemented in the `IGNode<T>` interface to keep DFS from an infinite loop.

We do this by insuring that the `firstChild` chain never has a cycle in it, within the same partial solution. Remember a partial solution is a path (held as the parent chain) from a starting node to the current node. When the current node is asked for `firstChild`, it *must not* return a node already in

that path. Similarly, when we ask the current node for its `nextSibling`, it must not return a sibling node that was previously returned, under the same parent, in the same partial solution.

Our 8-Queens solution avoided cycles by always advancing to the next row to get the `firstChild`, and always advancing the column to get the `nextSibling`. Because it never went backwards, there was never a loop created. The Caves solution was even simpler: it never returned the same node twice, even across the `nextSibling` and `firstChild` methods, or across partial solutions.

Note that a simple marking would not have worked with 8-Queens. In order to return *all* solutions in the DFS we will need to revisit nodes. What's more, because we recreated (constructed) the nodes as we went along, we would not have preserved any marks kept as member data anyway.

Caves did not find all solutions, and any path was acceptable so long as it followed edges from the start to the finish. Once a cave was entered, it did not matter how we got there. A previous cave on the path could not invalidate a solution that proceeded from the current cave to the exit. In 8-Queens, a prior node could have been invalidated when a partial solution failed. But that node could be reused (along a different `firstChild` chain) during backtracking.

You should study these two problems and understand how DFS can solve both, but that the requirements on the implementation of `IGNode<T>` differ between the two solutions.

Some books on graph theory define DFS as marking nodes when they are visited so that they are never visited again. This definition of DFS is guaranteed to visit all nodes in any connected graph, and to discover exactly one path from the root to each node. At this point, you should be able to see the limitations of that technique if we tried to apply it to 8-queens.

Here is an exercise for you to consider: how would you modify the Caves code so that you could explore *all* paths from the start to the exit?

A PARSER

Our first two problems in DFS, 8-Queens and Caves were pretty simple and not very useful (except, perhaps, to game programmers). Let's turn to a more complex use of DFS: building a parser.

The theory and construction of parsers has a large literature and we can only discuss the very basics. The purpose of a parser is to determine if an input string conforms to (matches) a set of rules (sometimes called *productions*). If it does conform, the parser builds a structure, called a *parse tree*. This is then used to perform a useful operation. For example, if we are building a calculator, then the parser would see if the input was a well-formed arithmetic expression, and build a structure that the calculator could evaluate. The set of rules is called a *grammar*. Here's an example:

0. expr → unit

1. expr → unit oper expr

2. unit → number

3. unit → (expr)

You can see that these rules are defined recursively. They would parse such input strings as

3

(3)

(3+4)

(3+4) * (7 + 9)

Take the time to determine how the rules recognize each of the above as **expr**'s. For example: **3** is an **expr** because: an **expr** is a **unit** and a **unit** is a **number** (and, implicitly, **3** is a **number**).

Our grammar would disallow (fail to parse) such strings as

(

3)

(3(

3 +/ 4

The left hand side of a rule is called its *head*; the right hand side is its *tail*. The symbols in the tail are called *goals*. If a given goal does not appear as a head (i.e. on the left hand side) of any rule, it is called a *terminal*. If it does appear as a rule's head, it is a *non-terminal*. The terminals in our grammar are **oper** (meant to stand for the operations of arithmetic: (**+, -, /, ***), the left and right parenthesis (**(,)**), and **number** (which stands for any number). The non-terminals are **expr** (meaning an expression) and **unit** (a sub-expression). In a grammar, the terminals are expected to be found in the input string, while the non-terminals are structures (sequences of non-terminals and terminals). Ultimately, the non-terminals will be built-up from the terminals. The grammar contains a special non-terminal that is to characterize the entire input string, called the *start symbol*. In our case, we are trying to recognize the entire input string as an **expr**, so that is our start symbol.

If a goal in a grammar rule is a non-terminal, we will try and match the head of some rule against it. If one matches, the goal is said to be *resolved* by the rule. We begin with the start symbol, find a rule that resolves it, and then examine the list of tail-goals of that rule. For each non-terminal therein, we find a rule whose head matches that goal. That leads to a new set of goals (the tails of the rules that resolved the goals of the rule that resolved the start symbol), and so forth. At any point of resolution there might be many rules that resolve the goal. In our grammar above, both rules 0 and 1 could resolve the goal **expr**.

We are going to use DFS to make a "recursive descent parser", or RDP. The "descent" part of it means that it performs a DFS. The "recursive" part of it usually means that the RDP is constructed by writing a separate procedure for each non-terminal, **N**. This procedure has embedded in it the rules that resolve **N**. For each, it calls the procedures that represent the goals of the rule. This results in a recursion since the head of a rule can often be found in the tail of some other (or the same) rule.

This kind of RDP is rather simple to construct if you don't mind "hard coding" the rules in this way. But then it will work only for a specific grammar; if the rules of the grammar change, then it will be hard to modify the parser.

Our RDP will take a set of rules as *input.* It does not embed them in the parser and will thus work for many grammars without recoding. It should be

mentioned, however, that only certain kinds of grammars can be parsed by any RDP. This will be explained later.

Parsers do not work with input strings directly, but expect the input to be "tokenized." This means the input is represented by a list of objects. Here is the class definition for these "token" objects:

```
public enum TOKEN {L_PAREN, R_PAREN, OPER, NUMBER, EXPR, UNIT};

    public class LexInput
    {
        public TOKEN token;
        public string text;

        public LexInput(TOKEN t, string txt)
        {
            token = t;
            text = txt;
        }
    }
```

Thus numbers are no longer represented by just a string of characters, but by the `LexInput` object that codes the type of terminal it is (`NUMBR`), and which contains the text that was in the input string. A `List<LexInput>` is thus a symbolic form of the input string, with all white space removed. In our grammar, the only tokens that will be found in the input (the terminal symbols) are `L_PAREN, R_PAREN, OPER,` and `NUMBER`.

Aside: The tokenizer program is called a "Lexical Analyzer". We do not consider its logic here, but it can be written easily using the C# library methods **String.Split**, **Int32.TryParse**, **Double.TryParse**, and the **Regex** class. It would be called by the parser to scan the input and deliver the LexInput objects.

Our grammar (list of rules) will contain objects that look like:

```
public class Rule
    {
        public TOKEN head;
        public List<TOKEN> tail;

        public Rule(TOKEN t)
        {
            head = t;
            tail = new List<TOKEN>(10);
        }
    }
```

So, our problem is: given a list of rules and a list of input tokens, determine if the latter conforms to the former and build a parse tree. Figure 3.7 shows a parse tree for the expression **(3 + 4) * 4**.

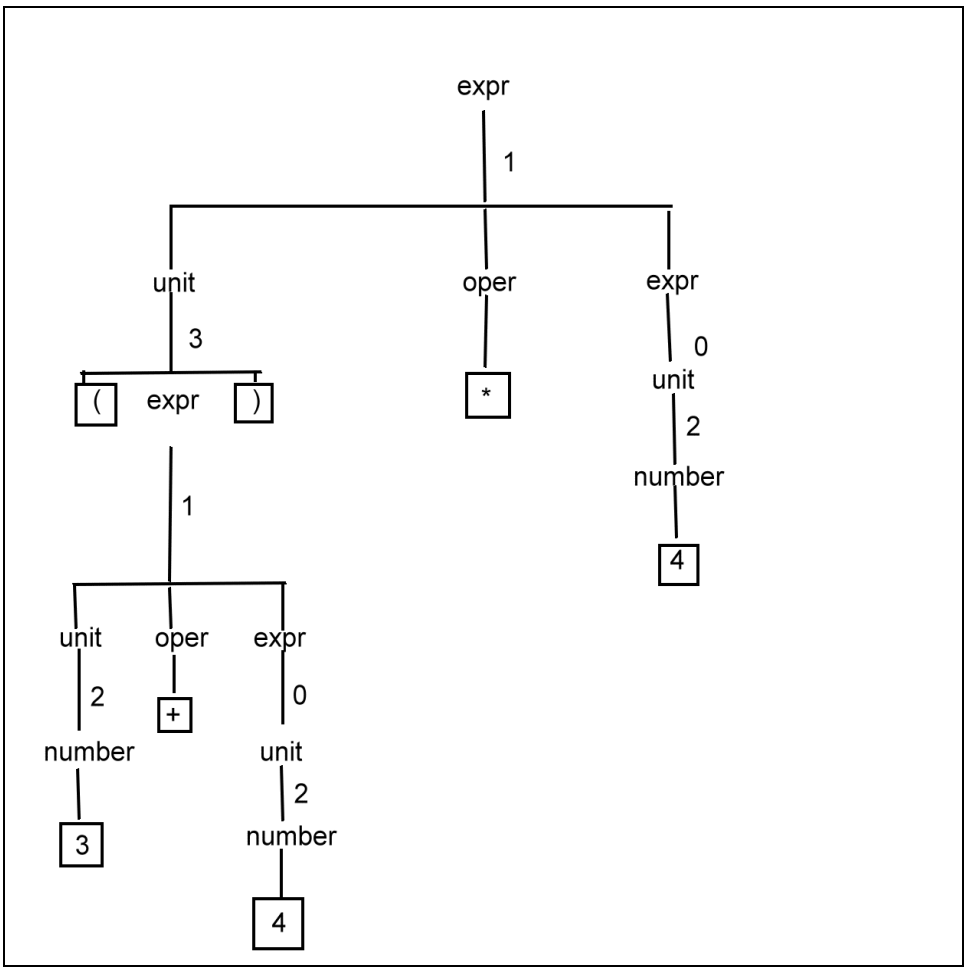

Figure 3-7. Parse Tree for (3+4) * 4

In the figure, the rules are indicated by numbers and the terminals are in boxes. Our tree indicates that the input can be recognized as an **expr** following rule 1. The goals for the tail of rule 1 are listed next, and the rules that resolve them are indicated by the rule number. This continues until all leaves of the tree are terminal symbols that represent the input string.

Taking a clue from previous problems, we recognize that a DFS pattern can probably be used if we can draw a tree that represents the output of our parser. We must recognize the two types of nodes and what it means to generate a `firstChild` and a `nextSibling`. The former will be a node generated by going down the tree, the latter by taking an alternative across the tree.

Our parse tree is the *result* of the DFS. It is a spanning tree where only the alternatives that were selected are shown. If you look at the root node, **expr**, you see it was resolved by rule 1, not by rule 0, because rule 0 would *not* have led to a tree that expressed the input as leaf nodes. Calculating the tree involves trying alternatives for rules at each node until a valid set is discovered that parses the input.

One strategy would be to use DFS to construct all possible trees. For each tree, see if the leaves match the input. This is not feasible because there are infinitely many trees that do not match (owing to the recursive nature of the rules). Rather, we must let the input string guide us as we build the tree so that we can eliminate trees that could not possibly match (just as we eliminated invalid queens when we built up the 8-Queens DFS tree).

Nodes in our Parser will represent the resolution of a goal by a grammar rule. The `firstChild` will be the resolution of a goal by the first rule that applies. The `nextSibling` of a node will be the application of a different rule to resolve the goal. The root node will be the start symbol that is to represent the entire string (in our case, **expr**).

Our node class will be `Parse`. Here are the instance variables, the constructor, and the `parent` method:

```
public class Parse : IGNode<Parse>
    {
        private Parse theParent = null;
        public TOKEN ruleHead;
        public int ruleNumber = -1;
        public Stack<TOKEN> goalStack;
        public List<LexInput> toParse;  //what is left to parse
        public static Parser parser;

        public Parse()
        {
            goalStack = new Stack<TOKEN>(10);
            toParse = new List<LexInput>(10);
        }
```

```

        public Parse parent
        {
            get
            {
                return theParent;
            }
            set
            {
                theParent = value;
            }
        }
```

The `ruleHead` is a goal we are trying to resolve (i.e. match against the head of some rule). The `ruleNumber` will be an index into a `List<Rule>` that is kept in the `parser`.

The `goalStack` requires some discussion. When we pick a rule to resolve the `ruleHead`, we will push the goals of that rule onto the `goalStack`. When a `firstChild` of this node is created, the stack will be copied to that child node. In the new child, the first node of the stack will be popped to become the `ruleHead` of the child. This pattern will continue down the `firstChild` chain in the tree. Thus the `goalStack` represents the complete list of goals that remain to be resolved if the `firstChild` chain is to resolve the root goal (the start symbol).

As terminal symbols are matched to the input string, we will reduce the `goalStack`. The amount of the input that is left to match is contained in `toParse`. The `parser` is shared across all nodes. It is driving the DFS and contains the grammar rules.

Here is the logic for `firstChild`:

```
public Parse firstChild()
        {
            if (goalStack.Count == 0 || toParse.Count == 0)
                return null;  //ran out of stack or out of input
            Parse clone = this.clone();
            clone.parent = this;

            clone.ruleHead = clone.goalStack.Pop();

            //find the first rule to fit clone.ruleHead
            for (int i = 0; i < parser.rules.Count; i++)
            {
                Rule rule = parser.rules[i];
                if (rule.head == clone.ruleHead)
```

```
                    {
                        clone.ruleNumber = i;
                        clone.pushTail(rule);
                        clone.reduceInput();
                        return clone;
                    }
                }

                return null;  //no rules match.  This means
                    //we encountered a terminal in a rule that
                    //does not match the next token in the
                    //input.
            }
```

The first line causes our parse to stop the `firstChild` chain if we run out of goals and have input left, or vice versa. To make a new child node (in `clone`), we copy the current node (`clone` does a deep copy of the `goalStack` and the `toParse`). Here is the code for clone:

```
public Parse clone()
        {
            Parse clone = (Parse)this.MemberwiseClone();

            TOKEN[] tempArray = goalStack.ToArray(); //"popped" out
            clone.goalStack = new Stack<TOKEN>(goalStack.Count+10);
            for (int i = tempArray.Length-1; i >= 0; i--)
                clone.goalStack.Push(tempArray[i]);

            clone.toParse = new List<LexInput>(toParse);

            return clone;
        }
```

After making the clone, we make a new `ruleHead` by popping the `goalStack`. Then we search for a rule that will resolve that.

If we find a matching rule, we note the `ruleNumber` (when we backtrack and seek an alternative rule, we use this to keep track of rules already tried. This will prevent looping on the `nextSibling` chain).

Then, we push the new set of goals (the tail of the rule that resolved the `ruleHead`) onto the `goalStack`.

The method `pushTail` is:

```
void pushTail(Rule rule)
        {
                    //tail goes on backwards: last in first out
            for (int i = rule.tail.Count-1; i >= 0; i--)
```

```
            {
                goalStack.Push(rule.tail[i]);
            }
        }
```

Next, we look at our input and compare it to goals on the stack via reduceInput. Its code is:

```
void reduceInput()
        {
            while (goalStack.Count > 0)
            {
                if (toParse.Count == 0)
                    break;
                if (goalStack.Peek() == toParse[0].token)
                {
                    goalStack.Pop();
                    toParse.RemoveAt(0);
                }
                else
                    break;
            }
        }
```

The purpose of this method is to match terminal symbols in the stack with tokens in the input string. This is like normal goal resolution, except instead of resolution via a grammar rule, we resolve by matching the input.

If we can resolve a terminal in the stack we can remove it from the `goalStack` and also from the `toParse`. The parser thus "consumes" the input as it progresses toward a complete parsing. We will know we are successful if we run out of goals in the `goalStack` and out of input at the same time. The *only* way to remove elements in `toParse` and terminal symbols on the `goalStack` is via this method, `reduceInput`.

The method `nextSibling` represents a different choice of rule to resolve the first goal on our `stack`. Here is its code:

```
public Parse nextSibling()
        {
                    //try a different rule
            if (parent == null)
                return null;

            if (ruleNumber + 1 >= parser.rules.Count)
                return null;  //out of rules

            Parse clone = parent.clone();
```

```
            clone.parent = parent;

            clone.ruleHead = clone.goalStack.Pop();

            for (int i = ruleNumber + 1; i < parser.rules.Count; i++)
            {
                Rule rule = parser.rules[i];
                if (rule.head == clone.ruleHead)
                {
                        //an alternative rule matches our ruleHead
                    clone.ruleNumber = i;
                    clone.pushTail(rule);
                    clone.reduceInput();
                    return clone;
                }
            }
            return null;
        }
```

If the parent of this node does not exist (it must be the root node containing the start symbol) we do not have an alternative rule to try and we tell DFS so by returning `null`. The current node (`this`) represents a resolution that needs an alternative rule. Our current node has already pushed the "bad" rule's goals onto the `stack` (and it may have reduced the input as well) so we clone the parent of this node to make the `nextSibling`. This will give us a fresh start on an alternative rule. We will get the same `ruleHead` we have in the current node via the `Pop`, but we start our rule search at `ruleNumber + 1.` This is the next available rule to use as an alternative to the one in the current node.

If we find a rule, we push its tail on the stack and try and `reduceInput` just as we did in `firstChild`. If we run out of rules before we match, we return `null` to signal DFS that we have no alternative to this node.

You can probably anticipate the code for our `Parser`:

```
public class Parser
    {
        public List<LexInput> toParse;
        public List<Rule> rules;
        public bool parseOK = false;
        public int nodesSearched = 0;
        public Parse root;
        public Parse result = null;
        public Parser()
        {
```

```

        }

        public bool doParse()
        {
            Parse.parser = this;

            Graph<Parse> graph = new Graph<Parse>(root);

            foreach (Parse node in graph.depthFirst())
            {
                nodesSearched++;

                if (node.toParse.Count == 0)
                {
                    parseOK = true;
                    result = node;
                    break;
                }
            }

            return parseOK;
        }
```

We assume that some external function has built the `Parser` and initialized it for us. It set up the root node, which contains the entire input (in the root's `List<LexInput> toParse`). It also set up the list of grammar rules (in `rules`).

We share our parser across nodes via the static `Parse.parser.` Then we set up a graph to do the DFS and start it up via the iterator `depthFirst`. In the loop, we count the nodes returned and examine each to see if we are done.

We have made a flexible RDP in roughly 200 lines of code.

Do not be dismayed if the entire design did not "leap out" at you. However, you may well have recognized that the resolution of a goal is progress toward the parse. Therefore, resolution logic is a candidate for `firstChild` nodes. Part of parsing is to try alternatives when more than one rule applies. That is a candidate for `nextSibling` nodes. Keeping track of which alternatives have already been tried is simple because the rules are in a sequential list.

Granted, some parser "domain knowledge" was helpful in recognizing it would be useful to keep unresolved goals in a stack in each node. We also

realized that we could treat the list of input tokens as a set of grammar rules (each of which had no tail). These "rules" could be used to resolve goals in the stack, but only in the order they were encountered in the input. These aspects of the design are less obvious, but if you worked a few parses using the rules and sample input, they might have occurred to you.

Notice how the separation of the DFS control logic from the parsing logic made the analysis easier since we could ignore the former entirely.

But where is the parse tree that represents the input? You can recover the tree, with some difficulty, by chasing the parent pointers back from the solution node (the one that set `parseOK` to true). Frankly, it would have been easier to build up the parse tree as we went along. It would have cluttered the code somewhat, so it was left out. An example in the next chapter will show how we can build up additional structures as the DFS progresses.

We also left out error reporting. It is not sufficient just to return a `bool` that indicates the input could not be parsed. We should indicate where in the input string the parse failed and which rule we tried last.

We mentioned above that RDP's cannot process certain kinds of grammars. Suppose our rule numbered one was not **expr → unit oper expr**, but **expr → expr oper unit**. Now when we push the goals of the second version on the stack, the first goal popped will be **expr**. If we use the same rule to resolve it, we get a loop in our parent chain. Our RDP will "descend" forever, eventually running out of space. This situation is called "left recursion" and is the nemesis of RDP's. There are ways to recognize troublesome grammars and convert them to ones that can be parsed by a RDP, but we will not go into that.

Could we detect that sort of loop and make our parser more robust? When we get a call to `firstChild` or `nextSibling`, we could look at the current node and chase it's parent chain to the root, searching for a node that matches the new node we are about to create. A match would be:

The `ruleHead`'s in the two nodes match.

The `toParse` is the same.

In this case, we should not resolve the `ruleHead` by the same rule we used in the matching parent node. Our rule search should exclude this rule as a

possibility. Note that there might be many parent nodes that satisfy the match (they used different rules to resolve the `ruleHead`), and all the rules used in the matches must be excluded as the resolution rule in our new node. We will see some other ways of dealing with loops in the parent chain in DFS in later chapters.

It is one of the strengths of DFS that it is so widely applicable. You may have been surprised to learn that two such unrelated applications as Parser and 8-Queens share a common design pattern. Not only are the problem domains vastly different, but 8-Queens works "bottom up" while Parser works "top down". In the former, the partial solution works toward the answer by adding more and more queens. In the latter, we start with the conclusion (that the input is an **Expr**) and "prove" it by disassembling the start symbol until we arrive at the terminal symbols in the input.

DFS Debugging Tips

Sometimes debugging the DFS can be tricky. You can use the debugger to follow the nodes as they are created, determine when `firstChild` returns null, and which nodes we backtrack to. It is easy to get lost, however, and the following techniques might be helpful.

Put a node number in the nodes. Assign it in the iterator loop that receives the DFS nodes; increment the `nodeNumber` in the same loop. This will help you identify nodes easily as they are encountered in the debugger.

Add a `prevSibling` field to the nodes. Set it to `null` in a `firstChild` node, but set it to `this` in the node created by `nextSibling`. Now if you have a question about how, where, and when a node was created you can look at the node number and chase either the parent or `prevSibling` chains to see the node that created it.

In the iterator loop for the DFS, add the nodes received to a `List<IGNode>` collection.

The last suggestion will allow you to examine the complete set of nodes that DFS has returned. The node number will be the index into the collection. The sibling pointer will tell you if the node was generated by `nextSibling` or `firstChild`. You can move backwards and forwards through the collection in the debugger. It should answer all your questions.

But you must be careful about creating references like `prevSibling`. One of the strengths of DFS is that only the current parent chain is kept in memory. The garbage collector can collect nodes that were created but are no longer on that chain. If you keep a sibling reference, the entire graph might remain in memory. That's ok for debugging but not for production code.

How DFS Works

The code for `Graph.depthFirstSearch()` can be found in the SEL library. Its logic is to keep track of the last node returned, and ask that node for its `firstChild` as the subsequent node it returns. If there is no child, it tries to return the `nextSibling` of the last node returned. If that is unsuccessful, it asks the parent of the last node returned for its `nextSibling`, and so forth until it finds a sibling to return. As remarked in the 8-Queens problem, nodes that have no parent (root nodes) can still return a `nextSibling` when our graph is a forest instead of a tree.

Conceptually, when a node is returned it is "marked" so that it is never returned again by `depthFirstSearch`, for so long as the node is on the current parent chain. As we have discussed, this is a responsibility of the `IGNode` class (i.e. code owned by the application, *not* the `Graph` class).

Some authors define the above logic as *Depth First Search with Backtracking*, or just *Backtracking*. They define pure DFS logic to use a stack (initially set with just the root node) as follows: pop the stack and return that node. Push that node's immediate successors on the stack (in our terminology, that would be the `firstChild` and its `nextSibling` chain), in no particular order.

Their version of DFS returns nodes in the same order as ours, but keeps more nodes in memory than DFS with backtracking. The latter has only the nodes on the chain from the current node to the parent. The former keeps that chain in memory, as well as all immediate children of nodes on that chain.

Depth Bound

A radical way to avoid loops in DFS, or excessive CPU time, is to put a depth bound in the `firstChild` method. The *depth* of a node is defined

recursively. The depth of the root is zero; the depth of a `firstChild` node is the depth of its parent plus one, as is the depth of a `firstSibing` node. We could try and estimate the maximum depth our graph could reach (for parser, this might be based on the size of the input we are parsing). When the depth in `firstChild` reaches the depth bound we could return null. That prunes the rest of the children under the current node so that the DFS can explore other routes. Of course there is the danger that we might miss valid solutions because we did not let the search extend deeply enough.

Summary of the Depth First Search pattern

1. Think of your application as a graph. Distinguish root nodes as the initial step in a solution to the problem. View a path from a root node to some other node as a partial solution (that may or may not lead to a complete solution). Conceptually, the graph contains all possible solutions and non-solutions. A complete solution will be represented as a path from some root node to some terminal node. Application context will determine the meaning of the nodes along the way. DFS will explore the graph moving down the graph toward the solution, backtracking when no further progress can be made on the current path. When a solution path is found, we can either stop, or continue the DFS to find more solutions.
2. Each node in our graph will be created by either the `nextSibling` method or the `firstChild` method. The node class will inherit from `IGNode`. Regard the generation of `firstChild` nodes as making tentative progress toward our solution. Consider a `nextSibling` node to be an alternative to the current node. When a new node is created it must contain a reference to its parent. The parent is the current node when a `firstChild` node is created; it is the parent of the current node when a `nextSibling` node is created. `IGNode` requires you to implement a `parent` method to return the parent of a node.
3. Insure that you do not create cycles in either the parent chain or the sibling chain. This must be insured by the logic in the `IGNode` functions. If the entire graph is in memory at the

outset, (and thus the `IGNode` constructor is not called by `nextSibling` or `firstChild)`, a system of marking nodes can be used.

4. Once the `IGNode` class is defined, create the solution by setting up a `Graph` object; pass its constructor a root node. Then, invoke the graph's iterator, `depthFirstSearch`, in a `foreach` loop. Within that loop, you can test for a solution node.

The DFS is stingy with space, but the entire parent chain will have to be kept in memory. Checking for cycles, especially in the parent chain, can be a problem in some complicated applications (like the Parser).

DFS forces a search order upon you. You can control this somewhat in `firstChild` and `nextSibling`, but only if you can see how to order the alternatives before you generate the nodes. This is the greatest limitation of DFS. We will see in subsequent chapters how to get around this limitation by guiding the search, and by pruning partial solutions before they can be expanded.

4 *Variations on Depth First Search*

THE PREVIOUS CHAPTER should have given you grounding in how to use DFS to solve problems. In this chapter, we will see how a programming technique called Divide and Conquer can be adapted to use the DFS pattern. We also investigate one way we can improve DFS performance if we are seeking only the best of many solutions.

Divide and Conquer (D&C)

Divide and Conquer (D&C) applications are usually formulated as recursive programs. It requires you to chop up a problem into smaller problems *of the same type*, and then stitch their solutions together to form a solution to the original problem. Here is some very abstract pseudo code:

```
solve(problem)
   {
     if (problem is small)
            determine solution directly, and return solution

     else:
        break the problem up into pieces
        foreach (piece)
          solutions.Add(solve(piece))  //recursion; gather solutions
        solution = stitch the solutions together to get a solution to problem
        return solution
   }
```

This code does not address how we would obtain all solutions if there were multiple ways to chop the problem up into pieces.

It turns out that D&C can be considered a DFS problem. The clue to the design is in the Parser problem in Chapter 3. Indeed, Parser itself might be

considered a D&C problem in that we have a single problem (the "start symbol"), which we divide into subproblems of the same type (new goals that result from the resolution of the start symbol). We continue breaking down goals via rule resolution, until all goals are matched to the input and thus eliminated from the goal stack.

The basic design feature that allows us to solve the D&C problem with a DFS is the stack which we keep in each node. As with the Parser, the stack represents subproblems yet to be solved.

The "stitch the solutions together" part of the D&C algorithm was not addressed in Parser (that part, building the parse tree, was left out of our discussion to simplify it). To illustrate the entire solution of a D&C problem with DFS, we will revisit a parenthesis problem, similar to that discussed in Chapter 2.

PARENTHESIZING A LIST

This form of the problem is: for a list of integers, produce all balanced ways of parenthesizing the list, wherein a maximum of 2 integers occur without intervening parentheses. For example, the ways to parenthesize [1,2,3,4] are:

(1(2(34)))

(1((23)4))

((12)(34))

((1(23))4)

(((12)3)4)

You can think of this problem as determining all ways to multiply n things together, wherein *multiply* is a binary operation which is associative.

With a little study, you can see that this is a D&C problem. We know that each parenthesizing will have two pieces, each of which is also a parenthesizing. For example, in the above list, the first two parenthesizings are broken down into the two pieces: the first piece, [1], is the same for each; the second is one of the two ways to parenthesize [2,3,4].

Thus we can look at the first "divide" as all ways to split the list [1,2,3,4] into two pieces (there are 3 ways:{[1], [2,3,4]}, {[1,2],[3,4]} and {[1,2,3],

[4]}). Subsequent "divides" will break these pieces into other pieces, as necessary.

Similar to Parser, our `firstChild` method will take the first goal off the stack and split it up. The `nextSibling` method will split up the same goal in an alternative way. We put the results of the split (which are two more goals) onto the stack. When the goal from the stack is a list with only one element, it can no longer be split and we will just remove it.

The "stitching together" part will be handled with a tree structure that is built up as we split the lists. Here is the C# code for that class, `Tree`.

```
public class Tree
    {
        public List<int> toSplit;
        public Tree parent = null;
        public Tree leftChild = null;
        public Tree rightChild = null;

        public Tree(List<int> root)
        {
            toSplit = root;
         }
     }
```

We will use class `Tree` to represent a splitting of a list. The intact list is held in `toSplit`, and the pieces are held in the `leftChild` and `rightChild`. A single parenthesizing will be held in a "tree of trees", starting at the root (whose `toSplit` is our original list), and continuing through the splittings until the leaves contain lists of only one integer. To recover the tree as a string containing the parentheses, we can invoke the following method, which is also in class `Tree`:

```
public string evaluate()
        {
            //evaluate right & left branches, then stitch them together

            if (toSplit.Count == 1)
                return toSplit[0].ToString();

            string lString = "";
            string rString = "";

            if (leftChild != null)
```

```
                lString = leftChild.evaluate(); //recursive!

            if (rightChild != null)
                rString = rightChild.evaluate();

            return "(" + lString + rString + ")";
        }
```

Following the terminology used in our parser, we will keep a stack of Goals in our nodes:

```
public class Goal
    {
        public List<int> toSplit;
        public Tree owner;

        public Goal(List<int> split, Tree own)
        {
            toSplit = split;
            owner = own;
        }
    }
```

The trick to stitching solutions together is to build up a tree that owns the goals. Our node class looks like:

```
public class ParenNode : IGNode<ParenNode>
    {
        ParenNode theParent = null;
        int splitAt = -1;

        Goal toSolve;
        public Stack<Goal> goalStack;

        public ParenNode firstChild()
         {
            if (goalStack.Count == 0)
                return null;

            ParenNode child = this.clone();
            child.parent = this;

            child.toSolve = child.goalStack.Pop();
            child.splitAt = 0;
            child.split();
            return child;
```

```
21        }
22
23        private void split()
24        {
25            if (toSolve.toSplit.Count <= 1)
26                return;     //nothing to split
27
28            List<int> lList = toSolve.toSplit.GetRange(0, splitAt+1);
29            List<int> rList = toSolve.toSplit.GetRange(splitAt + 1,
30                toSolve.toSplit.Count - splitAt -1);
31
32            Tree rTree = null;
33            if (rList.Count > 0)
34            {
35                rTree = new Tree(rList);
36                Goal rGoal = new Goal(rList, rTree);
37                goalStack.Push(rGoal);
38            }
39
40            Tree lTree = null;
41            if (lList.Count > 0)
42            {
43                lTree = new Tree(lList);
44                Goal lGoal = new Goal(lList, lTree);
45                goalStack.Push(lGoal);
46            }
47
48            toSolve.owner.leftChild = lTree;
49            toSolve.owner.rightChild = rTree;
50        }
51    }
```

The `firstChild` method clones the parent, takes the first goal off the stack, and splits it at position 0 (line 19). To split a node (via method `split`) we obtain the two lists that represent the split (in `lList` and `rList`, lines 28-30). We need to make two new goals for each of these lists (`rGoal` and `lGoal`) and put them on the stack. Each goal, including the one we are solving and the two new ones, have an owning tree (lines 36, 44). The goals are pushed onto the stack at lines 37 and 45. We stitch the two new trees to the tree that owns the goal we are solving (lines 48-49).

The `clone` method is similar to that in Parser and will not be repeated here.

The `nextSibling` method is:

```
        public ParenNode nextSibling()
        {
            if (parent == null)
                return null;

                //splitAt k means split *after* toSolve.toSplit[k], so
                //we *cannot* splitAt
toSolve.toSplit[toSolve.toSplit.Count-1]
            if (splitAt + 1 >= toSolve.toSplit.Count-1)
                return null;

            ParenNode sib = this.parent.clone();
            sib.theParent = this.parent;
            sib.toSolve = sib.goalStack.Pop(); //same as this.toSolve
            sib.splitAt = this.splitAt+1;    //next position to try split
            sib.split();
            return sib;
        }
```

Here we create a node that almost matches the node we created for `firstChild`. The difference is in where we split the goal we are solving (in `toSolve`). We just increment that position from the one in the current node.

Notice when the node creation methods return null: `nextSibling` when we run out of places to split the list in the `toSolve`, and `firstChild` when there are no more goals to pop.

With any DFS solution it is good to review how loops are avoided in the parent and sibling chains. Calling `firstChild` will replace the goal at the top of the stack with two goals of smaller size (in terms of the length of `toSplit`). Eventually, goals of length one are achieved, and then removed. No `firstChild` node can be repeated in a parent chain since the stacks in each one must be different.

In the sibling chain, the top goal on the stack of one sibling must differ from that on other sibling's stack since the method of splitting is guaranteed to be different. So `nextSibling` nodes on the same sibling chain cannot be the same.

Look carefully at how the tree is built up from each goal, and how a goal is "resolved" by splitting it into a left-goal and right-goal, with the associated trees stitched to the tree that owns the unresolved goal.

Now let's use the DFS machinery to solve the problem. The following method would be put in some class that is built to solve the problem:

```
1        public List<string> doParens()
2        {
3            Goal root = new Goal(toParen, new Tree(toParen));
4            ParenNode rootNode = new ParenNode(null);
5            rootNode.goalStack = new Stack<Goal>(1);
6            rootNode.goalStack.Push(root);
7
8            List<string> answers = new List<string>(3);
9
10            Graph<Paren> graph = new Graph<Paren>(rootNode);
11            foreach (ParenNode node in graph.depthFirst())
12            {
13                numberNodes++;
14                if (node.goalStack.Count == 0)
15                    answers.Add(root.owner.evaluate());
16            }
17
18            return answers;
19        }
```

The variable `toParen` contains our list of integers. We make up one goal at line 3, and use it to make our root node (lines 4-6). We will collect our answers in a list of strings (`answers`, line 8). As with parser, we know when the DFS reaches an answer when the node's `goalStack` has no goals on it. At that point, we have a tree based at the root that we can evaluate for a complete parenthesizing (line 15).

The code should be suggestive of how to build up a parse tree for our parser problem in the previous chapter. In that problem, we had a grammar that contained operators with only two operands. Thus it would be possible to use the same binary tree structure as in the parenthesizing problem. For more complicated grammars (where rules are resolved by more than two goals in the tail) the branching factor in `Tree` would depend on the number of goals in the tail of the rule. Thus you might have different types of trees, and different evaluation methods (one for each kind of tree). The evaluation methods could be virtual methods in subclasses of the class `Tree`.

Performance of D&C

If a problem's CPU time grows rapidly with its size, D&C can be quite efficient. This is because 2 small problems can sometimes be solved in much less time than one large one, even when we include the time to split the problem up and to stitch the 2 solutions back together. This would be true if the CPU time for a program grows exponentially or factorally with the size of the problem. Unfortunately, for some famous problems like TSP, no one has discovered a D&C solution that solves the problem optimally.

Recursion vs. DFS

Sometimes it is easy to see a recursive solution to a D&C problem. This is especially true if there is only one way to do the "divide". However, as you use DFS more and more, it may be that the DFS solution will appear more natural. Some advantages of DFS over recursion are:

1. DFS can handle alternative ways to do the "divide" more easily than recursion.
2. DFS solutions are a little easier to debug, since you do not have to examine the call stack to follow the logic. The data inspector in the debugger is all that is required to debug the DFS solution.
3. Some implementations of C# have a hard limit on the number of stack frames. After a certain number of recursive calls, the runtime system halts with an error. DFS is limited only in the amount of memory available.

Summary of the Divide and Conquer Pattern

1. Try and see how the original problem can be split up into smaller problems of the same type, and how you can stitch the solutions together.
2. Devise a `firstChild` method to make the split. If there are alternatives as to how to make the split, put these in the `nextSibling` method. If there is only one way to do the split, `nextSibling` can just return null.

3. Use a goal-stack in the nodes to hold the split up problems. The method `firstChild` removes the top goal and splits it, putting the subgoals back on the stack. The `nextSibling` method splits the top goal in the parent in a different way than the previous sibling did.
4. Use an auxiliary Tree structure to hold the goals. Build up the Tree as the goals are split. When the DFS is done, the Tree can be used to stitch the solutions together.

Remark

D&C can duplicate work because it solves the same problem multiple times. For example, when we split a problem, each sub-problem might contain the same problem. Thus if we split `[1 2 3 4 5 6 7 8]` into `[1 2 3 4 ], [5 6 7 8]`, and must parenthesize each half, obviously the ways of placing the parentheses are the same (because the size of each half is the same, 4). When we consider Dynamic Programming in the next chapter we will reexamine this defect of D&C.

Branch and Bound (B&B)

In the 8-Queens problem and in Parenthesizing, we wanted all solutions and no particular solution was better than any other. In Caves, we were willing to stop at one solution no matter whether it was the best or not. In Parser, we assumed there was only one way to parse the input (which is to say, the grammar was unambiguous).

Many problems, though, have a numeric "score" associated with solutions, and we seek a solution that maximizes or minimizes the score. We call these kinds of problems *optimization* problems. TSP is one such. Its score is the length of the tour and we seek to minimize that.

If we have an optimization problem, we can (perhaps) use B&B to find the best solution in less time than it would take to search out and examine all solutions.

The basic idea of B&B is to keep the best solution obtained so far. Then we can prune the search by quitting the current partial solution as soon as we realize it cannot possibly turn out better than our best one.

For example, suppose we are doing an exhaustive search for the TSP and have saved the best tour we have encountered so far. As we examine another solution, as soon as a partial solution exceeds our best tour, there is no point in continuing to add cities to the partial solution; it cannot possibly be extended to a better tour than the one we have in hand.

Note that this would not work if we sought the longest tour, since we would not be able to quit a partial solution until the tour was completed; there is always a chance that adding another city will cause it to exceed the length of our best tour.

Let's explore Branch and Bound with the "Knapsack" problem.

KNAPSACK

This problem is almost as famous as TSP. Like TSP, it is hard to find the best solution quickly when the problem gets large.

We have a knapsack that can hold at most 17 kilograms. We want to fill it with some ingots of different materials, weights, and values (worth). We must fill the knapsack in such a way as to maximize the worth (in dollars) without exceeding the 17 kg weight limit.

Thus Knapsack is an optimization problem. The "score" is the worth of a packed knapsack (whose capacity is not exceeded), and we wish to maximize it.

Here is a small version with a few kinds of ingots.

Weight	Worth/Ingot	Material
5	21	Gold
4	8	Silver
2	3	Copper
1	1	Lead

We have a large supply of ingots of each kind of material. How many ingots of each kind should we put in our knapsack? Let's begin our discussion with a straight-forward DFS solution to the problem.

Our ingots are represented by:

```
1     public class Ingot
2     {
3         public int weight;
4         public int worth;
5         public string name;
6         public static List<Ingot> ingot;   //all possible ingots
7         public static float highestDensity = 0.0f;
8
9         public Ingot(int weightP, int worthP, string nameP)
10         {
11             weight = weightP;
12             worth = worthP;
13             name = nameP;
14             if (this.density() > highestDensity)
15                 highestDensity = this.density();
16         }
17
18         public float density()
19         {
20             return (float)(worth) / (float)(weight); //assumes weight,
worth > 0
21         }
```

All of the ingots are kept in the static list, `ingot` (the code to fill `ingot` is not shown). The first part of our node class looks like:

```
1     public class KSnode : IGNode<KSnode>
2     {
3
4         KSnode theParent = null;
5         public int ingot;        //index into Ingot.ingot
6         public int qty;         //of ingots of type ingot
7         public int worth = 0;  //of knapsack, including these ingots
8         public int weight = 0; //of knapsack, including these ingots
9
10         public static int capacity;
11
12         public KSnode(KSnode parentP, int ingotP, int qtyP,
13             int weightP, int worthP)
14         {
15             theParent = parentP;
16             qty = qtyP;
17             ingot = ingotP;
18             worth = worthP;
```

```
            weight = weightP;
        }
    }
```

The node will represent the total quantity of ingots of a single type (the `ingot` index) that we have placed in the knapsack. If we follow the parent chain from a terminal node back to the root node, we will obtain one packing of the knapsack. The `firstChild` method puts ingots into the knapsack:

```
public KSnode firstChild()
        {
                //use the next type of ingot to fill the KS, starting with
                //0 qty of it

            if (ingot + 1 >= Ingot.ingot.Count)
            {
                return null;
            }

            Ingot newIngot = Ingot.ingot[ingot + 1];
            int newWeight = weight;
            int newWorth = worth;
            int newQty = 0;

            if (ingot + 1 == Ingot.ingot.Count - 1) //last type to add
                    //if this is the last ingot type, fill up knapsack as
                    //best we can.
                newQty = (capacity - weight) / newIngot.weight;


            KSnode child = new KSnode(this, ingot+1, newQty,
                weight + newQty * newIngot.weight,
                worth + newQty * newIngot.worth);

            return child;
        }
```

We just get the next type of ingot and put zero ingots of that type in the knapsack. When the last type is reached, we do the best we can to fill the remaining capacity of the knapsack with it.

You can anticipate that `nextSibling` will just place an alternative number of ingots in the knapsack:

```
public KSnode nextSibling()
        {
            //add in one more ingot of the same type as this one
            int newQty = qty + 1;
            Ingot ingotType = Ingot.ingot[ingot];

            int newWeight = newQty * ingotType.weight;
            int newWorth = newQty * ingotType.worth;

            if (parent != null)
            {
                newWeight += parent.weight;
                newWorth += parent.worth;
            }

            if (newWeight > capacity)
                return null;

            KSnode sib = new KSnode(this.parent, ingot, newQty,
                newWeight, newWorth);

            return sib;
        }
```

Figure 4.1 shows the DFS generation of the nodes.

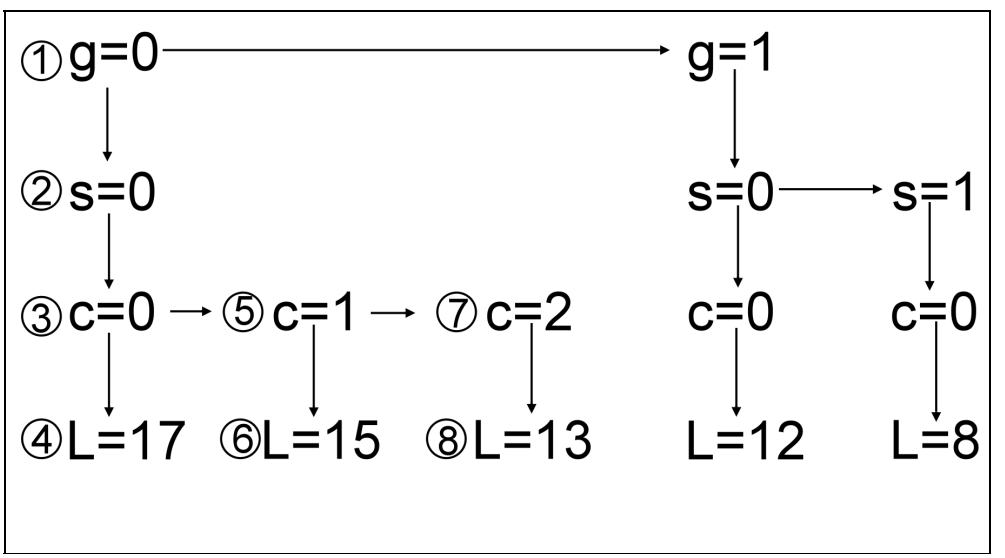

Figure 4-1. DFS generation of nodes for Knapsack

This is just a section of the graph, with many nodes left out. The first few nodes are tagged with the order in which they are generated (in the circle). The letter (g,s,c,L) indicates how many ingots of that type (gold, silver, copper, lead) are contained in the node. The down-pointing arrow shows the `firstChild` chain. The right-pointing arrows show the `nextSibling` chain. Remember that these are virtual chains: the only references actually held are `parent` references from a child (these are not shown). You can see how all possible combinations of the first three types of ingots are produced (the quantity for the lead ingots is filled in once the quantities of the first 3 are determined).

To obtain a packing, you start from a leaf node and follow the parent references (which are not shown). One such packing would be nodes [L=13,c=2,s=0,g=0].

The DFS to look at all packings of our knapsack is:

```
public class KSproblem
    {
        public static KSnode best = null;

        public int nodes = 0;

        public KSproblem(int cap, List<Ingot> ingots)
        {
            Ingot.setIngotTypes(ingots);
             KSnode.capacity = cap;
         }


         public void solve()
         {

             KSnode root = new KSnode(null, 0, 0, 0, 0);

             Graph<KSnode> graph = new Graph<KSnode>(root);

             foreach (KSnode ksn in graph.depthFirst())
             {
                 nodes++;
                 if (ksn.ingot == Ingot.ingot.Count - 1)
                 {
                     //leaf node: possible solution
                     if (best == null || ksn.worth > best.worth)
```

```
                    {
                        best = ksn;
                    }
                }
            }
        }
    }
```

This code should be familiar to you by now. The constructor for the class sets up the problem, including the ingots and the knapsack capacity. We create a root node and graph, and invoke the DFS iterator. We detect a leaf node when a node contains the last ingot type. We keep track of which packing is `best`.

If we submit a capacity of 17, with the ingots in the above table, we get:

0 of Lead 1-1

1 of Copper 2-3

0 of Silver 4-8

3 of Gold 5-21

Worth: 66, weight: 17

The total number of nodes visited was 114 to obtain the best solution (an exhaustive search).

The key to B&B is to prune the search when we are going down a path that cannot possibly lead to a solution better than one we have already obtained.

Heuristics

A heuristic is usually defined as a "rule of thumb"; a way of cutting through a complicated problem with an approximation that is easily arrived at. For our purposes, we will define a heuristic as a way of estimating the score of a complete solution, given a partial solution we wish to extend. In the context of Knapsack, we have a partially filled knapsack and wish to estimate the final worth of the knapsack once we fill it to capacity.

We want our heuristic to be "optimistic". That is, we don't mind if it overestimates the final score (for a maximization problem), but it must not

underestimate it. If it did the latter, we might abandon a partial solution that might turn out, when extended, to be the best one.

On the other hand, if the heuristic is too far above the actual final score, we will wind up not cutting off any partial search. It will always look as if the partial solution could be extended to one better than the best one so far.

Heuristics will play an important part in the searches discussed in subsequent chapters.

A good heuristic for knapsack is to assume the remaining capacity can be filled entirely with the densest (in terms of worth per kg) material. We ignore the actual weight of each ingot of that material, but assume the material will exactly fit the remaining capacity (as if we had gold dust and not a gold ingot of a specific size). We will call this the "gold dust" heuristic.

If you study this heuristic you will see that it is optimistic (always overstates the eventual worth of the knapsack). If filling the remaining capacity via the heuristic results in a smaller worth than some solution we already have in hand, then there is no use continuing to fill the knapsack; we can abandon the partial solution at that point.

Here are some additional methods to add to our `KSNode` class to support our heuristic:

```
35        public float estimateWorth()
36        {
37            return (float)(worth +
38                (capacity - weight) * Ingot.highestDensity);
39        }
40
41        bool hopeless()
42        {
43            //see if this node cannot possibly lead to a better KS
packing
44            if (KSproblem.best == null)
45                return false;
46
47            if (estimateWorth() < KSproblem.best.worth)
48                return true;
49            return false;
50        }
```

All we need do in our `firstChild` and `nextSibling` methods is to include the following code before we return the new node we have just constructed in the method:

```
if (sib.hopeless())
                return null;
```

which goes in nextSibling, and

```
if (child.hopeless())  //B&B test
                return null;
```

which is placed in `nextChild`.

These statements terminate the DFS's expansion of the current partial solution as soon as we detect that extending the search cannot possibly result in a better packing than the best one (which we have kept in the static variable, `KSproblem.best`). DFS reacts as if the chains have run out normally, and will backtrack to try a new partial solution.

It is important to realize that the reason we can abandon the search in `firstChild` and in `nextSibling` is because, as we progress along the `firstChild` chain and along the sibling chain, we *add* new ingots to the knapsack and the weight must increase. If we took out an ingot and replaced it with another, in either method, we could *not* abandon the search by returning `null` after testing against the heuristic. Another way of saying this is, that during the DFS search, continuing a partial search along either the `nextSibling` chain or the `firstChild` chain only increases the weight of the knapsack, it never decreases it. Since we cannot add material of greater worth than the highest density one, and can achieve no better packing than filling the remaining capacity with that material, there is no sense in continuing to fill the knapsack if our heuristic tells us to quit.

If you want to use B&B you will need to insure that your partial searches behave this way. Abandon the partial search (return `null`) only if you are sure that nodes that follow the current one in the DFS would also fail the heuristic test.

How does B&B perform?

With the B&B logic, for the same knapsack problem (capacity 17), we visit only 106 nodes. That does not seem like much of a savings over the 114 node solution without B&B. However, B&B is very sensitive to how soon

we obtain the best solution. If we can arrange for the DFS to find a "pretty good" solution early, we can prune more nodes from the search. One way to do this is to sort the ingots so that we fill the knapsack first with the highest density ingot. The code to do this (executed before we make the root node) is:

```
Ingot.ingot.Sort(
                delegate(Ingot ingot1, Ingot ingot2)
            {
                //smallest densities first gives best results:
                return ingot1.density().CompareTo(ingot2.density());

                //because it insures that highest density ingots
                //will be used first in the DFS (since we start with
                //0 qty, and count upwards for an ingot).
            });
```

With this addition, the B&B solution visits only 11 nodes! Lest you think it is the sort and not the B&B logic that achieves this efficiency, the non-B&B solution with the sort visits 488 nodes.

If we don't use B&B logic it is best to sort the nodes with high-density ingots first (that ordering of the ingots got us the original solution, with only 114 nodes visited). Can you see why?

Summary of the Branch and Bound Pattern

1. B&B is used in optimization problems, in concert with DFS.
2. You need a heuristic that you can use to estimate how good the complete solution will be, given a partial solution. The heuristic should be optimistic (understating the final score for minimization problems, overstating it for maximization problems).
3. You must organize the DFS so that continuing the search (without backtracking) does not change the conclusion of the heuristic to abandon the partial search. Note that the two chains, `nextSibling` and `firstChild` are independent in this sense. Perhaps your search would not change the conclusion of the heuristic along the `firstChild` chain but

not along the `nextSibling` chain. In that case, put the heuristic test in `firstChild`, but not in `nextSibling`.

4. In the DFS iterator, keep track of the best (complete) solution obtained so far. You will need its score to compare with the heuristic.
5. Abandon the current search path (return null) from `firstChild` and `nextSibling` if the heuristic suggests the completed solution will not be better than the one in hand. This forces DFS to backtrack.
6. B&B is very sensitive to how early in the search you obtain a reasonably good solution. The sooner that happens, the earlier the heuristic will begin to prune more and more nodes. Sometimes changing the order in which `firstChild` and/or `nextSibling` nodes are generated can make a dramatic difference.

5 *Dynamic Programming*

DYNAMIC PROGRAMMING (DP) is another optimization technique. It is similar to Divide and Conquer in that it solves a large problem by solving smaller ones and combining the solutions together. As with D&C we will replace the usual recursion by making use of DFS as a control mechanism.

DP is applicable to problems that can be solved in "stages", and which have the property that an optimal solution to the entire problem must result in optimal solutions for each stage, given the input and output for each stage. This is called the *Principle of Optimality*, originally formulated by Richard Bellman, the pioneer of DP.

KNAPSACK PROBLEM (REVISITED)

To illustrate, let's reexamine the knapsack problem of the previous chapter. Consider the two "stages" of the problem as:

1. add the correct number of gold, silver, and copper ingots to the knapsack
2. add the correct number of lead ingots to fill out the knapsack.

Suppose the total weight of the lead ingots in the optimal solution is `L`. Since we have the best packing, if we then remove the lead ingots, we must have an optimal solution of the following knapsack problem: fill a knapsack of capacity `17-L`, with ingots of the first 3 types. (Remember that 17 is the capacity of the original knapsack problem). This is easy to see: if we had a better packing for `17-L` (i.e. one of higher worth) with the first 3 ingot types, we could substitute that packing in the original knapsack, and thus obtain a better solution for the capacity 17 knapsack (and we would not change the same number of lead ingots in the final solution).

So it is easy to see that for those 2 stages, each is optimal under the assumption that the entire packing is optimal. Hence the principle of optimality is satisfied.

Suppose we had optimal solutions for the knapsack problem, using the first 3 types of ingots, for capacities of 17, 16, 15, …1, 0. Then we could try filling each of these optimal knapsacks with quantities of the lead ingots to a capacity of 17. Then we would pick the best knapsack (highest worth) among them. That would give us the solution to the capacity 17 knapsack.

This suggests that we could solve small knapsack problems (with a few types of ingots and various knapsack capacities) and then use these solutions to get solutions to larger knapsack problems (by extending the smaller ones in various ways to get the larger ones, picking the extension that gave the best solution). This is the essence of Dynamic Programming.

Using DFS in Dynamic Programming

The traditional way to solve DP problems is to build up a table starting with the solutions to the small problems and using a recursive program to build up the solutions to the larger ones.

The same effect can be obtained by using DFS to solve the problem top-down, solving the subproblems as we descend the graph, and tabling them for reuse. This is like the D&C pattern, except we avoid duplicate work by tabling results as we go along, looking them up in the table as they are encountered again. The idea of programming DP using a top down search instead of employing a bottom up table is discussed in Reference [6].

Another difference is that D&C is not an optimization technique; DP is such.

The trick in realizing a DP solution is to formulate an optimal solution as one among a set of smaller problems, along with ways to extend each to form a solution to the original problem.

Thus the nodes in our DFS solution will contain a “component knapsack problem”, along with an amount of another type of ingot that extends the component to a solution of a larger knapsack problem (in that it involves one additional type of ingot).

The `firstChild` method reduces the component knapsack in the parent to one of smaller type (fewer ingot types in it), and asserts 0 as the number of

ingots of the "missing" type needed to extend the child to a solution of the parent's component knapsack.

The method `nextSibling` adds one to the number of ingots of the "missing" type and reduces the capacity of its component knapsack. Thus siblings represent different ways to reduce the parent's component problem: the siblings' component knapsacks have a smaller capacity, and the number of ingots of the missing type increases as we travel along the `nextSibling` chain.

You may wish to glance at figure 5.1 before reading the code.

We will now discuss the DP solution to Knapsack in detail. The first class is not a node, but represents a knapsack. This will be used as the "component" knapsack in the nodes:

```
public class Knapsack
    {
        public int targetWeight;        //max weight allowed in this ks
        public int firstIngot;  //smallest index into ingot that can be used
                                //to fill this ks
        public int worth = 0;
        public int weight = 0;

        public KSDnode solution = null;  //best way to fill this ks


        public static KSDnode origProblem;  //i.e. orig knapsack problem
        public static Dictionary<String, Knapsack> dictionary =
            new Dictionary<String, Knapsack>(10);

        public Knapsack(int sizeP, int firstIngotP)
        {
            targetWeight = sizeP;
            firstIngot = firstIngotP;
        }

        public static Knapsack fetch(int targetWeight, int firstIngot)
        {
            Knapsack hit = null;

            string key = targetWeight.ToString() + " " +
                firstIngot.ToString();
```

```

            bool got = dictionary.TryGetValue(key, out hit);
            if (!got)
                {
                    hit = new Knapsack(targetWeight, firstIngot);
                    dictionary.Add(key, hit);
                }

            return hit;
        }
    }
```

This class represents a knapsack with capacity `targetWeight`, and permissible ingot types represented by `firstIngot`. The latter means the knapsack can be filled only with elements from the `Ingot.ingot` list, beginning with the index specified by `firstIngot`, and continuing through the list to the end. The class `Ingot` code can be found in the previous chapter. Thus as `firstIngot` increases, there are fewer types of ingots in the knapsack (and in that sense, the knapsack problem gets smaller).

The optimal solution to this knapsack will be found in `solution` (explained later).

This class holds a static dictionary of knapsacks, maintained by the method `fetch`. When a solution to a knapsack problem is discovered, it will update the entry in the dictionary. A knapsack problem is fully specified by the capacity (`targetWeight`) and the permitted types of ingots (the latter specified as `firstIngot`).

Our node class begins as follows:

```
public class KSDnode: IGNode<KSDnode>
    {
                //KSnode represents a solution to a KS problem of
                //targetWeight, given a
                //solution to the componentKS problem, and addition of an
                //ingot (ingotAllocated) and quantity thereof
                //(qtyAllocated).

        KSDnode theParent = null;
        int targetWeight;       //of the KS problem this node tries to solve
        public int ingotAllocated;    //index into ingots
```

```
        public int qtyAllocated;      //of the ingot
        public Knapsack componentKS = null;    //smaller ks problem to solve
                                               //optimally

        public KSDnode(KSDnode parentP, int ingotP, int qty, int targetWeightP)
        {
            theParent = parentP;
            ingotAllocated = ingotP;
            qtyAllocated = qty;
            targetWeight = targetWeightP;
        }

        public KSDnode(int targetWeightP)
        {
                //make the *root* node
            theParent = null;
            ingotAllocated = -1;  //designates root node
            qtyAllocated = 0;
            targetWeight = targetWeightP;
            componentKS = new Knapsack(targetWeightP, 0);
        }
}
```

The comments and constructors should be self-explanatory. As explained above, the node represents a solution to a knapsack problem in terms of a smaller knapsack problem (`componentKS`), and the addition of a quantity of ingots (`qtyAllocated`) of a type (`ingotAllocated`) that is not in the component knapsack.

Some auxiliary methods to figure the weight and worth of a node are found in the same class. They just extend the component knapsack with the new ingot type and its quantity:

```
        int weight()
        {
            //figure the weight of aNode

            int compWeight = 0;

            if (componentKS != null)
                compWeight = componentKS.weight;

            Ingot ingot = Ingot.ingot[ingotAllocated];

            return qtyAllocated * ingot.weight +
                    compWeight;
        }
```

```

        int worth()
        {
            //figure the worth of aNode
            int compWorth = 0;

            if (componentKS != null)
                compWorth = componentKS.worth;

            Ingot ingot = Ingot.ingot[ingotAllocated];

            return qtyAllocated * ingot.worth +
                    compWorth;
        }
```

The `firstChild` method in this class is:

```
        public KSDnode firstChild()
        {

            if (ingotAllocated + 1 >= Ingot.ingot.Count ||
                componentKS.solution != null)
            {
                doLeaf();
                return null;
            }

                    //make a node that tries to solve the component KS in
                    //this node, with the next selection of ingots, starting
                    //at 0 qty of the first ingot allowed

            KSDnode child = new KSDnode(this, ingotAllocated+1, 0,
                componentKS.targetWeight);

            if (child.ingotAllocated == Ingot.ingot.Count - 1)
            {
                //if this is the last ingot type, fill it up as
                //best we can.  No component KS will be created in
                //the node.
                child.qtyAllocated = child.targetWeight /
                    Ingot.ingot[child.ingotAllocated].weight;
            }

            else  //we need a component KS.  Since we allocated 0 weight in
                //the child node, the target weight of the component is the
                //same as the child's target weight
            {
```

```
31                  child.componentKS =
    Knapsack.fetch(child.targetWeight,
32                      child.ingotAllocated + 1);
33              }
34
35              return child;
36          }
```

The child node will contain a knapsack with one fewer ingot types. The capacity is the same, since we extend it with zero ingots of the type left out.

The purpose of the child node and its descendents is to solve the `componentKS` in the child's parent. It provides such, because if we take the child's component knapsack and add into it the child's `ingotAllocated`, as specified by the child's `qtyAllocated`, we would have a knapsack that is the same as the parent's component knapsack.

At line 4 we test to see if we already have a solution, or if there are no more ingot types to exclude to make the child. If so, we have a leaf node and return after processing it (returning null since there is no child to make). The leaf node processing is contained in the following method:

```
37          public void doLeaf()
38          {
39              //DFS has bottomed out at a leaf.
40              //see if we have a better solution to a component
41              KSDnode aNode = this;
42              while (aNode.parent != null)
43              {
44                  int nodeWorth = aNode.worth();
45
46                  Knapsack parentComp = aNode.parent.componentKS;
47
48                  if (nodeWorth > parentComp.worth ||
49                      parentComp.solution == null)
50                  {
51                          //aNode has a better solution to the
    parent's
52                          //componentKS problem, so we fix the
    parentComp
53                      parentComp.worth = nodeWorth;
54                      parentComp.solution = aNode;
55                      parentComp.weight = aNode.weight();
56                      aNode = aNode.parent;
57                  }
58                  else
59                      break;
60              }
61          }
```

Once we have a leaf, we have a solution to some knapsack problem. The purpose of this method is to propagate solutions based on this one, up the parent chain. This method follows the parent chain, replacing the solution in component knapsacks as necessary.

Note that the component knapsacks are also contained in the dictionary so that their (updated) `solution` will now be available in all nodes that reference the knapsack. It is the dictionary that keeps us from duplicating work.

The `nextSibling` method is:

```
public KSDnode nextSibling()
        {
            if (ingotAllocated == -1)
                return null;    //this is the root: no sibling to generate

            int newQty = qtyAllocated + 1;
            Ingot ingot = Ingot.ingot[ingotAllocated];

            int totWeight = newQty * ingot.weight;
            if (totWeight > this.targetWeight)  //current node has filled the KS
            {
                return null;   //no more of this ingot can be added
            }

            KSDnode sib = new KSDnode(this.parent, ingotAllocated, newQty,
                targetWeight);
            sib.componentKS = Knapsack.fetch(targetWeight -
                newQty * ingot.weight, componentKS.firstIngot);

            return sib;
        }
```

Here we just add one to the `qtyAllocated`, keeping the `ingotAllocated` the same to make the new sibling. The new sibling's `componentKS` is the same as in its sister (the current node), except that the `targetWeight` must be reduced by the weight we have added to the new sibling node.

Figure 5.1 is an illustration of how the nodes are created during the DFS.

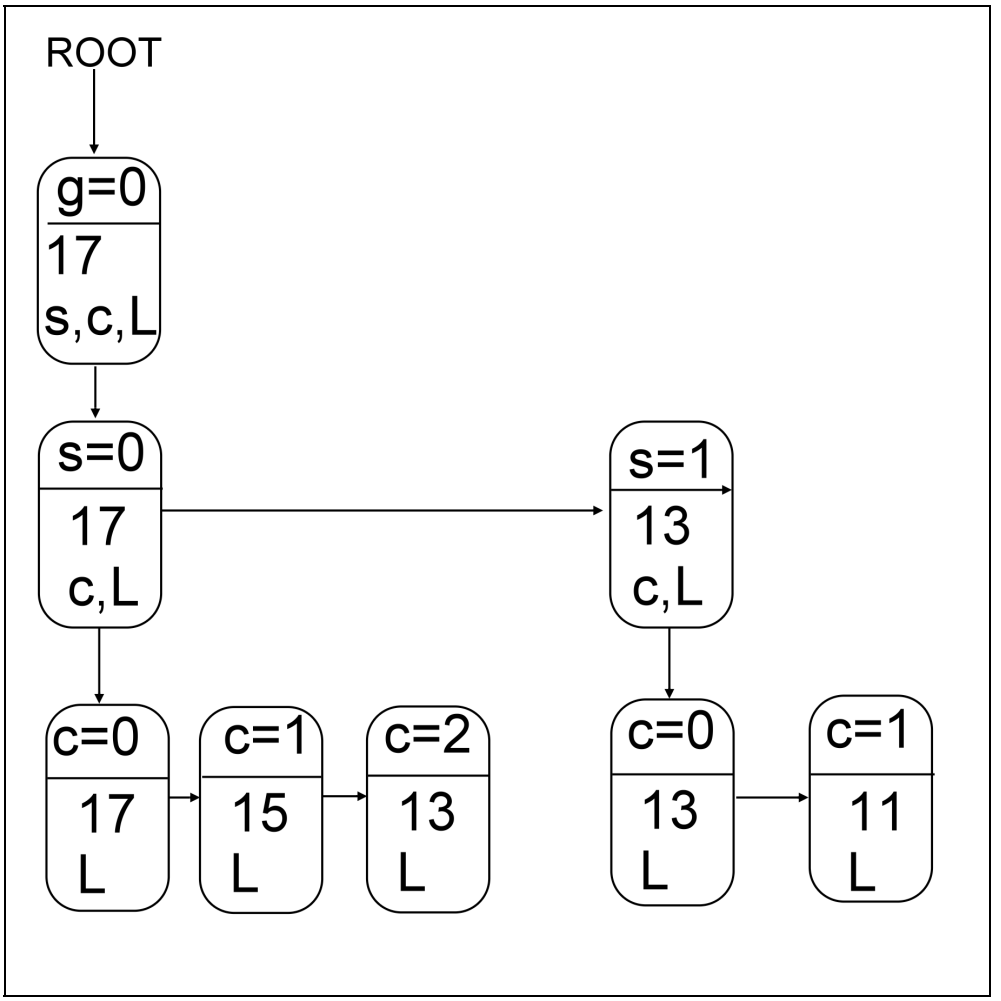

Figure 5-1. DFS node generation for DP version of Knapsack

Many nodes have been left out of the illustration. The top part of each node is the type of ingot (g,s,c,L for gold, silver, copper, lead) and corresponds to the `ingotAllocated` variable. We include the number of ingots allocated of that type, corresponding to the variable `qtyAllocated`.

The bottom half of the node represents the component knapsack. It shows the capacity (`targetWeight`), and the allowable types of ingots (corresponding to `firstIngot`).

The `firstChild` chains are downward-pointing arrows, the `nextSibling` are right-pointing arrows.

You can see that each `firstChild` node solves the component knapsack problem in its parent. The `nextSibling` node adds one to the ingot quantity in its sister, and has a component knapsack with the same ingot types as its sister. But it reduces the capacity of its component knapsack because the new sibling has more of the "missing" component in its `qtyAllocated`.

The siblings (including the `firstChild`) all solve their parent's component knapsack problem, each in a different way.

Look at the nodes at the bottom of figure 5-1. You will see that two have the same component knapsack: the ones with a target capacity of 13, and an ingot type of lead. When the latter of these is encountered, its component knapsack will already have been entered into the dictionary, and we will not generate the `firstChild` node under it, but will reuse the `solution` found in the dictionary.

Before programming a DP solution you should verify that potential exists to reuse solutions. Otherwise, your DP program is no better than an exhaustive search, and DP is probably not a good design choice.

The leaf nodes are not shown under the bottom row. They would not contain a component knapsack, but just the solution in lead ingots as indicated.

When a leaf node is generated, we put the solution represented in its parent's component knapsack, and continue upward via the parent chain. This propagation ends if the parent already has a better solution in it for its component knapsack. This is the `doLeaf` logic.

For example, the node for g=0 would have its component knapsack solved by the nodes s=0 and s=1. Only the one that solves it best (has the best worth) would find its way to the g=0 node, and upward to the root.

You should compare this figure with the similar one in the previous chapter, for the B&B version of Knapsack.

At this point, we take note of why the principle of optimality is important. In solving a subproblem, we need only consider optimal solutions. Thus if our subproblem is to fill a knapsack of capacity 13 with lead ingots, it does not

make sense to leave some capacity unused if we don't have to (i.e. take a suboptimal solution). Such a solution cannot be part of an optimal solution to the "big knapsack" problem, if we know that the principle of optimality holds.

The code to do the DFS to solve our knapsack problem, via DP, is:

```
public int solve()
{
    Graph<KSDnode> graph = new Graph<KSDnode>(root);
    int nodes = 0;

    foreach (KSDnode ksn in graph.depthFirst())
    {
        nodes++;
    }

    return nodes;
}
```

To recover the entire solution from the root, after the DFS is complete, we just examine the component knapsacks, starting at the root (remember the solutions to component knapsacks were propagated upward in the `doLeaf` method):

```
public string asString()  //return answer as a string
{
    string ans = "";
    KSDnode ks = root.componentKS.solution;
    ans = "weight filled: " + root.componentKS.weight.ToString() +
        " value: " + root.componentKS.worth.ToString() +
        Environment.NewLine;

    while (ks != null)
    {
        ans +=
            ks.qtyAllocated.ToString() + " of " +
            Ingot.ingot[ks.ingotAllocated].name +
            Environment.NewLine;
        if (ks.componentKS != null)
            ks = ks.componentKS.solution;
        else break;
    }
    return ans;
}
```

For a knapsack problem of capacity 17, with the ingots in order [gold, silver, copper, lead], our DP version visits 82 nodes. This is in contrast to the 114

nodes visited in DPS without B&B logic (in the previous chapter), and 106 nodes visited when we add B&B logic to that implementation. But B&B performed much better (11 nodes visited) if we pre-sorted the ingots in the reverse order ([lead, copper, silver, gold]).

The use of DP to solve a variety of knapsack problems is well developed in Reference [7].

Branch and Bound Revisited

Branch and Bound works by pruning the search tree via a heuristic. Dynamic Programming works by saving intermediate results and reusing them, instead of recalculating them.

Could we introduce B&B into our DP solution? Yes!

We will need some way to detect when pursuing the current path in the DFS graph would not result in a better solution than one we already discovered. We will augment the DP nodes to contain the current value of the knapsack (given by the worth determined by `ingotAllocated` and `qtyAllocated`, added up all the way back to the root).

We will estimate the worth of the component knapsack (whose true worth can only be obtained once we get to a leaf node) via the "gold dust" heuristic used in the B&B version of knapsack in the previous chapter.

We will introduce a new variable in the node, `currentWorth`, which is obtained by adding the parent's `currentWorth` and the worth of the `qtyAllocated` (of the `ingotAllocated`) in the child node:

The worth of the `componentKS` will be estimated by taking its `targetWeight` times the value per unit of the highest density ingot (the "gold dust" heuristic).

If the estimated worth of a node (its `currentWorth` plus the estimated worth of its `componentKS`) is less than the best solution to the knapsack discovered so far (which is in the root node), we can abandon the methods `firstChild` and `nextSibling`, returning null, instead of creating the corresponding graph node.

The new code at the tail end of `nextSibling`:

```
21              sib.currentWorth = parent.currentWorth +
```

```
                sib.qtyAllocated * ingot.worth;
            if (hopeless())
                return null;
                 return sib;
```

and the corresponding code at the tail end of `firstChild`:

```
            child.currentWorth = currentWorth +
               child.qtyAllocated *
    Ingot.ingot[child.ingotAllocated].worth;

            if (hopeless())
                return null;
            return child;
```

Here is the code for hopeless, which we put in class `KSDnode`:

```
        bool hopeless()
        {
            if (currentWorth + componentKS.targetWeight *
                Ingot.highestDensity <
                KSDproblem.root.componentKS.worth)
                return true;
            return false;
        }
```

Before executing the `solve` method, we would sort the ingots as we did in the B&B solution:

```
            Ingot.ingot.Sort(
                delegate(Ingot ingot1, Ingot ingot2)
            {
                //smallest densities first gives best results:
                return
    ingot1.density().CompareTo(ingot2.density());

                //because it insures that higest density ingots
                //will be used first in the DFS (since we start
    with
                //0 qty, and count upwards for an ingot).
            });
```

After doing all this work, we find the nodes visited in our DP/B&B solution is 16 (for knapsack problem with capacity 17). This is a bit worse than the straight B&B solution, which had 11 nodes visited (after we added the sort to B&B).

The reason why our combination DP/B&B did not perform better than B&B alone is because there was no reuse. The dictionary did not contain any problems that we could reuse before the best solution to knapsack was

found. Larger versions of the problem, with more ingot types and larger capacities (and a sort that was not so fortunate) would see the DP/B&B combination improve over B&B alone.

However, this example emphasizes that DP gets its advantage from reuse: no reuse implies there is no performance advantage over other techniques.

Knapsack was a maximization problem. Let's work a classic minimization problem from the field of Operations Research.

LOT SIZING PROBLEM

Suppose we have a factory with one machine and a 4-period (e.g. 4-month) schedule for production. The machine can produce as much as the entire scheduled production in a single period, but there are carrying costs (a charge, per unit, per period) for product in inventory at the start of the period. Furthermore, there are setup costs for the machine every period, if it is to produce anything in that period.

So the tradeoffs are: produce all of the schedule as quickly as possible to save setup costs (but incur carrying costs), or produce as little as possible each period to save carrying costs, but incur setup costs. Complicating the issue is that production costs per unit can vary from period to period.

The important data, a schedule for 4 periods, is in table 5.1.

Table 5-1. Lot Sizing Problem

Period	Demand	Setup Cost	Production Cost (per unit)	Carrying Cost (per unit)
0	3	$7	$2	$1
1	5	$9	$2	$1
2	9	$11	$4	$1
3	1	$7	$5	$1

The demand must be met at the end of the period. We need to figure out the amount of units to produce each period, so as to minimize the total cost of all the production. Making product early minimizes setup costs (which are 0 in a period that we do not make product), but early production will incur carrying costs. For example, the cost of producing all 4 periods' demand (18 units) in period 0 is:

7 (for setup) + 2 * 18 (production cost for all 4 periods), + 15 (periods 2-4 demand carried during period 1) + 10 (periods 2 and 3 demand carried during period 2) + 1 (period 3 demand carried during period 3), or $69.

In does not make sense to both:

1. have some inventory at the beginning of a period
2. make units during that period

To see this, suppose we had 3 units in inventory at the start of period 2, and thus must produce 6 to meet at least period two's demand. If one of the units in inventory has incurred less than the cost to make a unit in this period (period 2), then it would be less costly to have produced all the period 2 units at that time. On the other hand, if one of the units in inventory has incurred more costs than that to produce in this period, we should not have made it. We should have postponed making it until this period.

The upshot of all this is that there are really only a limited number of choices for production in a period: either produce 0, or produce enough to cover some integral number of periods. Likewise, either arrange for the inventory to be 0 ahead of a period, or else enough to cover at least that period's production.

If you do not follow that argument, it is not material. Just accept the constraint that our choices for production in any period are: 0, or current period's demand plus some integral number of subsequent periods' demand. If there is inventory ahead of a period, we will not produce during that period.

Let's construct a DFS solution. We could construct a "top down" solution by making our root node the production in period 3. Alternatively, we could construct a "bottom up" solution by constructing the root node as production in period 0. Let's try the first alternative.

Our schedule is captured by the class Schedule:

```
public class Schedule
    {
        public int demand;          //units of demand
        public int setupCost;       //incurred once in a pd, if any prodn
        public int prodnCost;       //per unit
        public int carryCost;       //per unit carried in inv, at pd start
```

```

        public static List<Schedule> schedule;  //index is the period
    }
```

Our “subproblem” is in class Plan:

```
public class Plan
    {
        public int period;        //index into Schedule.schedule: this period
        public int makeForInv;    //amount we will make this period for inv
                                  //ahead of next pd. If we make anything, we
                                  //must also make this pd's demand, additionally

        public int cost = 0;      //total for all periods through this period


        public PlanNode solution = null;  //contains plan for prev pd, and prodn

        public static Dictionary<String, Plan> dictionary =
            new Dictionary<String, Plan>(10);

        public Plan(int periodP, int makeP)
        {
            period = periodP;
            makeForInv = makeP;
        }

        public static Plan fetch(int period, int makeInv)
        {
            Plan hit = null;

            string key = period.ToString() + " " +
                makeInv.ToString();

            bool got = dictionary.TryGetValue(key, out hit);
            if (!got)
            {
                hit = new Plan(period, makeInv);
                dictionary.Add(key, hit);
            }

            return hit;
        }
    }
```

This class should be self-explanatory, since it follows the design of the previous problem, Knapsack. The `cost` field is the total cost. So if the plan is for period 3, `cost` represents the total cost for periods 0, 1, 2, and 3. The plan represents a production plan for the `period` (an index into the list of schedules). The only other parameter that characterizes the plan is the amount to make for inventory (which shows up as inventory ahead of the next period). Remember that if we make anything, we must have had zero inventory ahead of this period, and consequently we must make not only the inventory we want ahead of next period, but also this period's demand.

`Plan` represents the "stages" of our DP design. Hence it is appropriate to ask if our design satisfies the principle of optimality. Since there is only *one* way to execute the plan (make the amount of production called for in `makeForInv + Schedule.schedule[period].demand`), we satisfy the principle of optimality vacuously.

Here is the start of our node class:

```
1       public class PlanNode : IGNode<PlanNode>
2       {
3           //PlanNode represents a solution to a Lot Size problem
4           //for a period, given a solution to the lot size problem
    for
5           //the previous period (in componentPlan), and the prodn
    (=make).
6
7
8           PlanNode theParent = null;
9           public int period;              //of the plan
10          public int make = 0;
11          public int invAhead = 0;       //inv wanted ahead of this
    pd
12          bool isFirstChild = true;
13
14          public Plan componentPlan = null;    //smaller plan to
    solve
15                                              //optimally
16
17          public PlanNode(PlanNode parentP, int periodP, int makeP)
18          {
19              theParent = parentP;
20              period = periodP;
21              make = makeP;
22          }
23
24          public PlanNode()
25          {
```

```
                //make the *root* node.  Purpose is to gather the optimal plan
            theParent = null;
            period = Schedule.schedule.Count;
            componentPlan = new Plan(period-1, 0);
        }
    }
```

Since we are formulating a "top down" design (starting with the last period of production), we have two principle data that can vary in the node: the amount we want in inventory ahead of the period (`invAhead`), and the amount we propose to make during the period (`make`). Remember (again), that not both of these will be non-zero. Both could be zero, but if we make any production, there should be zero inventory ahead (and vice versa).

Our `firstChild` node:

```
public PlanNode firstChild()
        {
            int prevPeriod = period - 1;

            if (prevPeriod < 0 ||
                componentPlan.solution != null)
            {
                doLeaf();
                return null;
            }

            int toMake = invAhead + Schedule.schedule[prevPeriod].demand;

            PlanNode child = new PlanNode(this, prevPeriod, toMake);
            child.invAhead = 0;

            if (prevPeriod != 0)
                child.componentPlan = Plan.fetch(prevPeriod-1, 0);

            return child;
        }
```

Remember that we start with the last period at the root node, and so a `firstChild` will represent the previous period (we go backwards in time as we descend the graph from the root). That child can either be a "make" node, wherein we have no inventory ahead of it, or it can be an "inventory" node, wherein we make no production during the period.

We have elected to make all `firstChild` nodes, "make" nodes. The `this` node (the parent of the `firstChild` we are making) specifies how much it wants in inventory (`invAhead`). Our new node must make that, and its own production demand. Since it is making production, its own `invAhead` must be zero. Note that for a "make" node, there are no alternatives as to how much to make. It is completely determined by what its parent wants in inventory, and how much it must make to satisfy its own period's demand.

The `componentPlan` is the object we want to put in our dictionary, and reuse if the problem is reencountered. As such, it must allow us to prune the tree (create no children under the current node) if it has already been solved. Our `componentPlan` just represents the `firstChild` node that we will create if the plan has not already been calculated. It is completely characterized by the period (one less than the current period), and the amount of inventory desired ahead of the current period.

Our pruning logic, `doLeaf`, is about the same as with Knapsack. It is invoked if we need not create a child node (because the `componentPlan` has been calculated, or because we are at the bottom of the graph). The C# code is:

```
53  public void doLeaf()
54          {
55              //DFS has bottomed out at a leaf.
56              //see if we have a better solution to a component
57              PlanNode aNode = this;
58              while (aNode.parent != null)
59              {
60
61                  Schedule sched = Schedule.schedule[aNode.period];
62
63                  int nodeCost = aNode.cost();
64
65                  Plan parentComp = aNode.parent.componentPlan;
66
67                  if (nodeCost < parentComp.cost ||
68                      parentComp.solution == null)
69                  {
70                      //aNode has a better solution to the parent's
71                      //component problem, so we fix the parentComp
72                      parentComp.cost = nodeCost;
73                      parentComp.solution = aNode;
74                      aNode = aNode.parent;
75                  }
```

```
                else
                    break;
            }
        }
```

You can see that if follows the pattern of the corresponding method in our Knapsack problem.

The code to make `nextSibling` nodes:

```
        public PlanNode nextSibling()
        {
            if (theParent == null || period == 0)
                    //is root. no siblings;  or
                    //else, pd=0 => no choice but to make prodn,
                    //which firstChild handled
                return null;
                //there are only two children: the firstChild, and one sib
                //firstChild makes prodn, sib makes 0 prodn

            if (!isFirstChild) //we already made the sib
                return null;

                //nextSib is an "inv node". It will
                //make nothing, but assume its requirements are all in
                //invAhead

            PlanNode sib = new PlanNode(parent, period, 0);
            sib.isFirstChild = false;

                //since we make nothing in sib,
                //sib must have invAhead enough to make inv for
                //parent, and to fill demand of the sib.

            sib.invAhead = parent.invAhead +
                Schedule.schedule[period].demand;

            sib.componentPlan = Plan.fetch(period - 1, sib.invAhead);

            return sib;
        }
```

As the comment indicates, under a given node, we have but two children (which are alternatives for the previous period): either a node that makes production (which we made the `firstChild` node) or a node that does not produce, but has inventory ahead (this is our one and only `nextSibling` node under its parent node).

The code to report the solution (`asString`) and to do the DFS (`solve`) is:

```
        public string asString()  //return answer as a string
        {
            string ans = "";
            PlanNode plan = root.componentPlan.solution;
            int cost1 = root.componentPlan.cost;
            int cost2 = 0;

            while (plan != null)
            {
                ans +=
                    "period " + plan.period.ToString() +
                    " inv " + plan.invAhead.ToString() +
                    " prodn " + plan.make.ToString() +
                    " cost " + plan.pdCost().ToString() +
                    " demand " + Schedule.schedule[plan.period].demand.
                        ToString() +
                    Environment.NewLine;
                cost2 += plan.pdCost();

                if (plan.componentPlan != null)
                    plan = plan.componentPlan.solution;
                else break;
            }

            if (cost2 != cost1)
                throw new ApplicationException("costs do not add");
            return ans + "total cost " + cost2.ToString();
        }

        public int solve()
        {
            Plan.dictionary.Clear();

            Graph<PlanNode> graph = new Graph<PlanNode>(root);
            int nodes = 0;

            foreach (PlanNode pln in graph.depthFirst())
            {
                nodes++;
            }

            return nodes;
        }
```

The answer to our problem is:

nodes visited: 17

period 3 inv 5 **prodn 0** cost 5 demand 5

period 2 inv 14 **prodn 0** cost 14 demand 9

period 1 inv 0 **prodn 19** cost 47 demand 5

period 0 inv 0 **prodn 3** cost 13 demand 3

total cost 79

Figure 5.2 shows how the nodes are created during the DFS:

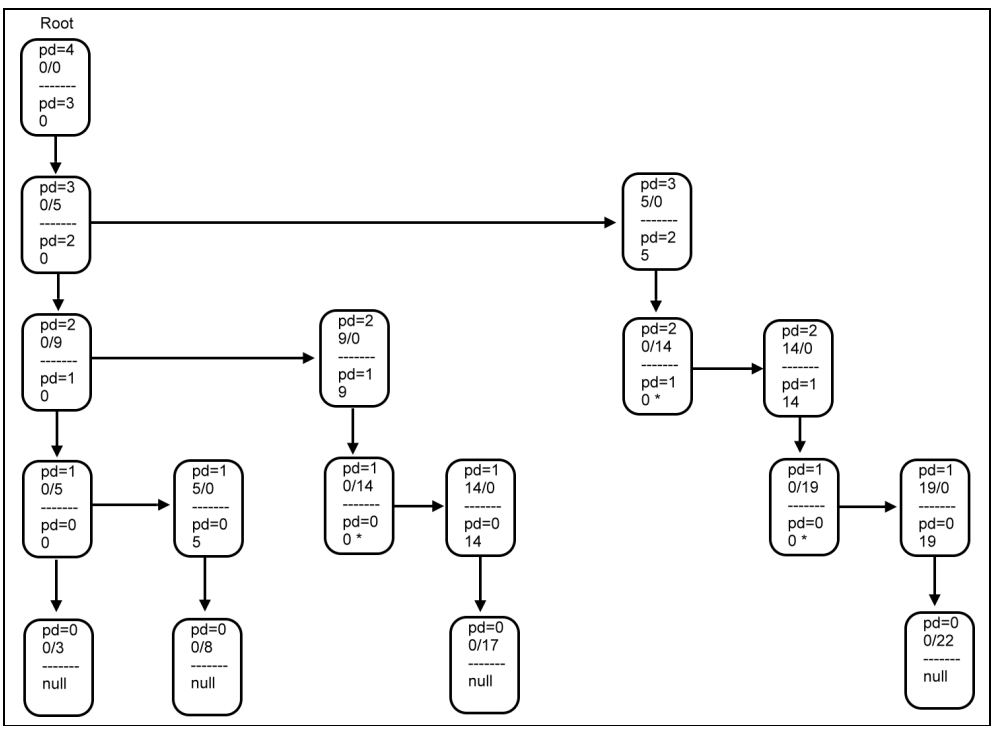

Figure 5-2. Lot Size node generation during DFS

This is a complete graph of all nodes. The top half of each node shows the period, and the `invAhead`/`make`. The bottom half shows the `componentPlan`'s `period` and its `makeForInv`. The `componentPlan`s that are marked with an '*' are those that will be in the dictionary when we look them up (and are thus reused). You can see that

each node has but two children: the first is a "make" node, the second is an "inventory" node.

It is straightforward to add Branch and Bound logic to our design. Just figure the cost of a partial solution. If it is larger than a complete solution (covering all 4 periods) already obtained, there is no reason to extend the search to prior periods.

Summary of the Dynamic Programming Pattern

- You need to break your problem into "stages", and the Principle of Optimality must be satisfied.
- You need to make nodes that contain "subproblems". Children of the node should "solve" the subproblem.
- You should table the subproblems and try to look them up instead of recreating them. You should prune the search (return null in the `firstChild` method) if the subproblem can be found in the table.
- You need to insure that reuse is likely to occur. Otherwise, another design might be more appropriate.
- It takes imagination to create a DP design.
- Once a leaf node is reached, you need to propagate subproblem solutions upward in the graph. You stop when a newly calculated solution is not better than one already cached in a node.
- Consider including B&B logic in the nodes.

6 *Breadth First Search*

BREADTH FIRST SEARCH (BFS) is another graph searching algorithm supported by SEL. It is defined as follows:

1. Let a node in the graph be distinguished as a *root node*. Define it as the only member of a list: **generation 0 nodes**. Return the root node as the first node in the search.
2. Put all (immediate) successors of generation `n` nodes in a list: **generation n+1 nodes**. Return each generation `n+1` node, sequentially, in the BFS, after all of the generation `n` nodes have been returned.
3. Repeat 2 until all nodes have been returned by the search.

The SEL obtains all the successors of a node, N, by setting `X = N.firstChild()` and then repeatedly invoking `X=X.nextSibling()` until the latter returns `null`.

In contrast to DFS, BFS does not return a node of depth `x` until all nodes of depth less than `x` have been returned (depth is defined as the distance from the root, in terms of number of nodes in the shortest path from the root to the given node). DFS, remember, seeks to return a successor to the current node until it reaches maximum depth, and must backtrack.

You can do a BFS without altering the methods you used to do a DFS. For our 8-queens problem, this would be:

```
foreach (Queen q in queenGraph.breadthFirst())
{
    nodesSearched++;

    if (q.row == Queen.max)
    {
        makeSolution(q);
       break;
    }
}
```

The only difference from our DFS solution is that we invoke the Graph's `breadthFirst` iterator instead of the `depthFirst` one.

The number of nodes that must be kept in memory is usually larger for BFS than for DFS (this is its main defect over DFS). We must keep the latest generation of nodes in memory. Often, as we go deeper into a graph there are more and more nodes at the same depth. For some graphs there will be too many nodes at the leaf levels, and memory will be exhausted before we can complete the search.

Because we also keep a parent reference in each node (so we can trace back to the root from the leaf, to obtain a total solution), we must keep all nodes in memory that can be part of a complete solution. This means that all the nodes in the current generation's parent chains are also kept in memory. Later in this chapter, you will see some techniques to prune the sometimes explosive growth of a BFS.

Figure 6.1 shows how BFS operates on our 8-Queens problem.

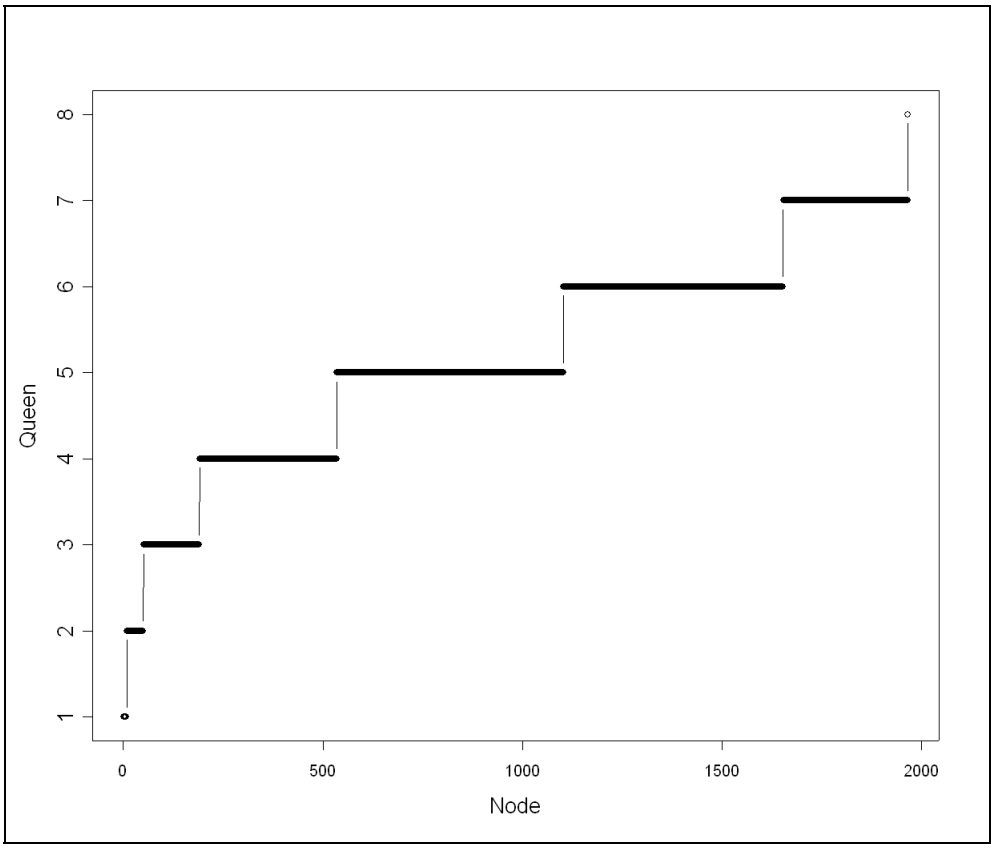

Figure 6-1. 8-Queens BFS node generation

The Y-axis is the row a queen is placed on. The X-axis is a sequential number that increases by one as nodes are generated, and serves as a tag for the node. You can see the generations of nodes clearly; there are 8 of them, and they are generated sequentially.

Nodes 1-8 represents queens on row 1, one per column. Subsequent rows have more than 8 queens on them. How can this be? Each queen represents a possible partial solution. Thus 2 nodes on (say) row 3 column 3 will be in different partial solutions (if we follow them up to their root, we will find some difference in the chains).

At row 5 we have roughly 600 nodes for generation 5. All of these are potential partial solutions and in memory at the same time when we build this generation of nodes.

Nodes in their paths up to the root are also in memory. However, some will be dropped by the C# garbage collector as they are eliminated when we try and place queens on subsequent rows. Contrast this with our DFS solution where we never needed more than 8 nodes in memory at the same time. At about node 2000 we place a queen on row 8, for our final solution (we quit at that point).

Compare this plot with the one for DFS 8-Queens in Chapter Three.

So, what good is a BFS? The techniques we discuss next allow us to make use of the nodes in memory to guide our search. This is in contrast to DFS, where our only opportunity to guide the search was by doing an initial sort of the data before the search began (as in Knapsack), and in opportunistic pruning if we were lucky enough to encounter a good solution early in the search (as in Branch and Bound).

Best-First

In our DFS for Knapsack, we had no alternative but to accept the nodes in the order in which they were presented. With BFS, we can order the nodes of a generation anyway we please. They are all in memory at the same time and can be examined. After they are ordered, we can process them to continue the BFS to the next level.

The version of BFS that orders each generation of nodes before they are returned to the application is called *Best-First*. Remember that since this is a version of BFS, all nodes of depth `n` are returned before those of depth `n+1`.

It is easy to invoke a best-first search on our B&B Knapsack problem. The only additional consideration is to tell the Graph how to order the nodes. This is done as a parameter to the constructor for the Graph:

```
11      public class KSproblemBFS: KSproblem
12      {
13          public KSproblemBFS(int cap, List<Ingot> ingots):
14              base(cap, ingots)
15          {
16          }
17
18          public int nodeCompare(KSnode node1, KSnode node2)
19          {
20              return node1.estimateWorth().CompareTo(
21                  node2.estimateWorth());
22          }
```

```
23
24          public void solveBestFirst()
25          {
26
27              KSnode root = new KSnode(null, 0, 0, 0, 0);
28
29              Graph<KSnode> graph = new Graph<KSnode>(root,
    nodeCompare);
30
31              foreach (KSnode ksn in graph.bestFirst())
32              {
33                  nodes++;
34                  if (ksn.ingot == Ingot.ingot.Count - 1)
35                  {
36                      //leaf node: possible solution
37                      if (best == null || ksn.worth > best.worth)
38                      {
39                          best = ksn;
40                      }
41                  }
42              }
43          }
44      }
```

The code is the same except for our method `nodeCompare`, the `Graph` constructor employed, and use of the `bestFirst` iterator (instead of the `depthFirst` iterator). The node class and the body of the iterator loop are unchanged.

The second parameter to the Graph constructor is of type `Comparison<T>.` This type, a delegate, is defined in the dotNet framework and you can consult its help file there. The parameters are each of type T. The returned integer is less than zero, zero, or greater than zero according to whether the first parameter is less than, equal to, or greater than the second (as with the `CompareTo` method).

We are using the "gold dust" heuristic to sort the nodes. It is an estimate as to how good the knapsack will be at the leaf node, if we expand the current partial solution.

For efficiency, it would have been better to put an `estimatedWorth` variable in the node itself (set when the node was constructed) instead of invoking the method `estimateWorth()` for each compare.

In the B&B design for Knapsack, we did a presort. That ordered the list of ingots, lowest density first. We don't do the sort in our Best-First design, so

the ingots remain in their original order: [gold, silver, copper, lead], or highest density first. The results of the `bestFirst` search are that 77 nodes are visited. If we reinstate the sort (so that lowest densities are first in the ingot list), we visit 488 nodes with `bestFirst`.

We cannot beat the DFS version of B&B, with its optimal sort (low density first; it visited only 11 nodes), because it takes longer for BFS to reach a leaf node: it must create all of the intervening generations of nodes before it reaches the leaf depth.

This exposes the main defect of best-first search. Even though we sort each generation, we still process every node in every generation. There is no inherent pruning of nodes. If realizing a complete solution depends on reaching the deepest level of the graph, no complete solutions can be generated until all nodes of all previous generations are visited.

This also suggests that adding B&B logic does not help much in a Best-First search. If we are almost done expanding *all* partial solutions to their leaf node, before any *one* of them reaches its leaf node, we do not have a complete solution against which to compare partial solutions. Early pruning is not possible in that case, and our search is very nearly exhaustive.

The next two techniques will prune nodes to reduce the search space.

Greedy Search

Greedy search is a kind of best-first search with a twist: at each generation we keep only the single best node. This means that the maximum number of nodes we will have in memory is the path from one leaf node to the root. Furthermore, since there is no backtracking, the maximum number of nodes generated is the sum of the number of (immediate) successors of the nodes in that path.

Thus Greedy Search takes the prize for speed and minimal memory use. Of course it often fails to find the best solution because it throws away so many possibilities. In many cases, however, the solution it comes up with is acceptably close to the optimal.

Greedy Search is also called “hill climbing”, since it selects what looks like the single best path from its current position. One technique in hill climbing is to take the path that is going up most steeply. Of course you might hit a

"local maximum" that is not at the top of hill, but for which all paths lead down (presumably, before at least one goes up again). Greedy Search has a comparable problem: we may throw away the best path and cannot recover when we do so.

We are going to examine the Traveling Salesman Problem (TSP) with 26 cities. To do an exhaustive search would require examining 25! (or over 1.5 * 10^{25}) different tours. This is far too many nodes for an exhaustive DFS. Furthermore, it is not clear that a B&B solution would help much. We could stop the DFS when the tour length exceeded our current best, but we would still have to generate many nodes for each tour before that happened. There seems to be no way to sort the nodes beforehand so that we get a good tour early in a DFS either.

To review TSP: We have a number of cities, one of which is designated as the start. We must construct a tour that visits each city exactly once before returning to the start city. Our problem is to minimize the length of the tour. The designs below will all use the same 26 cities so that we can compare their results.

TSP, VERSION 1, GREEDY SEARCH

Our City class is:

```
45      public class City
46      {
47          public string name;
48          public Point coords;
49          public int cluster = 0;
50          public static List<City> allCities = new List<City>(26);
51
52          public City(string nameP, int x, int y)
53          {
54              name = nameP;
55              coords = new Point(x, y);
56              allCities.Add(this);
57          }
58
59          public static int completeCircuit(List<City> citiesP)
60          {
61              if (citiesP.Count == 0)
62                  return 0;
63              City lastCity = citiesP[citiesP.Count - 1];
64              citiesP.Add(citiesP[0]);
65              return lastCity.distSq(citiesP[0]);
66          }
```

```

        public static int sumLegSq(List<City> cities)
        {
            if (cities.Count == 0)
                return 0;
            int dist = 0;

            City prev = cities[0];
            for (int i = 1; i < cities.Count; i++)
            {
                dist += prev.distSq(cities[i]);
                prev = cities[i];
            }
            return dist;
        }

        public int distSq(City other)
        {
            return distSq(other.coords);
        }

        public int distSq(Point other)
        {
            return (coords.X - other.X) * (coords.X - other.X) +
                (coords.Y - other.Y) * (coords.Y - other.Y);

        }

    }
```

We keep all the cities in the static list, `allCities`. In order to make a tour, we need a list of cities that ends with starting city (which is `allCities[0]`). That is the function of `completeTour`, which returns not only the completed tour (by updating the input list), but also the "distance" added by completing the tour.

Rather than calculating the true distance of a tour (which would require taking square roots), we calculate the `sumLegSq`, which is the sum of the squares of the distances for each leg of the tour. Since if we minimize this quantity, we will have minimized the true distance, it is an acceptable performance optimization.

To find the square of the distance of a leg, we use the method `distSq`.

Our greedy search will have a root node for the start city. The next generation consists of depth-one nodes, one for each of the cities that are left. Here is the first part of our node class:

```
    public class TSPnode: IGNode<TSPnode>, IComparable<TSPnode>
    {
        public City thisCity;
        public List<City> citiesLeft;  //after thisCity removed
        int indx = -1;                  //into parents list of citiesLeft.
                                        //-1 means the root node
        public int sumLegSqToRoot = 0;
        TSPnode theParent = null;

        public TSPnode(City cityP, List<City> leftP, TSPnode parent)
        {
            thisCity = cityP;
            citiesLeft = leftP;
            theParent = parent;
        }

        static List<City> cloneCities(TSPnode node)
        {
            List<City> cities = new List<City>(node.citiesLeft.Count);
            foreach (City c in node.citiesLeft)
                cities.Add(c);

            return cities;
        }
}
```

Each node adds a city (`thisCity`) to the tour, which will be represented by the parent chain from a leaf to the root. I.E. a partial tour is represented by the parent chain of the current node. The cities that are not yet in the tour and must be added in subsequent `firstChild` nodes are kept in `citiesLeft`. The current tour length (in terms of the sum of the squares of the legs in the tour) is kept in `sumLegSqToRoot`.

As usual, a `nextSibling` node will represent an alternative to its sister. This means we take a different city from the `citiesLeft` to form the new node. The `indx` is used to pick the next city for the sibling.

We will need the `cloneCities` method when we construct new nodes.

Here is the `firstChild` code:

```
public TSPnode firstChild()
        {
            if (citiesLeft.Count == 0)
```

```
                return null;

            List<City> cities = cloneCities(this);
            City aCity = cities[0];
            cities.RemoveAt(0);
            TSPnode child = new TSPnode(aCity, cities, this);
            child.indx = 0;
            child.sumLegSqToRoot = sumLegSqToRoot +
                child.thisCity.distSq(thisCity);

            return child;
        }

```

We just take the first available city and add it to the current path, updating the `sumLegSqToRoot`.

The `nextSibling` method will pick an alternative city to add to the route:

```
        public TSPnode nextSibling()
        {
            if (indx < 0)
                return null;  //this is the root; no sibs

            int newIndx = indx + 1;
            if (newIndx > parent.citiesLeft.Count - 1)
                return null;

            List<City> cities = cloneCities(parent);
            City aCity = cities[newIndx];
            cities.RemoveAt(newIndx);
            TSPnode sib = new TSPnode(aCity, cities, parent);
            sib.indx = newIndx;
            sib.sumLegSqToRoot = parent.sumLegSqToRoot +
                sib.thisCity.distSq(parent.thisCity);

            return sib;
        }
```

You should verify that there can be no loops in either the `firstChild` or the `nextSibling` chains.

Because we chose to inherit from `IComparable`, we need to implement that interface:

```
        public int CompareTo(TSPnode other)
        {
            return
    this.sumLegSqToRoot.CompareTo(other.sumLegSqToRoot);
        }
```

```

        public bool Equals(TSPnode other)
        {
            if (other == this)
                return true;
            return this.CompareTo(other) == 0;
        }
```

The class we will use to solve the TSP problem begins:

```
    public class TSP_BFS
    {
        TSPnode root;
        public int nodesSearched = 0;
        public int tourSumLegSq = 0;

        public TSP_BFS(TSPnode rootP)
        {
            root = rootP;
        }

        public int nodeCompare(TSPnode first, TSPnode second)
        {
            return first.CompareTo(second);
        }
    }
```

We will use the `nodeCompare` method to pass to the Graph constructor.

The method in this class that actually solves the problem is:

```
        public void solveGreedy()
        {
            Graph<TSPnode> graph = new Graph<TSPnode>(root, nodeCompare);
            foreach (TSPnode node in graph.greedy())
            {
                nodesSearched++;
                if (node.citiesLeft.Count == 0) //last node
                {
                    solutionNode = node;
                    tourSumLegSq = solutionNode.sumLegSqToRoot +
                            root.thisCity.distSq(solutionNode.thisCity);
                    return;
                }
            }
        }
```

You can see this is like the best-first solution except that we have a new iterator supplied by the SEL, `greedy()`. The code that calls `solveGreedy` makes the root and outputs the answer thus:

```
            TSPnode root = new TSPnode(startCity, City.allCities, null);

            TSP_BFS tsp = new TSP_BFS(root);

            TSPnode bestNode = null;
            int bestDist = 0;

            tsp.solveGreedy();
            bestNode = tsp.solutionNode;
            bestDist = tsp.tourSumLegSq;

            List<City> route = new List<City>(10);

            TSPnode node = bestNode;

            while (node != null)
            {
                route.Add(node.thisCity);
                node = node.parent;
            }

            route.Reverse();

            City.completeCircuit(route);

            foreach (City city in route)
            {
                outText.Text += city.name + "-" +
                    city.category.ToString() + " ";
            }

            outText.Text += Environment.NewLine +
                "distanceSq is " + bestDist.ToString();
```

We recover the route by tracing the leaf node backwards to the root, and placing these nodes in a new list (`route`). We reverse the list, and complete the tour by adding in the start city (via `completeCircuit`).

Figure 6.2 is a picture of the solution for our 26 cities.

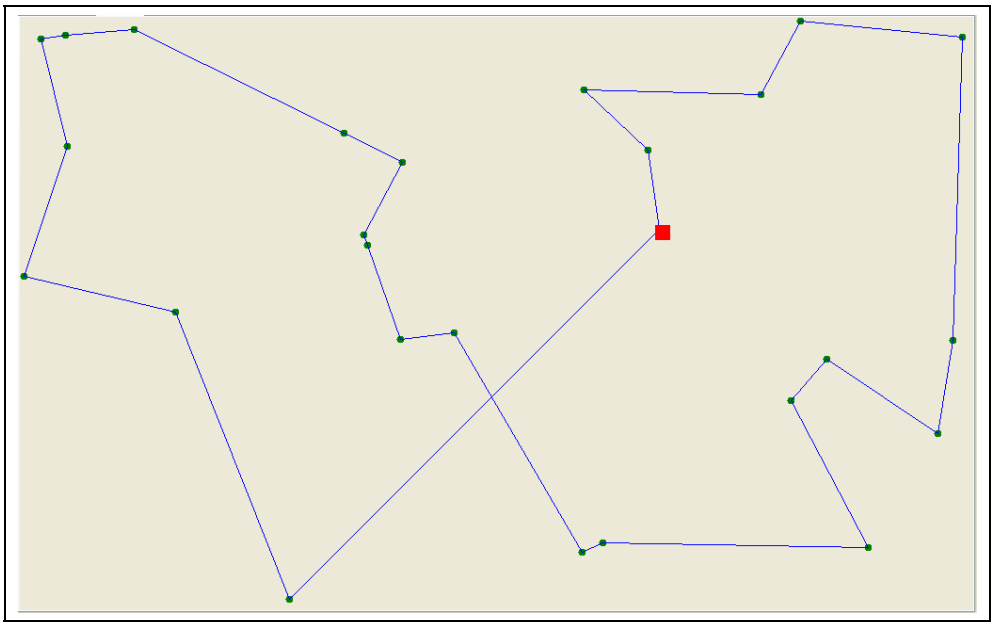

Figure 6-2. TSP, Version 1, Greedy Solution

The square is the start city. You can see that the route is generated by picking the closest city to the current one. It looks pretty good until we come to connect the last city back to the start. A good rule of thumb is that a tour can be improved if it has any lines crossing. As you might expect, our last city is pretty far from the start, and that is one reason why the tour is less than optimal.

The "sum leg square" distance of the tour is 1,213,892. The best tour we have been able to construct solves this problem with a comparable distance of (roughly) 877,060. This is achieved by Simulated Annealing, discussed in a later chapter.

Thus our greedy search is within about 62% of that solution. Not very good, but we visited only 326 nodes to achieve it.

Beam Search

Best-first is exhaustive; greedy search radically prunes all but one node at each generation. Beam Search is a compromise. The designer picks how many nodes to save at each generation.

TSP, VERSION 2, BEAM SEARCH

Our design is exactly the same except for the code that solves the problem:

```
        public void solveBeam(int cutoff)
        {
            tourSumLegSq = 0;
            solutionNode = null;
            nodesSearched = 0;

            Graph<TSPnode> graph = new Graph<TSPnode>(root, nodeCompare,
                cutoff);
            foreach (TSPnode node in graph.beam())
            {
                nodesSearched++;
                if (node.citiesLeft.Count == 0) //last node
                {
                    if (solutionNode == null)
                    {
                        solutionNode = node;
                        tourSumLegSq = solutionNode.sumLegSqToRoot +
                            root.thisCity.distSq(solutionNode.thisCity);
                    }
                    else
                    {
                        int newDist = node.sumLegSqToRoot +
                            root.thisCity.distSq(node.thisCity);

                        if (newDist < tourSumLegSq)
                        {
                            solutionNode = node;
                            tourSumLegSq = newDist;
                        }
                    }
                }
            }
        }
```

This method is also in the TSP_BFS class we used for the `solveGreedy` method. Because we obtain more than one solution, we need to keep the best one, which accounts for the extra code.

The Graph constructor needs a third parameter, the number of nodes to keep at each generation. The iterator is now `beam()`, which is supplied by the SEL.

Our node class does not change at all.

The results of the search, with various cutoffs, is shown in a plot, figure 6.3:

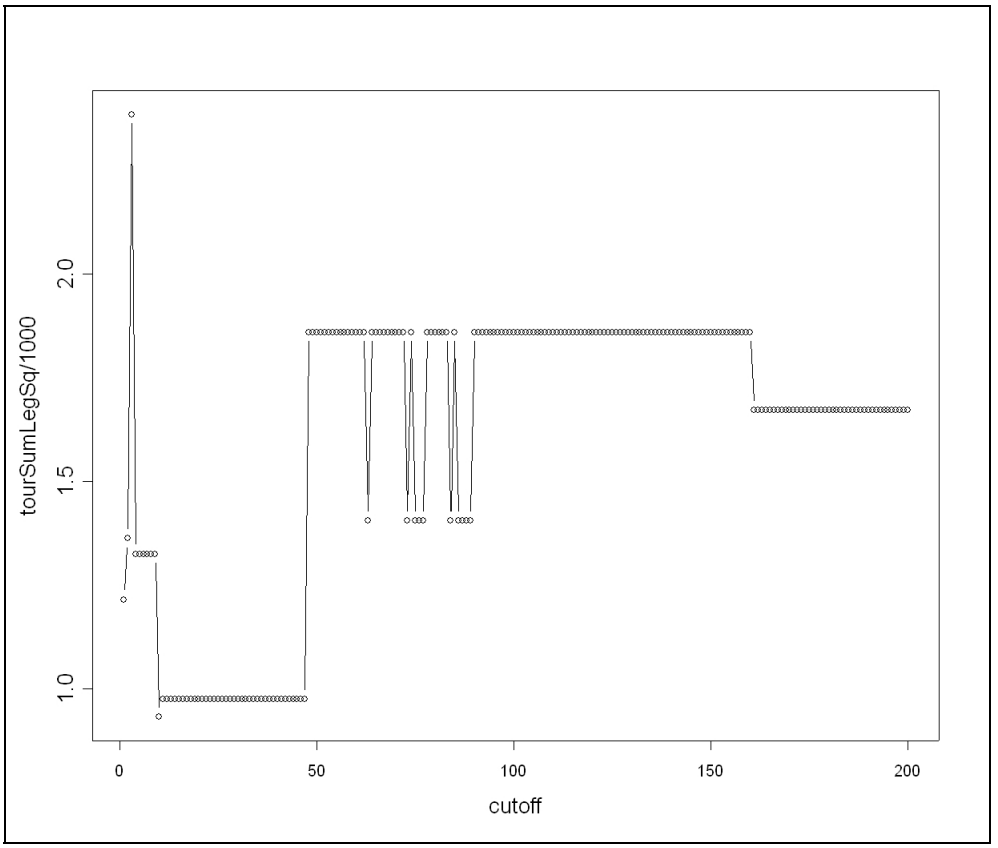

Figure 6-3. TSP Beam Search, Version 2

With a cutoff of one, we have the same search as a greedy search. The best results, with a `tourSumLegSq` of 931,522, occurs when the cutoff is 10. With a cutoff of between 11 and about 48, we get a tourSumLegSq of 973,797.

The number of nodes visited increases linearly with the cutoff. If we increment the cutoff by one, we increase the number of nodes visited by about 300, to obtain about 59K at a cutoff of 200.

Note that a larger cutoff does not necessarily produce better results.

An example will explain this latter point. Suppose we are keeping 100 nodes at each generation. Let's suppose each node has an average of 20 successors

that will be part of the next generation. Suppose a fourth generation node (5-city path) has distance of (say) 500. While not the shortest 5-city path in generation four, let's presume it is part of the optimal tour.

At the fifth generation there might be as many as 2000 nodes (100 * 20) of which we keep the best 100. Our fourth generation node, which is assumed to be part of the optimal tour, may survive to the fifth generation because it is among the best 100 nodes (shortest distance) of the 2000 nodes.

Now suppose instead we are keeping 200 nodes at each generation. Our extension of the 5-city path to the next generation must compete with 4000 nodes in the fifth generation (200 * 20). While it might have been among the best 100 nodes of 2000, it might not be among the best 200 nodes of the 4000 (because there are 200 nodes whose paths are shorter among the 4000). So it will be crowded out because it is too long and thus will not participate in the final, completed, tour.

All this is to say that TSP is a hard problem because the best tour cannot be constructed by optimally extending partial results. If you consider two non-adjacent cities on the best tour, there may be a shorter path between the two cities (involving different intervening cities).

To show the flexibility of the Beam Search we are going to solve TSP in another way, making just a few changes to our design. The idea is to "cluster" the cities around "pivot points". These will be the midpoint of each quadrant, giving us four pivot points. We will assign a "cluster" to each city, which represents the pivot point the city is closest to.

TSP, VERSION 3, BEAM SEARCH

Here is the code:

```
            Point max = allCities[0].coords;
            Point min = allCities[0].coords;

            foreach (City c in allCities)
            {
                if (c.coords.X > max.X)
                    max.X = c.coords.X;
                if (c.coords.Y > max.Y)
                    max.Y = c.coords.Y;
                if (c.coords.X < min.X)
                    min.X = c.coords.X;
                if (c.coords.Y < min.Y)
```

```
                    min.Y = c.coords.Y;
            }

            //pivot points are centers of each of 4 quadrants.
            pivot.Add(new Point(min.X + (max.X - min.X) / 4,
                                min.Y + (max.Y - min.Y) / 4));

            pivot.Add(new Point(min.X + (3*(max.X - min.X)) / 4,
                                min.Y + (max.Y - min.Y) / 4));

            pivot.Add(new Point(min.X + (3 * (max.X - min.X)) / 4,
                                min.Y + (3 * (max.Y - min.Y)) / 4));

            pivot.Add(new Point(min.X + (max.X - min.X) / 4,
                                min.Y + (3*(max.Y - min.Y)) / 4));


            foreach (City c in allCities)
            {
                c.cluster = 0;
                for (int i = 1; i < pivot.Count; i++)
                {
                    if (c.distSq(pivot[c.cluster]) >
                        c.distSq(pivot[i]))
                        c.cluster = i;
                }
            }
```

In order to do the beam search by clustering, we need a field in each node which represents the number of times we cross from one cluster to another in a tour. We want the tour to keep to cities within one cluster for as long as possible (this is somewhat like solving 4 TSP problems, one for each quadrant, and then stitching them together). We add the following code to our node creation methods:

```
            sib.transitions = parent.transitions;
            if (sib.thisCity.cluster != parent.thisCity.cluster)
                sib.transitions ++;
```

which is placed in `nextSibling`. Similar logic is placed in `firstChild`:

```
            child.transitions = transitions;
            if (child.thisCity.cluster != thisCity.cluster)
                child.transitions++;
```

The `transitions` are accumulated, so that if we were to trace the path from a node to the root (following the parent chain), `transitions` will count the number of times we move from one cluster to another on the path.

Now we change our ordering of the nodes at each generation as follows:

```
309         public int CompareTo(TSPnode other)
310        {
311            if (this.transitions != other.transitions)
312                return
    this.transitions.CompareTo(other.transitions);
313
314            return
    this.sumLegSqToRoot.CompareTo(other.sumLegSqToRoot);
315        }
```

Our new sort has the effect of putting nodes whose path to the root contains few transitions ahead of those with more transitions. That is to say, paths that do not move back and forth between cities in different clusters will be favored over those that do. If two nodes' paths have the same number of transitions we will order the nodes by the `sumLegSqToRoot` of their path.

If we experiment with different cutoffs for this design, we get the plot in figure 6.4.

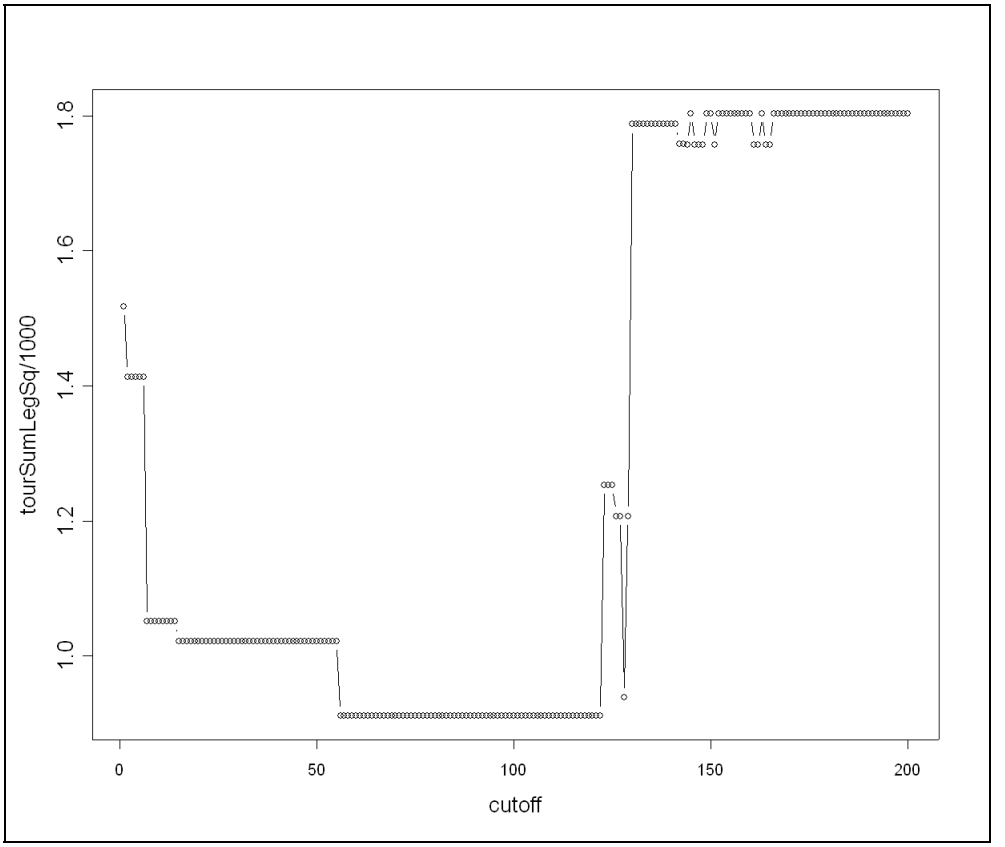

Figure 6-4. TSP, Version 3, Beam Search

The best result is a `sumLegSq` of 911,068, occurring at about cutoffs 55 through 125. This is a tad better than our first design, but still not quite as good as the Simulated Annealing result.

Note again, that a larger cutoff is not necessarily best. If we look at the tour when the cutoff is 200 we would see that we have 5 transitions, whereas the best tours have only four.

How can this happen? The problem is as with the previous design. The shortest partial tours with the fewest transitions (which are those that survive to later generations) might require more transitions as they are extended to the complete tour. In fact, at the last generation (using a cutoff of 200), all paths have either 5 or 6 transitions. The optimal path found with the best cutoff values (around 75) had but 4 transitions.

We included this second design to show you how flexible the Beam Search is. It shows that you can experiment with various ideas within a Beam Search, without too much additional coding. There is great power and flexibility in reordering the nodes at each generation.

A Storage Optimization

In some problems involving a BFS we do not need the parent chain from a node to the root. This can happen if we do not need to chase the parent chain to obtain our solution (because enough information is kept in a leaf node to regenerate the solution), or because we only need to determine that a solution exists, but do not have to produce it. To handle these situations we have added a get/set attribute to the `Graph` class: `removeParents.` You should set this attribute (only once) just after calling the graph's constructor. As the BFS proceeds, the graph will set a node's parent to `null` when it is no longer needed by the BFS itself. This will allow the garbage collector to remove these parent nodes. But if your application tries to follow a parent chain from such a node, it will reach `null` instead of the root node. The storage required with this optimization should not exceed two generations of nodes; hence the reduction can be substantial. Note that even with this optimization, `Graph` will insure that its calls to the methods `firstChild` and `nextSibling` will still allow you to access the parent of the "this" node.

Summary of the Breadth First Search Design Pattern

- Use BFS when you wish to control the order in which nodes are visited. If you are doing an exhaustive search another design might be more appropriate.
- Unless the search space is small, you will need to prune the search with either a Greedy or a Beam search. These searches may miss the optimal solution. Consider using another design if you have a large search space and need the very best solution.
- You will need to specify the order in which you want the nodes returned, and the number of nodes you are willing to keep in a generation (the cutoff). Both of these quantities are parameters to the Graph constructor.

- The ordering of the nodes in a generation will probably be an ordering of the partial results. This will likely require you to keep summary information in each node that represents the partial solution (gathered from the node to the root node). Remember that the delegate that does the ordering will be called many times, so it should be as efficient as you can make it.
- Learn from TSP that larger is not necessarily better when specifying the cutoff. Do not be afraid to conduct the search with different cutoff values to home in on the best results.
- If you are sure you do not need the parent chains, set `removeParents` to `true` just after constructing the graph. This can save memory, but you will not be able to recover the nodes from the leaf to the root.

Beam Search has its deficiencies. But for large search spaces where the very best solution is not required, it often combines effectiveness with simplicity of implementation.

7 *A**

WE TURN NOW to a celebrated algorithm, A* (pronounced "A-Star"). This is an optimization technique, suitable for applications that seek to minimize or maximize some parameter. Two examples of these applications from previous chapters are the Traveling Salesman, which seeks to minimize a route between a set of cities, and Knapsack, which tries to maximize the value of the ingots in the knapsack. We will call the parameter we are trying to optimize the "score".

A* searches a graph by expanding the most promising node first. It puts the node's successors on a list that is in order by "best first". Then, it picks the first node (which is the "best") on the list, expands that node, again putting the successors on the list, maintaining the best first order of the list. The list of nodes is called the "open nodes".

Nodes on the open list can be at any depth. A* will go deeper into the graph for so long as the deep nodes are best. If they become worse than some shallower node, A* can move closer to the root to find the best node to expand next. Contrast this strategy to DFS, which tries to go as deep as possible before backtracking, and BFS which will not move deeper until all nodes of a shallower depth have been expanded.

A* stops when it thinks it has arrived at the best solution.

A good analysis of A* can be found in Reference [2].

Heuristics

In this book we have used a graph to represent possible configurations, or solutions to some problem. A single solution is represented by the parent chain from a leaf node to the root. Before the chain reaches a leaf, we have a partial solution/configuration that we are expanding to an eventual leaf node.

To order the nodes on the open list, A* depends on a heuristic. This is a number that the application supplies for each node. It is our best guess as to what the final score will be if we extend the node to a leaf node.

The heuristic has to be optimistic ("admissible" is the term generally used), which means it can overstate how good the score will be, but must not understate it. So, for a minimization problem, the heuristic should return a value that is less than or equal to the (final) score. For a maximization problem the heuristic should be greater than or equal to the score that will be achieved when the node is extended to the full solution.

The heuristic must be optimistic so that A* can stop when the optimal solution is reached. Let's see how that works. Suppose we have a maximization problem and the open node list is [10,8,5,2], where the node is designated by the value of its heuristic.

Suppose that when A* expands node 10, it reaches a leaf and the final score is 9. A* realizes that the other nodes on the open list, [8,5,2], cannot possibly expand to a score better than 9 (because the heuristic is overstating how good the nodes can possibly be after expansion to a complete solution). So, A* can stop the search at this point. If node 10 generated a solution with score 7, A* would have to continue expanding the nodes on the open list until if found a solution better than, or equal to, the first one (and therefore better than all of them).

Now suppose our heuristic were pessimistic. A* could not assume that a score of 9 is better than that generated by the other nodes on the open list. Their solution scores might be of any size since a pessimistic heuristic only guarantees a floor for the score, not a ceiling. A* does not know when to stop expanding nodes on the open list.

The heuristic is usually broken into two parts. One is the actual value of the partial solution; the other is the estimate of how the value will change when the score is reached. For historical reasons, these parts are called g and h, respectively. Their sum, then, represents the heuristic that estimates the score. If h is admissible in the above sense, A* is guaranteed to return an optimal solution, but not otherwise.

NB: in the **SEL** version of A*, it is absolutely crucial that for a leaf node the heuristic return the actual value (score), and not an estimate. Otherwise, A*

might stop prematurely and not return the optimal solution. In terms of the above terminology, `h` must be 0 for a leaf node.

Figure 7.1 shows an A* graph and the order in which nodes are returned to the application.

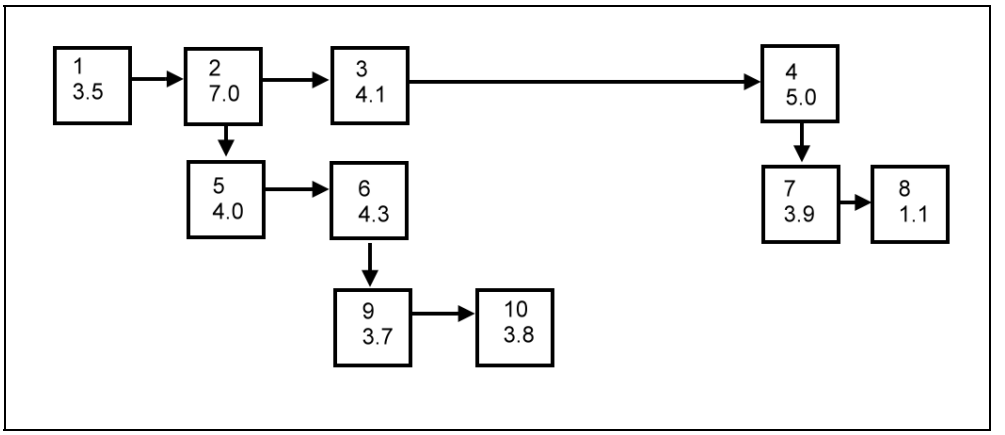

Figure 7-1. A Operation*

In figure 7.1, the top number in the node shows the order in which the nodes are returned to the application by the `Astar` iterator. The bottom number is the heuristic (estimate of the final score). Right pointing arrows represent the `firstSibling` chain, downward ones the `firstChild` chain.

One important thing to note is that all children of a node are returned in order. Thus nodes 1,2,3,4 are all children of a parent node (not shown) and are returned in that order.

After these are returned to the application, the open node list would be [2,4,3,1], which is in order by the heuristic (greatest first, assuming a maximization problem). Because node 2 has the largest heuristic it is removed from the open list and its children (5,6) are returned to the application in the order they are generated. These are also placed on the open list, in order by the heuristic, giving [4,6,3,5,1]. The node 4 is now first on the open list, so it is removed from the list and expanded. Its children are returned in order: 7,8.

Nodes 7,8 are added to the open list, which becomes [6,3,5,7,1,8]. Node 6 is expanded and its children, nodes 9, 10 are returned in that order to the

application, and added to the open list. You may wish to continue studying how nodes are returned and the open node list updated in this example

Since we have already solved the Knapsack problem with a heuristic that is "admissible" (the "gold dust" heuristic of a previous chapter), let's use it an A* design.

KNAPSACK VIA A*

We make no changes to the node design of Knapsack at all (see Chapter 4), except we can remove the B&B logic. Additionally, we remove the presort of the ingot list. These two aspects are completely taken over by A*, simplifying the solution significantly. The code that finds the solution via A* is:

```
public int compareHeuristic(KSnode first, KSnode second)
{
    //highest potential worth sorts first
    return second.estimateWorth().CompareTo(
        first.estimateWorth());
}

public KSnode solveAstar()
{
    KSnode root = new KSnode(null, 0, 0, 0, 0);

    Graph<KSnode> graph = new Graph<KSnode>(root,
       compareHeuristic);

    KSnode solution = null;

    foreach (KSnode ksn in graph.Astar())
    {
        nodes++;
        if (graph.quit(solution))
        {
            nodesToBest = nodes;
            break;
        }

        if (ksn.ingot == Ingot.ingot.Count - 1)
        {
            if (solution == null ||
                solution.worth < ksn.worth)
                solution = ksn;
        }
    }
    return solution;
```

```
        }
```

We pass the `compareHeuristic` to the graph constructor. This method must order the nodes so that the "best" is first (in a minimization problem, the one with the smaller heuristic is first. This is reversed for Knapsack, which is a maximization problem). Be careful to specify this function correctly!

For efficiency, it might have been better to store the estimated worth in each node, since the compare method is executed so often.

The iterator on the graph is called `Astar`.

In the body of the iterator loop we must save the best solution (because the heuristic is optimistic, the first solution encountered need not be the best one).

We need to ask the graph if we should quit. The SEL version of A* returns all the successors of the first node on the open node list, before it goes back to that list for the next "most favorable" node to expand. Some of these successors might be better than the current solution node, so we cannot quit until we know that none could provide a better solution than the best so far discovered. The graph makes this determination for you via the `quit` method.

This A* solution finds the optimal knapsack packing (capacity 17) after visiting 47 nodes. This does not beat our best DFS-B&B solution with presort (11 nodes visited) because A* must expand every node it removes from the open list. But remember that without the presort, our B&B solution visited 106 nodes. A* performs better in this case, and in applications where such a fortunate presort may not be available.

The accuracy of the heuristic is very important. If it does not discriminate well between nodes (i.e. it values nodes the same when they differ greatly in their "goodness"), our A* search might become close to an exhaustive search. If it greatly overvalues the nodes (even when they are close to the leaf), it must process almost all of them before it determines the best one. Again, we will have close to an exhaustive search. Note that the same comments can be made when we use the same heuristic in a DFS-B&B search.

For example, if our knapsack heuristic returns 1000 for all non-leaf nodes (and the true value for the leaf nodes, as is required by A*), we visit 110 nodes. With a bad heuristic, A* (like B&B) is sensitive to the order of the input (in this case, the ingot list).

In fact, our knapsack for A*, with the "gold dust" heuristic is still slightly sensitive to a sort on the ingot list. If we sort it by lowest density first, [lead, copper, silver, gold], we return 34 nodes (compared to the 47 without the sort). This is largely because of the successors that need to be generated and the bushiness of the tree at the top. But it is much less sensitive than B&B was to the sort (11 vs. 106 nodes returned). Remember that A* is doing what amounts to a dynamic reordering after each node is expanded. B&B cannot duplicate this logic. For some problems, A* greatly outperforms any B&B logic.

It is difficult to estimate the number of nodes that A* must keep in memory at one time. If the heuristic does not obtain the best solution quickly, the open node list will grow. Every node returned by A* results in its successors being placed on the open node list. Remember that the parent references will force all of the parent chains of the open nodes also to be kept in memory. Nodes are removed from the open list, but their successors (via the parent chain) will keep them in memory, until a leaf node is reached (which usually causes A* to terminate quickly thereafter).

Could we use a cutoff in A* as we did with Beam Search? Such a cutoff would trim the size of the open list so that it never grew beyond a certain point. The tail end of the list would be chopped, since these are sorted last via the heuristic. As in Beam Search, such a pruning might lose the very best solution, but might return a solution close to the optimum.

The problem with such a strategy is that the deeper nodes are likely to be eliminated in favor of the shallow ones. This is because the deeper we go, the more accurate the heuristic is likely to be, and we will probably retreat from the optimistic heuristic value that is still present in the shallow nodes. This will push the deeper nodes toward the back of the open node list.

If we prune the deep nodes (assumed to be at the back of the open node list), we are discarding nodes that were costly to obtain, in order to retain shallow nodes that we have little work invested in. Furthermore, the shallow nodes' estimate of the goal is likely to be suspect anyway. For this reason, it might

be better to use a Beam Search where nodes at the same depth are given an equal chance to survive. Nevertheless, you might find it interesting to experiment with a cutoff in A*.

Would it be fruitful to add B&B logic to A*? As with BFS, A* postpones finding *any* solution until it is close to returning the best one. Hence we do not have a solution in hand early in the search (as we do with DFS). Hence B&B logic is not likely to be of great value in an A* search.

We will now attack a problem that is easily solved by A*, but is difficult to solve by any of the other techniques in this book.

15-PUZZLE

The 15-puzzle was invented by Sam Lloyd. It consists of a 4 by 4 grid which contains 15 sliding tiles, and one "hole". The tiles are numbered 1 through 15. Tiles adjacent to the hole (but not diagonal to it) can be moved into the hole, which effectively "moves" the hole to occupy the former position of the tile. The puzzle starts out scrambled and the goal is to move the tiles around until they are in sequence, left to right, top to bottom. Figure 7.2 shows the layout.

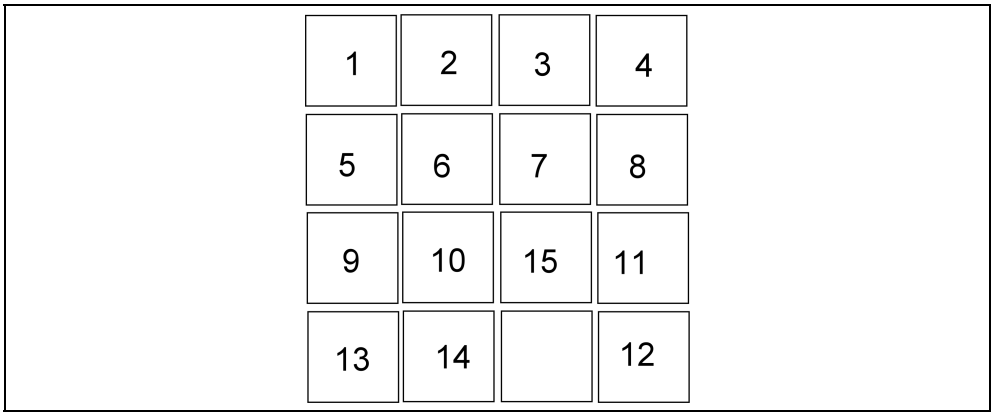

Figure 7-2. 15-Puzzle

To solve the puzzle from this position, you would move the 12 to the left, the 11 down, the 15 to the right, the 12 up, the 11 to the left, the 15 down, 12 to the left, 11 up, and 15 to the left. This amounts to "rotating" the 4 tiles about their center, clockwise, until the 15 is in the proper spot.

We are going to devise an A* solution that not only solves the 15-puzzle from any legitimate starting position, but does so in the fewest number of moves.

The node data and its constructor are defined thus:

```
1       public class SqNode : IGNode<SqNode>, IComparable<SqNode>
2       {
3           public int[,] position;
4           public Point zero;
5                       //the empty space is at position[zero.x,
    zero.y]
6           public SqNode theParent = null;
7           public List<Point> okMoves;        //is ok to swap the zero
    and the points
8           public int movesFromStart = 0;  //same as this node's
    "depth"
9           public int movesToGoal = 0;     //estimate based on
    heuristic
10
11          public SqNode(int[,] positionP, SqNode par, Point zeroP)
12          {
13              if (SqPuzzle.width < 0)
14                  SqPuzzle.width = positionP.GetLength(0);
15
16              theParent = par;
17              position = positionP;
18              zero = zeroP;
19
20              okMoves = this.generateMoves();
21              if (par != null)
22                  movesFromStart = par.movesFromStart + 1;
23              movesToGoal = this.getMovesToGoal();
24          }
25      }
```

Our puzzle is represented with an array of integers, `position,` assumed to be square (width=length). The variable `zero` marks the coordinates of the "hole" in `position`. We will generate a list of valid moves, from the current puzzle position (given in `position`), in `okMoves`. The root node in our graph will represent the starting position of the scrambled puzzle. A leaf node will be a solution (the puzzle in its solved position), and the parent chain from the root to the leaf will represent the moves we must make to get to the solution.

The optimization part of the program is to find the fewest moves, which equates to the shortest length of a parent chain from leaf to root.

The constructor calls the following method (also in `SqNode`), which is our A* heuristic:

```
        int getMovesToGoal()
        {
                //make optimistic estimate of number moves to the goal
                //we assume position is square
            int dist = 0;
            int goali, goalj, hasValue;
            for (int i = 0; i < position.GetLength(0); i++)
                for (int j = 0; j < position.GetLength(0); j++)
                {
                    hasValue = position[i,j];

                    if (hasValue == 0)
                    {
                                //the "empty" square in the puzzle solution
                        goali = position.GetLength(0)-1;
                        goalj = position.GetLength(0)-1;
                    }

                    else
                    {
                        goalj = (hasValue - 1) % position.GetLength(0);
                        goali = (hasValue - 1) / position.GetLength(0);
                    }

                    dist += Math.Abs(i - goali) +
                        Math.Abs(j - goalj);
                }
            return dist;
        }
```

This method measures the distance (vertical plus horizontal) from each tile to its “home square”, where it is supposed to end up in the solution. Obviously, it is not likely that we can move a tile to its home square in this small number of moves. Hence our heuristic is optimistic in that it underestimates the number of moves necessary to achieve the solution.

On the other hand, you can see intuitively that the sum of the distances of the tiles from their home squares does give a feel for how far away the position is from the solution. Thus it discriminates well between two positions.

The next method, also in `SqNode` class will generate all valid moves in a position:

```
public List<Point> generateMoves()
        {
                    //legal swaps between the zero and adjacent squares
            List<Point> okMoves = new List<Point>(4);

            if (zero.X - 1 >= 0)
                okMoves.Add(new Point(zero.X - 1, zero.Y));
            if (zero.X + 1 < position.GetLength(0))
                okMoves.Add(new Point(zero.X+1, zero.Y));

            if (zero.Y - 1 >= 0)
                okMoves.Add(new Point(zero.X, zero.Y-1));
            if (zero.Y + 1 < position.GetLength(1))
                okMoves.Add(new Point(zero.X, zero.Y+1));

            return okMoves;
        }
```

This just enumerates the tile positions that can be moved into the "hole". There are at most four valid moves in any position.

Our `firstChild` method is very simple:

```
        public SqNode firstChild()
        {
            if (movesToGoal == 0)
                return null;        //take no moves away from success

            while (okMoves.Count > 0)
            {
                Point move = okMoves[0];
                okMoves.RemoveAt(0);

                SqNode child = new SqNode(makeMove(move), this, move);
                if (!child.occurred())  //do not repeat a prev. position
                    return child;
            }
            return null;
        }
```

We recognize a solution when all tiles are in their home positions (`movesToGoal` is zero). If there are no `okMoves` we also return `null`.

This means that the current partial solution cannot be extended to a full solution.

The reason why this might occur is that all valid moves result in a position that has occurred before. Remember that the `firstChild` and `nextSibling` chains must not cause loops (the same node generated again on the chain). In other graph searches we were able to insure this as part of the design. In the 15-puzzle we keep the previous positions and insure we never get there again. This keeps us from moving the same tile back and forth aimlessly, and forces the moves taken to advance toward the goal.

Here is the code that keeps track of previous positions:

```
        public bool occurred()
        {

            if (this.movesToGoal == 0)  //do not keep solution from reocurring
                return false;

            //if not already in prevPos, add it and return false.
            int hit = SqPuzzle.prevPos.BinarySearch(this);
            if (hit < 0)
            {
                hit = ~hit;
                SqPuzzle.prevPos.Insert(hit, this);
                return false;
            }
            return true;
        }
```

We keep the positions in a list, `prevPos`, and do a binary search on it. In order to employ the binary search, we need to be able to order the nodes (this order is different than that imposed by the heuristic). `BinarySearch` will use the `CompareTo` function we supply as part of the `IComparable` interface. The code is:

```
        public int CompareTo(SqNode other)
        {
            for (int i = 0; i < position.GetLength(0); i++)
                for (int j = 0; j < position.GetLength(1); j++)
                {
                    if (position[i,j] != other.position[i,j])
                        return position[i,j].CompareTo(other.position[i,j]);
                }
```

```
            return 0;
        }

        public bool Equals(SqNode other)
        {
            if (other == this)
                return true;
            return (this.CompareTo(other) == 0);
        }
```

This just compares each tile value in the nodes (position by position) until two do not match. Then it returns the obvious `CompareTo` value.

The other method that `firstChild` calls, `makeMove`, will make a move, transforming the current position to a new one:

```
        public int[,] makeMove(Point move)
        {
            //return new postion that results from current pos, taking move
            int[,] newPos = (int[,])position.Clone();
            newPos[zero.X, zero.Y] = position[move.X, move.Y];
            newPos[move.X, move.Y] = 0;
            return newPos;
        }
```

We need to clone the position since a fresh copy is kept in each node.

The `nextSibling` method just picks a different valid move from the valid move list:

```
        public SqNode nextSibling()
        {
            if (parent == null) //root has no sibs
                return null;

            while (parent.okMoves.Count > 0)
            {
                Point move = parent.okMoves[0];
                parent.okMoves.RemoveAt(0);

                SqNode sib = new SqNode(parent.makeMove(move),
                    parent, move);
                if (!sib.occurred())  //do not repeat a previous position
                    return sib;
            }

            return null;
        }
```

Notice that in this method, and in `firstChild`, we remove the move taken from the parent. In this way, we exhaust the valid moves, placing each in either a `firstChild` or a `nextSibling` node.

The class that we use to solve the 15-puzzle is:

```
    public class SqPuzzle  //Puzzle based on Sam Lloyd's "15-Puzzle"
    {
        public SqNode root;
        public static int width = -1;  //length of a side; puzzle is a square
        public static List<SqNode> prevPos = new List<SqNode>(10);
        public int nodesSearched = 0;
        public SqNode solutionNode = null;

        public SqPuzzle(int[,] pos, Point zeroP)
        {
            width = pos.GetLength(0);
            if (pos.GetLength(1) != width)
                throw new ApplicationException("puzzle not square");

            root = new SqNode(pos, null, zeroP);
        }

        public void solve()
        {
            Graph<SqNode> graph = new Graph<SqNode>
                (root,
                 delegate(SqNode first, SqNode second)
                    {
                        return (first.movesFromStart + first.movesToGoal).
                            CompareTo(second.movesFromStart + second.movesToGoal);
                    });

            foreach (SqNode node in graph.Astar())
            {
                nodesSearched++;

                if (graph.quit(solutionNode))
                    break;

                if (node.movesToGoal == 0)  //solution node
                {
                    if (solutionNode == null ||
                        solutionNode.movesFromStart > node.movesFromStart)
                            solutionNode = node;
```

```
                    }
                }
            }
        }
```

This code should be clear from our previous A* problem. Note that we are optimizing the total `movesFromStart`, and that this is a minimization problem.

The delegate we pass to the graph constructor is used by A* to order the nodes on the open list. You can see that an actual value for the node (`movesFromStart`) is combined with an approximation of the moves to the goal from the current position (`movesToGoal`) to get an estimate of the total moves needed to achieve the solution, from the start position.

Notice that the sense of the compare in the delegate is reversed from that we used in our knapsack maximization problem.

The actual puzzle positions, from starting position to solution can be obtained by chasing the parent chain in the solution node, and then reversing the list of nodes.

Figure 7.3 is a plot that shows how A* goes deep into the graph until a shallow node becomes more promising.

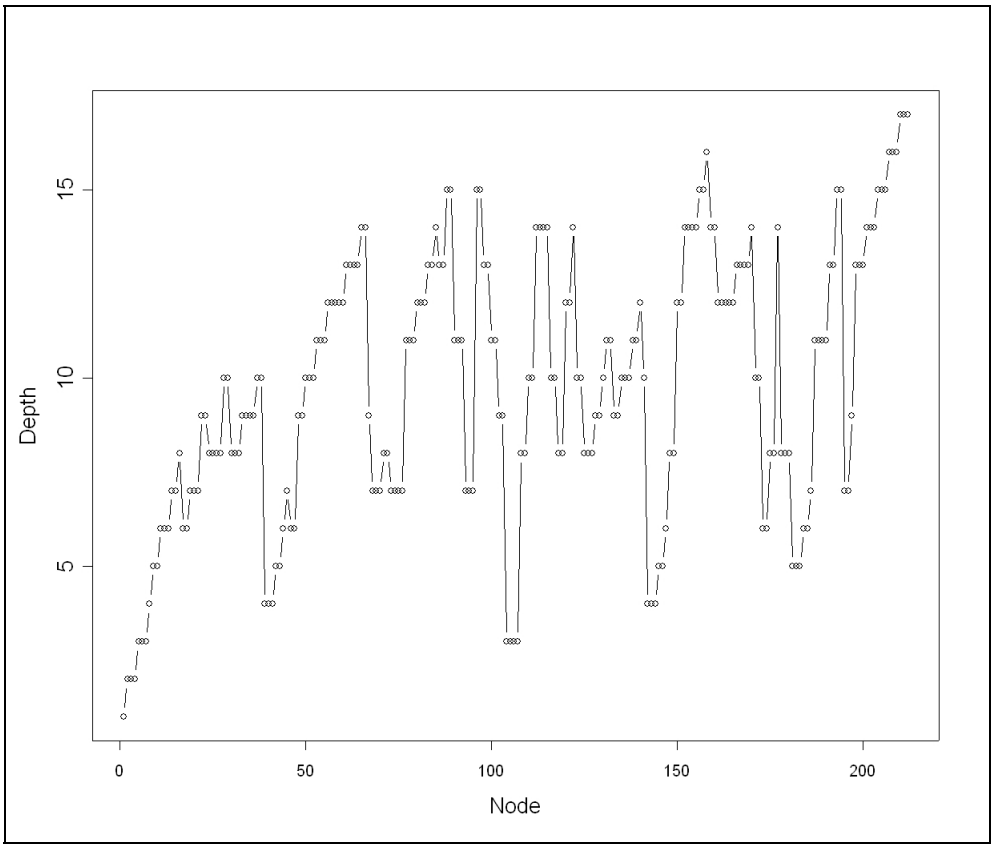

Figure 7-3. A Operating on the 15-Puzzle*

The depth is the Y-axis and is the number of moves in the partial solution represented by the node. The sequential node number is on the X-axis. The starting position of the puzzle required 16 moves to reach a solution. About 210 nodes were generated. Compare this graph with a DFS graph. Notice that the A* "backtracking" is based on when the current partial solution is no longer as promising as an earlier one (which has fewer moves from the root).

Summary of the A* Design Pattern

- A* is an optimization technique. Your problem must involve a numeric score you are trying to maximize or minimize.

- You must supply an optimistic heuristic. It can overstate the "goodness" of a node, but must not understate it. This is used as a node comparison delegate to the graph constructor.
- For a leaf node, the value of the heuristic must be the actual score of the solution.
- Make sure that your delegate orders the nodes appropriately, depending on whether you have a maximization or a minimization application.
- The more accurate the heuristic, the faster the search will be.
- Test for an optimal solution by asking the graph if you can `quit`. You will have to keep track of the best solution node returned by A* since the first solution is not necessarily the best (but the best one is usually not far behind the first one in the search).
- As in our other graph searching designs, your `firstChild` nodes proceed toward a solution (deepen the graph towards a leaf node). The `nextSibling` nodes represent alternative choices and are all at the same depth.

8 *Game Trees*

A GAME TREE is a device for finding a good strategy to win a 2-person, turn-based game. The latter phrase means that the two players take turns in making moves. In the applications we will study, the allowable moves depend only on the current position (i.e. there is no randomness, as in card and dice games). Program designs for computer play against a human in such games have been extensively studied, especially in chess. The game tree underpins the most successful engines discovered to date.

A game tree is a graph and the heart of the playing engine is a DFS with a depth bound. Thus the tree is searched to a certain depth, at which backtracking is forced. In all but the simplest games, the huge number of possible moves precludes searching the tree to the leaf nodes.

You can find much material on game trees in Reference [2].

Preliminary notions

A game tree represents the possible moves for each player in various positions. For reasons made clear later, we will name the players MIN and MAX. We will assume that MAX moves first. In our tree, the root node, and all nodes at even depth will be called MAX nodes. The others are MIN nodes. A MAX node will represent a position where it is MAX's turn to move; the MIN node is a position where it is MIN's turn to move. The nodes under a MAX node represent positions that MAX can achieve from the parent position. These nodes are MIN nodes since they represent positions in which it is MIN's turn to move.

We assume that each game has a final, numeric score. If MAX has won, the score is assumed to be positive; if MIN has won, it is negative. Zero will represent a draw. At each position, we will have a heuristic that estimates the game's final score if the game were to be taken to a conclusion from that position. Thus MAX is trying to maximize the heuristic at each node,

whereas MIN is trying to minimize it (hence the players names, MAX and MIN).

For some games, like chess, there is no inherent score when the game is over. For these games, we will just assume a very large numeric value will represent a win for MAX, a large negative number a loss for MAX, and zero a draw. The heuristic is a kind of evaluation of the position: a positive number means that MAX is winning, a negative number that MIN is winning. The size of the number indicates how large the advantage is.

Each node in the game tree will include the current position, the predicted score at the end of game, and an indicator as to whose turn it is to move.

Here is an example of a game tree (figure 9.1) for a familiar pencil and paper game:

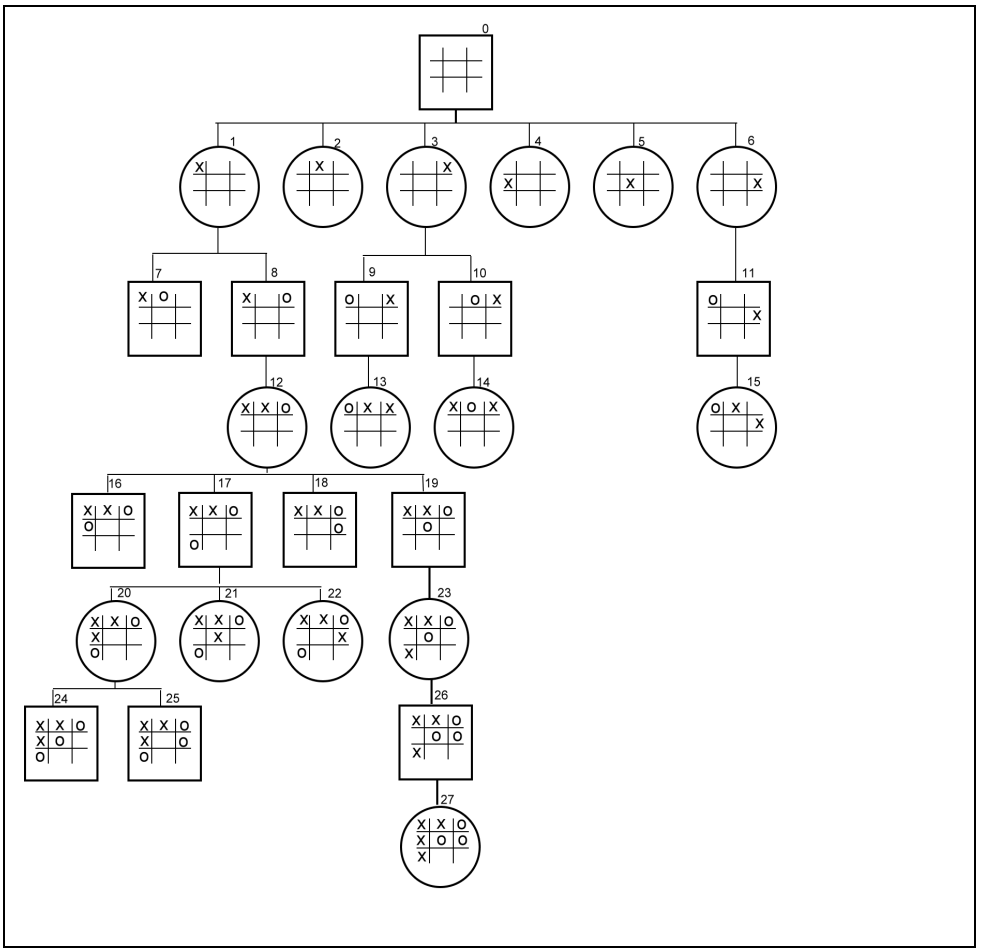

Figure 8-1. Game Tree (Many Nodes Omitted)

The root node shows an empty board. Because MAX moves first, this is a MAX node. We have represented MAX nodes by squares, MIN nodes by circles. MAX's moves on the board are shown by crosses ('X"), MIN's by circles ('O').

Under the root node, we see the possible positions that could result from the initial MAX move. There should be 9 of these, but we have shown only six.

Under some of those MIN nodes we have shown the positions resulting from some of MIN's possible moves. Each of these nodes represents the position

after one move for MAX and one for MIN. These are all MAX nodes, since MAX is to move from the position.

Each row in our graph represents all possible moves for a single player after a given number of moves have been made. The row is called a "ply", and the depth of the graph is given in plies. In the graph shown, the depth is seven plies, since the position reached represents three moves for MAX and 4 moves for MIN.

The left node on the last row, node 24, is a leaf node: MIN has won since he has three circles along a diagonal. If we scored the final result of a game as 100, 0, or -100 (for MAX win, draw, or lose), the score on this leaf node would be -100.

The score on the MIN node above that leaf node, node 20, would also be -100, since if MIN can reach node 20, he can force the game to node 24. Hence he is assured of a win if the game reaches node 20.

In this simple game, it is possible to develop the complete game tree from the initial position to all of the leaf nodes. Since each leaf node has a score determined (100, 0 or -100), it is possible to propagate all of those scores up to the root using an algorithm called minimax.

Minimax

The minimax algorithm is based on the assumption that at each position, the player to move will make the best move, given that his opponent makes no mistakes. This is the move that has the best score from that player's perspective. For MAX, it means he will pick the node with the highest score. For MIN, it means he will pick the node with the lowest score (since the game's score is always defined to be that reached by MAX at the end of the game).

To obtain the score for a node, **N**, minimax looks at all of the children nodes immediately underneath it. Suppose these are all leaf nodes and thus have scores associated with them based on the conclusion of the game (win, lose, or draw for MAX). Suppose **N** is a MIN node (MIN to move). Then minimax takes the *minimum* score of the leaf nodes immediately beneath **N**. That is the score for **N**. It is easy to see why this works: since MIN is to move, he will chose the position that leads to a win for him, a draw if a win is not available, and (if the game has a score associated with the final

position), the position with the smallest score if neither a win nor a draw is possible.

If the node **N** is a MAX node, MAX will chose the move that leads to the highest score. This is the maximum score of all the nodes underneath **N**.

Once we have the scores propagated from the leaf nodes to their parents, we use the same algorithm to propagate the score at that ply up to the parent nodes of that ply. Again, we just take either the maximum or the minimum score of the nodes underneath a node, depending on whether the node to be scored is a MAX or a MIN node.

In order for minimax to work, all nodes immediately underneath a node must be scored, in order for the parent node to be scored. Since most games will not have their game tree expanded to the leaf nodes until late in the game, we will use the heuristic evaluation as a surrogate for both the minimum and maximum of the scores underneath our maximum depth.

For example, suppose we set a depth bound of four. That means we will expand the game tree, using a depth first search, until we reach a depth of 4 (counting the root as depth zero). When we reach depth four we will assign a score to that node, N, using a heuristic; then we propagate that score upwards, via minimax. Suppose the score for N is X.

Since we are doing a DFS, we may not have yet visited all of the nodes under N's parent. But we can still process the nodes in the parent chain, from N to the root. Let P be a node in the parent chain: either N itself or some ancestor of N. We process P as follows:

1. If P has no score yet, just assign it the value X and continue up the parent chain.
2. If P is a MAX node and has a score: if that score is greater than X, stop. If it is less than X, assign it the value X and proceed up the parent chain.
3. If the node is a MIN node and has a score already: if that score is less than X, stop. If it is greater than X, assign it the score X and proceed up the parent chain.

You should be able to see that this algorithm will propagate to the root node the one score that both MIN or MAX can achieve if both players play perfectly. Either side might be able to attain a better score if the other side

picks a less than optimal move, but both are assured that they can not do worse than the root score, if they play correctly.

If an evaluation at the root were not needed after each node was generated, it would be less costly if we just propagated the score from a child to its immediate parent. Then we would wait to propagate the score from that parent upwards until all the children had been evaluated.

Assume that our game always has one human player and one computer player, and that we are writing a program to calculate the best computer move. Let's make the computer the MAX player. Then the root node will always represent the current position with the computer to move. To calculate the best move for MAX, we will do a DFS from the root to the depth bound, using the minimax technique. At the end of the search the root will have the best score MAX can achieve from the current position.

During the minimax processing we will keep track of when we change the root node's score. At that point, we will save the node (under the root) that changed the score. This will be the best move for MAX, our computer player. If we wish, we can also keep track of the "leaf" node (the one at the depth bound) that led to the root score as well. Then we will have the best moves for both players, if we chase the parent chain from the "leaf" to the root.

REVERSI

We are going to illustrate the game tree and the minimax algorithm with the game Reversi (sometimes called "Othello"). This game is played on a standard (eight by eight) chessboard. Each player has an unlimited number of flat, circular "stones" which she places on the squares of the chessboard, one at a time. The stones are colored white on one side, black on the other.

A move consists of placing one stone in a square on the board. The players are designated "white" and "black", and that is the color that they place face up when they move.

A valid move is one which "flanks" the other player. If white is to move, she must place a white stone adjacent to a black stone. The black stone can be on the same horizontal, vertical, or diagonal row with the white stone at the end. Furthermore, there must be a terminating white stone on the same row.

Thus there is a sequence of black stones terminated by the newly played white stone, and some other (already played) white stone.

After white's move, the intervening black stones are flipped to become white stones.

The "flipping" stops as soon as the first white stone in the row is encountered, even though there may be some more black stones following the terminating white stone on the same row. A newly played white stone might form two or more rows, flanking black along both rows. In this case, all of the flanked black stones are flipped.

There are no "chain reactions". I.E., a flipped stone does not cause a new flanking and thus more flips.

Black's rules for moving are the same as white's, with the colors reversed.

If a player cannot move, she passes and allows the opponent to make a move.

The game is over when neither player can make a flanking move.

When the game is over, the black and white stones are counted. The player with the most stones wins.

The initial position of Reversi is shown in figure 8.2. Black always moves first.

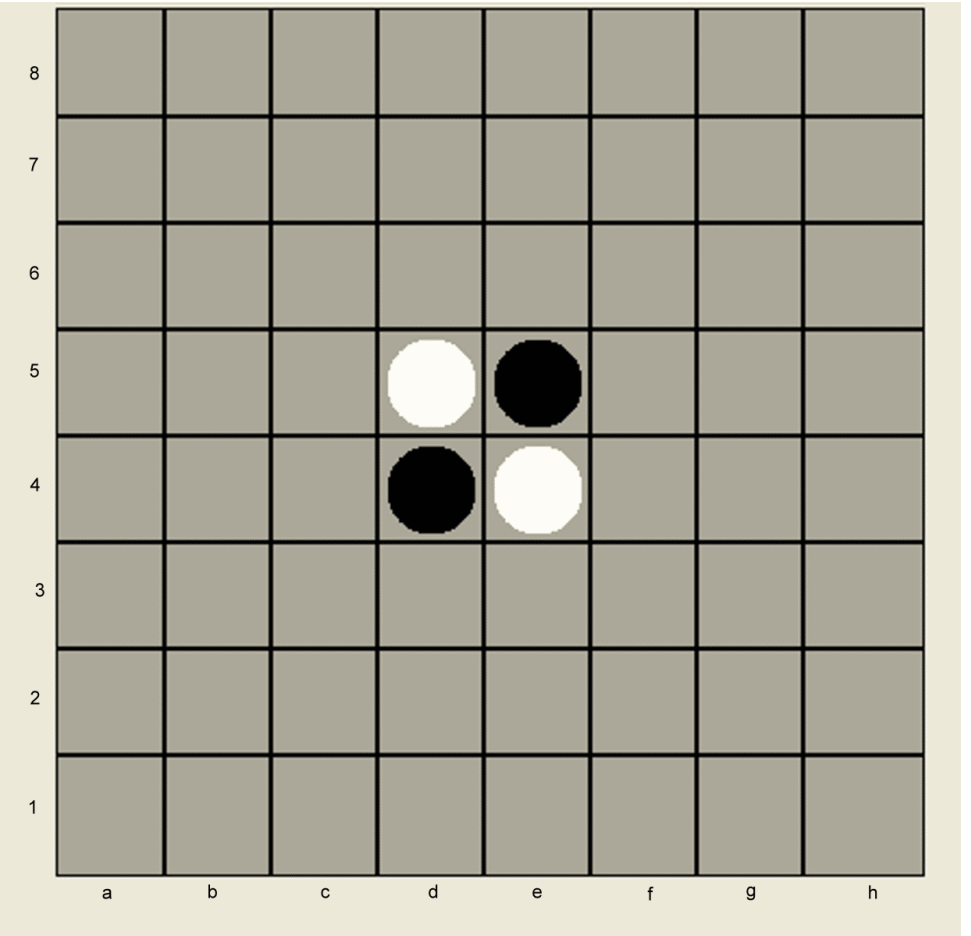

Figure 8-2. Initial Position for Reversi

The only valid moves for black are on squares c5 (that would flip d5 to black), and f4 (that would flip e4). You can find the rules and strategy for playing Reversi on the internet, along with many sites that let you play online. Reversi is of sufficient complexity that winning a game takes skill and practice.

We are going to present a C# program to play Reversi, leaving many details out (the complete code is available on the book's website).

```
public enum PIECE { BLACK, WHITE, EMPTY };

```

```
3       public class Move
4       {
5           public PIECE sideMoving;
6           public Point move;
7           public List<Point> flips = null;
8
9           public Move(PIECE side, Point mv)
10          {
11              sideMoving = side;
12              move = mv;
13          }
14
15          public Move(PIECE side, Point mv, List<Point> flipsP):
16              this(side, mv)
17          {
18              flips = flipsP;
19          }
20      }
```

A `Move` is represented by the color of the piece played, a `Point` which contains the coordinates of the square where the new piece was placed, and a list of points that indicate where on the board existing pieces must be flipped.

```
21      public class Position
22      {
23          public static int boardSize = -1;
24
25          public int computerScore = 0;
26          public int humanScore = 0;
27
28          public PIECE[,] piece;
29
30          public Position():this(boardSize)
31          {
32
33          }
34
35          public Position(int bSize)
36          {
37
38          }
39
40          public Position Clone()
41          {
42              Position clone = (Position)this.MemberwiseClone();
43              clone.piece = (PIECE[,])piece.Clone();
44              return clone;
45          }
46      }
```

The `Position` class represents a board (the piece array) and the pieces on it. We also hold the number of white pieces and black pieces on the board, as either the `computerScore` or `humanScore`. We will need to clone an existing position when we generate nodes in our game tree.

The following methods are all in class `Position`.

```
47          public List<Point> flips(Move move)
48          {
49              //details left out…
50          }
```

The method `flips` returns a list of positions whose "stones" must be flipped, given the `move` parameter passed in.

```
51          public void playerMoves(Move move, PIECE playerPiece)
52          {
53              piece[move.move.X, move.move.Y] = playerPiece;
54
55              if (playerPiece == Reversi.computersPiece)
56                  computerScore++;
57              else humanScore++;
58
59              foreach (Point p in move.flips)
60              {
61                  piece[p.X, p.Y] = playerPiece;
62                  if (playerPiece == Reversi.computersPiece)
63                  {
64                      humanScore--;
65                      computerScore++;
66                  }
67                  else
68                  {
69                      humanScore++;
70                      computerScore--;
71                  }
72              }
73          }
```

The method `playerMoves` executes a move. It adjusts the scores and accomplishes the flips, given the move.

```
74          public List<Move> validMoves(PIECE sideMoving)
75          {
76              //details omitted
77          }
```

The method `validMoves` determines all moves that can be played, given the current position and the side that is to move. You will see that this is used in our game tree to generate nodes.

The next class, Reversi, does the bookkeeping for the game.

```
78
79      public class Reversi
80      {
81          public Position current = null;
82
83          public Move lastComputerMove = null;
84          public Move lastOpponentMove = null;
85          public static PIECE computersPiece;
86          public static PIECE humansPiece;
87          public int evaluation = 0;
88          public int totNodesExamined = 0;
89          public int nodesExaminedthisMove = 0;
90
91          public List<Point> gameMoves = new List<Point>(10);
92
93          public Reversi(int boardSizeP, PIECE plays)
94          {
95
96          }
97      }
```

The member variables are self-explanatory. We keep track of the color of the pieces for the human and computer, the last moves made by each, and all the moves made during the game. The current position of the game is given in `current`.

The next two methods are in class `Reversi`. They effect the moves on behalf of the human and computer player. The human interface is not shown, but it calls the method `humanMoves`.

```
98
99          public bool humanMoves(int x, int y)
100         {
101             List<Point> flipsToDo = current.flips(
102                 new Move(humansPiece, new Point(x, y)));
103
104             if (flipsToDo.Count == 0)
105                 return false;
106
107             lastOpponentMove = new Move( humansPiece,
108                 new Point(x, y), flipsToDo);
109
110             current.playerMoves(lastOpponentMove, humansPiece);
```

```

            return true;
        }
```

The method `humanMoves` accepts the square on which the human has placed her piece. It obtains the flips that result. If there are none, the attempted move is invalid and false is returned. Otherwise, it executes the move via the method `playerMoves`.

```
        public bool computerMoves()
        {
            ReversiNode root = new ReversiNode(humansPiece, current,
                5);

            Graph<ReversiNode> graph = new Graph<ReversiNode>(root);

            nodesExaminedthisMove = 0;
            foreach (ReversiNode rn in graph.depthFirst())
            {
                nodesExaminedthisMove++;
            }

            totNodesExamined += nodesExaminedthisMove;

            evaluation = root.bestChild.evaluation;

            if (root.bestChild.moveTaken == null) //computer must pass
            {
                lastComputerMove = null;
                return false;
            }

            current = root.bestChild.position;
            lastComputerMove = root.bestChild.moveTaken;

            return true;
        }
```

The method `computerMoves` does the DFS on the game tree. The nodes for that graph will be discussed below. After the DFS is done, the root will contain the evaluation of the position, as well as the move to make (`moveTaken`). If no move is possible, the computer must pass (we return false).

The classes discussed support indirectly the game tree graph. These are DFS nodes and are defined in class `ReversiNode`.

```
    public class ReversiNode: IGNode<ReversiNode>
    {
        public ReversiNode bestChild = null;
        PIECE sideThatMoved;   //side that made the moveTaken, achieving position
        public Move moveTaken = null;
        public int evaluation = 0; //computer's tiles- human's tiles.
                                   //human tries to MINimize,
                                   //computer to MAXimize this.
        public Position position = null;   //after moveTaken was made
        List<Move> moves = null;

        int depth = 0;

        ReversiNode theParent = null;

        static int depthBound = 4;

        public ReversiNode(PIECE side, Position pos, int depthB)
        {
                    //use for setting up root node
            sideThatMoved = side;
            position = pos;
            depthBound = depthB;
        }

        public ReversiNode(ReversiNode par)
        {
            theParent = par;
            if (theParent != null)
            {
                if (theParent.sideThatMoved == PIECE.BLACK)
                    sideThatMoved = PIECE.WHITE;
                else sideThatMoved = PIECE.BLACK;
                depth = theParent.depth + 1;

                position = theParent.position.Clone();
            }
        }
    }
```

The node represents a position, and the last move taken to achieve the position. The computer is taken to be MAX, so we are trying to maximize our heuristic. This is simply the advantage, in terms of number of stones, that the computer has over the human. We have written the program so that the computer can play either white or black. We have set the depth bound to four. This is somewhat arbitrary, but achieves rapid play on moderately

powered computers. The heart of the DFS are the methods `firstChild` and `nextSibling`.

```
        public ReversiNode firstChild()
        {
            if (depth == depthBound)
            {
                //leaf node: propagate values
                evaluation = position.computerScore -
                    position.humanScore;
                return null;
            }

            ReversiNode child = new ReversiNode(this);

            child.moves =
    child.position.validMoves(child.sideThatMoved);
            if (child.moves.Count > 0)
            {
                child.moveTaken = child.moves[0];
                child.moves.RemoveAt(0);
                child.position.playerMoves(child.moveTaken,
                    child.sideThatMoved); //make the move
            }
            return child;
        }
```

If we are at the depth bound, we just calculate the evaluation and return null. This will force backtracking, and a call to `nextSibling` against this node. We do not do any minimax processing in `firstChild` since an evaluation for a node N is not valid until all nodes beneath N have been evaluated. We will do all minimax processing in `nextSibling`, since the target node for that method is guaranteed to have all children visited. This is the central feature of DFS.

The `firstChild` processing just creates a new node with all `validMoves` in it. If there are none, the player represented by the node must pass, and the node created is identical to its parent.

If there are valid moves, the first one is executed and removed from the child.

```
        public ReversiNode nextSibling()
        {

            if (theParent == null)  //no sibling on root
                return null;

```

```
            /*We know that all children under this node must have
              been processed, since we are doing a DFS. Hence it
              is OK to update parent evaluation at this point
              (via minimax).
            */

            minimax();

            if (moves.Count == 0)
            {
                return null; //no legal moves (left)
            }

            ReversiNode sib = new ReversiNode(theParent);

            sib.moves = moves;

            sib.moveTaken = sib.moves[0];
            sib.moves.RemoveAt(0);
            sib.position.playerMoves(sib.moveTaken, sib.sideThatMoved); //make the move
            return sib;
        }
```

After doing the minmax processing, we just take the next valid move (if any), removing it from the list.

```
        bool minimax()
        {
            if (theParent == null)
                return false;

            bool propagate = false;
            if (theParent.bestChild == null)
                propagate = true;

            else if (theParent.sideThatMoved == Reversi.computersPiece)
            {
                /*parent is position after computer moved. It's evaluation
                  assumes human will pick the minimum of nodes at
                  this level.  Parent inherits that minimal value.
                  Because parent is "human to move", parent is a
                  MIN node. Human tries to minimize the evaluation, which
                  is computerScore-humanScore.
                */
                if (theParent.evaluation > evaluation)
                    propagate = true;
            }
```

```
            else
            {
                //parent a MAX node
                if (theParent.evaluation < evaluation)
                    propagate = true;
            }

            if (propagate)
            {
                theParent.evaluation = evaluation;
                theParent.bestChild = this;
                return true;
            }

            else return false;
        }
```

Remember that `minimax` is called against a node when all of the node's children have been processed. The purpose of this method is to propagate the node's evaluation to its parent. If the parent has no evaluation (`bestChild` is null), it is propagated. Otherwise, it is propagated depending on a comparison against the parent's current evaluation. The comparison depends on whether the parent is a MAX node or a MIN node.

We return a Boolean that says if we changed the parent's evaluation. This will be used in our alpha/beta pruning logic.

Alpha/Beta Pruning

Minimax is a fine algorithm, but it examines more nodes than are necessary. This takes a bit of explanation. We start with a definition: a node is *fully evaluated* if all the nodes beneath it have been evaluated (have received a `bestChild` and hence an `evaluation`). If a node is fully evaluated, we know that its evaluation is the best score obtainable by both players, if both play correctly, assuming the game reaches that node.

A node is *evaluated* if it has a non-null `bestChild`. How does this happen? We update the `bestChild` in a parent node, when one of its children has become fully evaluated. If you reexamine the `nextSibling` code, you see that calling `minimax` updates the parent of the target node. We would not be in method `nextSibling` (in a DFS), unless we had backtracked to a node for which all children had been visited.

What does it mean for a node, N, to be evaluated? It means that if the player to move at N selects a certain position (i.e. a child node of N), she can be assured of the score in N's evaluation. Perhaps she can do better than that score (because not all possible moves under the node N have been visited yet), but she can do no worse. It also means that for the person who is *not* to move, it is not possible to do better than the evaluation, and she might well do worse.

If you refer back to figure 8.1 we will illustrate the above ideas. Ignoring the rules and positions of this particular game, let's suppose the DFS has backtracked to node 16. That means it is calling `nextSibling` against node 16. Hence, all children under node 16 have been processed and node 16 is fully evaluated. Let's suppose its evaluation is 50. The minimax processing will look at node 16's parent, node 12, and update it with an evaluation of 50. Node 12 is a MIN node (MIN to move). We now know that MIN can get a score of 50 or better by picking node 16, if she ever gets to node 12. Perhaps she can do better (maybe a score of 40, 0, -100…) when we go on to evaluate nodes 17, 18, and 19 but she is guaranteed of an evaluation of 50 because she can pick node 12 as her next move.

Now suppose the depth search continues. The method `nextSibling` will return node 17, and calls to `firstChild` will continue by returning nodes 20 and 24.

Suppose that our DFS (eventually) backtracks to node 27 and calls `nextSibling` against it. This will make minimax update the parent of node 27, which is node 26. Suppose the evaluation is 100. This is a MAX node; thus if MAX can get there he can assure a score of +100. But node 12 is an ancestor of node 26: the only way the game can get there is to go through node 12 first. But in that case, we know MIN can force the game to a conclusion of at least 50 (maybe less). So MIN will never let the game proceed to node 26.

This means that we can return `null` from our `nextSibling` call against node 27. All potential siblings of node 27, along with their children, are thereby pruned from the graph.

When we drop a subtree because a node's evaluation is less than some parent's (i.e. some node in the parent chain), it is called an *alpha prune*. If

we drop the subtree because the node's evaluation is greater than a parent's evaluation, it is a *beta prune*.

If a MIN node has an evaluation that is smaller than a MAX node in the parent chain of the MIN node, the latter and all of its children can be pruned (MAX will never let the game get to the MIN node: he has a better strategy available). If a MAX node has an evaluation larger than that of some MIN node parent, the MAX node and all of its children can be dropped from the tree.

The above logic is called alpha/beta pruning. To implement it we need make one change to the `nextSibling` code, and add two new methods. The change to `nextSibling` just replaces the call to `minimax()` with the following code:

```
        if (minimax())
            {
                //minimax changed parent evaluation: try alpha/beta

                bool prune;
                if (theParent.sideThatMoved == Reversi.computersPiece)
                    prune = theParent.alphaPrune();   //for MIN node
                else prune = theParent.betaPrune();

                if (prune)
                    return null;  //says no more kids under this parent
            }
```

There is no need to do the pruning test unless the evaluation has changed (note that parents' evaluations cannot change until we are done processing all the children, since we are doing a DFS). Remember that `minimax` returns true if the parent's evaluation was changed by `minimax`.

The two pruning methods (both are in class `ReversiNode`) are thus:

```
        bool alphaPrune()
        {
                //we have a MIN node (computer has moved)
            ReversiNode node = theParent;
            while (node != null)
            {
                    //look for a human-moved (MAX) node with an
                    //evaluation, that is larger than the MIN node
                    //and that is an ancestor of this node
                if (node.sideThatMoved == Reversi.humansPiece &&
```

```
                    node.bestChild != null &&   //means evaluated
                    node.evaluation >= evaluation)
                        return true;

                node = node.theParent;
            }
            return false;
        }

        bool betaPrune()
        {
            //we have a MAX node
            ReversiNode node = theParent;
            while (node != null)
            {
                //look for a MIN node with an
                //evaluation, that is smaller than the MAX node
                //that is an ancestor of this node
                if (node.sideThatMoved == Reversi.computersPiece &&
                    node.bestChild != null &&   //means evaluated
                    node.evaluation <= evaluation)
                    return true;

                node = node.theParent;
            }
            return false;
        }
```

These methods just chase the parent chain looking for a node that causes a pruning.

Alpha/beta pruning can eliminate the generation of many nodes. In one game of Reversi that took 38 moves to finish (19 by each side), the number of nodes returned by the DFS was 520,461 without the pruning. With alpha/beta pruning, the number of nodes in the DFS was 29,644. Alpha/beta pruning is a powerful optimization.

Summary of the Game Tree Design Pattern

- Game tree is used to calculate computer moves in two-person, turn-based games.
- The game tree is searched with a DFS. When it is complete, the root node will contain information as to the best computer move. It also contains the sequence of expected moves by both computer and human, to the depth bound.

- The depth bound is set before the DFS starts. It is used in `firstChild` to return null when the depth bound is reached.
- The method `nextSibling` contains the calls to do the minimax and alpha/beta logic.
- The method `firstChild` must get the list of legal moves for the position, execute the first move in the list, put the resulting position in the new child, then remove the move from the list.
- The method `nextSibling` forms a new node from the list of moves in the current node. The new node forms a new position by executing the first move from the list, and then removes the move from the list.

You will find that most of the logic in your game is outside of the game tree logic, probably in the user interface and in the legal move generation.

Reversi can not duplicate positions because it adds a stone at each move (unless one player passes). Hence we did not need any logic to prevent the computer from repeating a position. In a game where positions can be repeated, you may need to keep track of previous positions and prevent their reoccurrence.

This version of Reversi is not hard to beat, with a little practice. Two suggested improvements are to increase the depth bound and the power of the heuristic. The latter is about as simple as it can be. Some enhancements might be to give a premium to corner and edge moves, and to favor computer moves that limit the number of human moves that are possible from the resulting position. You can use these additional factors to break ties (i.e. where the two moves result in the same stone advantage), or in a formula that weights them, along with the stone advantage to get a numeric evaluation that better predicts the game outcome.

Iterative Deepening and Move Ordering

To use the computer's time efficiently, you might implement "iterative deepening". This does many depth-first searches, starting with a depth bound of one, and incrementing that until the computer runs out of time. After each DFS, we sort the possible first moves by the computer by the evaluation obtained for each (this is called "move ordering"). Thus the next DFS will start with the most promising moves. Although it may seem

wasteful to redo a DFS many times, iterative deepening has proven its value in games like chess. It gives an evenhanded chance for all moves to be evaluated, given a time limit that varies from move to move.

There are many other techniques you can employ in writing a game program. The literature is vast and we have only touched the surface.

9 *Simulated Annealing*

WE HAVE SEEN that an exhaustive search for TSP (Traveling Salesman Problem) is not feasible when we have a large number of cities. However, if we are willing to accept a "pretty good", but not necessarily the best solution, we can find a tour that is reasonably close to the best one. Furthermore, we can acquire the solution in a reasonable amount of time.

Simulated annealing (SA) is one of a number of techniques called "local search". These techniques start with some solution (which may be far from the best), and change it to another solution that is "not far" from the previous one. They proceed by making small changes, according to their algorithm, until they have altered the starting solution to one that is fairly close to the optimal.

Simulated annealing is a stochastic algorithm, meaning that it uses random numbers in its execution. It produces a sequence of solutions, each one derived by slightly altering the previous one, or by rejecting a new solution and falling back to the previous one without any change.

When SA starts, it accepts almost any alteration of the previous solution, even if it is much worse than the previous one. However, the probability with which it will accept a worse solution decreases with time, and with the "distance" the new (worse) solution is from the old one. It always accepts a new solution if it is better than the previous one.

You can look at the alteration of the previous solution as a "move". The application designer has to devise the moves. SA will either accept the solution resulting from the move or reject it (i.e. take the previous solution without change as the next "move").

The name "simulated annealing" is derived from its analogy to the annealing process wherein a material like steel is heated to high temperature and then gradually cooled. The gradual cooling allows the material to cool to a state in which there are few weak points. It achieves a kind of "global optimum" wherein the entire object achieves a minimum energy crystalline structure.

The contrasting process wherein the material is rapidly cooled allows some parts of the object to settle to areas of strength, but there are also places where the object is easily broken (areas of high energy structure). The object has achieved some local areas of optimal strength, but is not strong throughout, with rapid cooling.

The SA Algorithm

There are analogies to the cooling process: the SA algorithm has a "temperature" which is gradually lowered, and an "energy" which corresponds to the number (the "score") we are trying to optimize. In the case of TSP, the score is the length of the current tour.

When a new move is made (for TSP, an alteration of the previous tour), a delta is calculated with the previous solution. For TSP, this is just the length of the new tour minus that of the previous one. If this delta is less than 0 (i.e. the new tour is better than the previous one), we always accept the new tour. Otherwise, we accept it with a certain probability. This probability gets smaller as the temperature decreases, and is also smaller for larger deltas than for smaller ones.

For a maximization problem the delta will be formed by taking the old score minus the new one. So again, a negative delta represents an improvement and is always taken. Else, we can assume the delta is positive (representing a worse solution than the previous one).

The algorithm takes a random number (between 0 and 1) and compares it to $dE = \exp(-delta / T)$, where T is the temperature. If the random number is less than this, we accept the new (poorer) solution. Otherwise, we reject it and continue with the old (better) solution.

TSP SOLVED VIA SA

We will now present a C# solution to TSP, using simulated annealing. The SEL has an `AnnealGraph` class and some interfaces to help with this. Although SA is quite different from the other graph searching algorithms we have explored, you should be able to see analogies.

The first requirement for the designer is to invent the object we are trying to optimize. For TSP, this is a `Tour`. This will be analogous to a node in a graph, and this object must inherit from `IAnnealNode`

```
public class Tour:IAnnealNode<Tour>
{
    public List<City> cities;
    public int sumLegSq;
    bool isSwitchBack = false;

    static Random rand = new Random();

    public Tour(List<City> citiesP):
        this(citiesP, City.sumLegSq(citiesP),false)
    {

    }

    public Tour(List<City> citiesP, int distSqP, bool isSwitchBackP)
    {
        cities = citiesP;
        sumLegSq = distSqP;
        isSwitchBack = isSwitchBackP;
    }
}
```

It might be helpful to look back at Chapter 6 to review the TSP problem and the class `City`. Our `Tour` will contain a list of cities (which starts and ends with the start city), and the length of the tour (actually, the sum of the squares of the distances of each leg of the tour) in `sumLegSq`. We also set up a `Random` (`rand`) to be used in generating alterations of a tour (the "moves").

This class should contain the methods that make "moves", or alterations of a given tour into another one. You only need one method, but we have written two. The first one, `cutPaste`, removes a segment (a contiguous sublist of cities) from the current tour and inserts it in another spot.

```
    public Tour cutPaste()
    {
        /*
        Remove a segment, then insert it somewhere else.  OK if
        we insert back where we started.
        */
        List<City> middle;
        List<City> frontBack;

```

```
            int start, end;

            extract(out middle, out frontBack, out start, out end);

            //reduce distance by dist's from endpts we are removing.
            //add back in the dist between the points that are now adjacent

            int newDistSq = sumLegSq -
                (cities[start - 1].distSq(cities[start]) +
                cities[end].distSq(cities[end+1]))
                +
                cities[start-1].distSq(cities[end+1]);

            int position = rand.Next(1, frontBack.Count - 1);

            //subtract out the connection at position we are breaking
            newDistSq -=
                    frontBack[position-1].distSq(frontBack[position]);


            frontBack.InsertRange(position, middle);

            int length = end - start + 1;

                //add back in the dist's of segments between endpts
                //of the piece we have inserted back in.

            newDistSq += frontBack[position - 1].distSq(
                frontBack[position]) +
                frontBack[position + length - 1].distSq(
                    frontBack[position + length]);

            /*
            if (newDistSq != City.sumLegSq(frontBack))
                throw new ApplicationException("idiot");
            */

            return new Tour(frontBack, newDistSq, false);
        }
```

The majority of the logic is figuring out the length of the new tour without having to do the arithmetic for each leg of the new tour. We could have just called `City.sumLegSq` to do this, but we are trying to be a bit more efficient.

The method `extract` just finds the segment to extract (`middle`), and returns the piece that is left over (`frontback`):

```
        public void extract(out List<City> middle,
            out List<City> frontBack, out int start, out int end)
        {
            start = 0;
            end = 0;

            while (start == end)
            {
                start = rand.Next(1, cities.Count - 1);
                end = rand.Next(1, cities.Count - 1);
            }

            if (start > end)
            {
                int hold = start;
                start = end;
                end = hold;
            }

            middle = new List<City>(10);
            frontBack = new List<City>(10);

            for (int i = 0; i < cities.Count; i++ )
            {
                if (i >= start && i <= end)
                    middle.Add(cities[i]);
                else frontBack.Add(cities[i]);
            }
        }
```

Our second method for finding a new tour, given a previous one, is to extract a segment of cities, reverse it, and put it back where it was. Here it is:

```
 public Tour switchBack()
        {
            if (cities.Count < 3)
                return this;

            List<City> middle;
            List<City> frontBack;

            int start, end;

            extract(out middle, out frontBack, out start, out end);

                //subtract out the dist's to the endpts of segment
                //we are removing.
```

```
            int newDistSq = sumLegSq -
                (cities[start - 1].distSq(cities[start]) +
                 cities[end].distSq(cities[end+1]));

            middle.Reverse();
            frontBack.InsertRange(start, middle);

                //add back in the lengths of new segments to the endpts
            newDistSq += frontBack[start - 1].distSq(frontBack[start]) +
                frontBack[end].distSq(frontBack[end+1]);

            /*
            if (newDistSq != City.sumLegSq(frontBack))
                throw new ApplicationException("idiot");
            */


            return new Tour(frontBack, newDistSq, true);
        }
```

To implement the `IAnnealNode` interface, you need to write a method to return the next node and to return a number (`score`) that is the number we are trying to optimize (minimize or maximize). The `nextNode` method follows:

```
        public Tour nextNode(AnnealGraph<Tour> graph)
        {
            Tour tour;
            if (!isSwitchBack)
                tour = switchBack();

            else
                tour = cutPaste();

            return tour;
        }
```

In `nextNode`, we have elected to construct two different kinds of nodes, depending on the type of "move" we make. We alternate between these types of moves. Ordinarily, you need only one type of move in your applications. We installed two as a frill for this problem.

The `nextNode` method is going to be called by the graph we are searching. This is analogous to other graph searching logic in SEL. But here we have only one method that is called by the graph (instead of the `firstChild` and `nextSibling` methods).

If you return `null` in `nextNode`, the annealing iterator will stop. We have elected to stop the annealing in another way (explained below).

SEL will pass in the graph that is doing the annealing (in the call to the `nextNode` method) in case your logic needs it. We elected not to use this parameter in our TSP problem.

You also need to implement a `score` method as part of the interface:

```
143         public double score
144         {
145             get { return sumLegSq; }
146         }
```

The above method, also in class `Tour`, just returns a double that our annealer will try and minimize.

Note that there is no `parent` method or reference in the `Tour` node definition. The solution will be found in the last node generated, and no others are required. In fact, the annealing process generates many nodes (perhaps tens of millions), so you should not hold references to them.

The majority of your work in using SA will be to construct a class like Tour. The "moves" you make should be random, and (potentially) small (i.e. some moves should return an object that is not much different than the previous one). Furthermore, it should (in theory) be possible to reach any configuration from any other configuration, by taking enough moves.

To solve the annealing problem, you will need to make an `AnnealGraph` and execute its iterator. You will also have to control the temperature and stopping conditions. We have set up another application class to do this for TSP:

```
147     public class TSPanneal
148     {
149         public Tour rootNode;
150         public Tour currentNode;
151
152         public double startTemp;
153         public int iterations = 0;
154
155         int maxTempSteps = 1000;
156         int maxTriesAtATemp;
157         int maxSuccessAtATemp;
158
159         public AnnealGraph<Tour> graph = null;
```

```
160
161         public TSPanneal(List<City> cities)
162         {
163             City.completeCircuit(cities);
164             rootNode = new Tour(cities);
165
166             //initial temp must be >> largest delta(sumLegSq)
167             startTemp = rootNode.sumLegSq * rootNode.sumLegSq;
168
169             maxTriesAtATemp = 100 * cities.Count;
170             maxSuccessAtATemp = 10 * cities.Count;
171
172             currentNode = rootNode;
173         }
174     }
```

The constructor takes a list of cities, with the first one as the start city. We construct a tour via `completeCircuit`, and save it as our initial tour (the `root` node). The `startTemp` corresponds to the temperature. It will be adjusted downward as annealing proceeds, but must be set initially to a number much larger than the largest delta possible (the delta being the difference between two tours' `sumLegSq`).

The `maxTriesAtATemp` is used by the application to reduce the temperature, as is the `maxSuccessAtATemp`. The first number will cause the application to lower the temperature as soon as we have made that number of moves at a single temperature. We may lower the temperature sooner than that if we have enough successes (accepted tours) at a given temperature, as specified by `maxSuccessAtATemp`. These two parameters will cause the application to run faster if they are small (but with poor results, perhaps), or slower (but with a better final result, perhaps) if they are large. You should experiment with various settings depending on your application.

The TSP solution is obtained from an `AnnealGraph`:

```
175         public void solve()
176         {
177             graph =
178                     new AnnealGraph<Tour>(rootNode, startTemp,
179                         ANNEALTYPE.MIN);
180
181             foreach (Tour t in graph.search())
182             {
183                 iterations++;
184
```

```
                currentNode = t;

                if (temperatureCheck())
                    break;
            }
        }
```

We have placed `solve` method in the `TSPanneal` class. The constructor for the `AnnealGraph` wants a root node, a starting temperature, and the kind of annealing to do (maximize or minimize). Since we are trying to minimize the length of the tour, we set it up as MIN.

The node returned by the iterator is obtained when the iterator calls your `nextNode` method (discussed above). The iterator stops when you return null from that method. We have elected to stop the iterator with the `temperatureCheck` method instead. The solution to the TSP is found in the last node returned by the iterator, which we have deposited in the `currentNode` variable.

At any given time, there are at most two `Tours` in memory (not counting the root). These are the last two nodes that were returned by `nextNode`. They are kept by the `AnnealGraph`. One reference will be dropped as soon as one is returned to you by the iterator `search`. Thus although many nodes may be generated, the `AnnealGraph` is quite stingy with memory.

Because the `AnnealGraph` can reject the `nextNode` tour and return the previous one instead, you must not alter the target `Tour` in `nextNode`. Thus you need to construct an entirely new one in that method, so that it can be rejected if necessary and the previous (unaltered) tour be returned by the iterator.

Here is a memory optimization for your own `nextNode`. Instead of allocating a new node you could declare a set of static nodes (preallocated) for reuse, say `node1` and `node2`. If the graph calls `nextNode` against `node1` (i.e. the "this" object is `node1`), then fix up `node2` and return that. If it is called against `node2` then fix up `node1` and return that. This will relieve the garbage collector of having to clean up many (perhaps millions) of short lived nodes. But remember that the graph is holding the last "this" node it delivered to `nextNode` and may need to deliver it again on its very next call. So you *must not* alter the "this" node in method `nextNode`; you *can,* however, alter any other node.

The temperature is entirely controlled by the application. We call `temperatureCheck` to decide if the temperature should be lowered, or the annealing stopped:

```
1           bool temperatureCheck()
2           {
3               if (graph.tempSteps >= maxTempSteps)
4                   return true;        //stop
5
6               if (graph.triesAtCurrentTemp >= maxTriesAtATemp ||
7                   graph.successesAtCurrentTemp >= maxSuccessAtATemp)
8               {
9                   graph.temperature *= 0.9d;    //lower temperature
10                  if (graph.temperature < 0.0000001)
11                      return true; ;
12              }
13
14              return false;
15          }
```

The above method is owned by the application class, `TSPanneal`. The `graph` has an accessor (`temperature`) that you can get and set. The `graph` will also update the `triesAtCurrentTemp` and `successesAtCurrentTemp` for you. The application class can test these before lowering the temperature, if desired.

Actually, you can lower the temperature any way you please. You can even raise it if you find that useful. There are an almost infinite number of ways to conduct the annealing. They are all controlled in this method.

Note that line 9 provides the "annealing schedule", which is the rate at which the temperature is lowered. If you lower it too fast, the results may not be as good as a slower schedule. But a slow schedule will take longer. You will need to experiment with this.

You could have put the temperature check in the `nextNode` method, since the `graph` is passed into that method. You would have stopped the graph search by returning `null` from that method. Our designer felt that putting the code in the body of the iterator was a bit clearer, because the temperature is a global concept, not a node by node one.

Figure 9.1 shows the results of TSP on 26 cities.

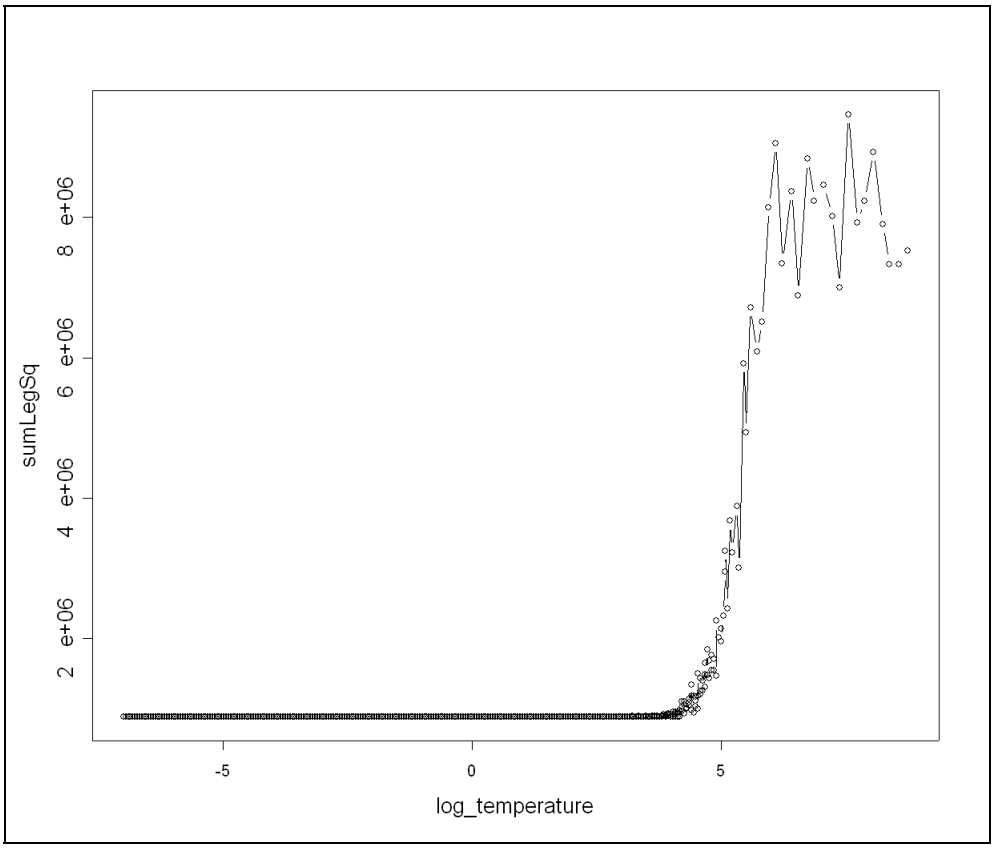

Figure 9-1. TSP (26 cities) Simulated Annealing

In Figure 9.1, we have graphed the log(temperature) against the `sumLegSq`. Remember that the temperature decreases with time, so that time starts at the right of the figure and increases to the left as the temperature decreases. You can see how SA accepts tours with longer distances at higher temperature. But as the temperature decreases, the probability of accepting a tour worse than the preceding one decreases. The knee of the graph is at about 3500 "degrees", and we see little improvement beyond that temperature.

There is a large literature on SA and consequently you can find many variations and tweaks. Some of them are:

- Decrease the total run time by starting the temperature at sqrt(bestRun). Some researchers conclude that there is little point spending too much time at high temperatures. If you start the annealing by accepting tours

that are about 1.5 times the best tour, you can speed up the run with little loss of quality of the final result.

- Do not generate long edges. The idea is to impose a grid on the area containing the cities. The granularity of the mesh is determined by the number of cities. For example, insure that no cell in the mesh contains more than 100 cities. Then, when selecting a "move" pick a random grid cell and two random cities within that cell to form the move in such a way that resulting new edges do not thus exceed the grid cell size. (Details intentionally left to the reader).
- Use adaptive cooling. The idea is to change the rate at which you lower the temperature by looking at the variance of the tour lengths at a given temperature. The higher the variance, the longer one should spend at a given temperature.

  Reference [4] contains other ideas on SA, as well as a discussion on methods used to attack the TSP. The basis for our implementation of TSP is in reference [3].

Summary of the Simulated Annealing Design Pattern

- SA is good for optimization problems.
- You need to be able to construct an initial solution, and method for changing one solution into another. This method should be able to construct new solutions that are "close" to the existing one, and to be able to reach any possible solution, given enough applications.
- You should use the data available from the `AnnealGraph` to determine how to lower the temperature. This "annealing schedule" should be determined by experimentation.
- You must implement a node class that inherits from `IAnnealNode<T>`. This allows the `AnnealGraph` object to access the score for the current node and to call `nextNode` against it. The `nextNode` method that your application implements must not alter the current node, since this is cached by the `AnnealGraph` and might be returned in the iterator after the next node is created. Each node represents a complete solution to the problem.

- To solve the SA problem, you will create an `AnnealGraph<T>` object and invoke the `search` iterator on it.

SA is among the easiest of algorithms to implement. However, more than most it is unpredictable in its execution and will require the designer's experimentation.

Envoi

The techniques in this book are quite powerful. You may wish to test them, and your mettle, in a programming contest. You can grapple with some suitable problems (along with some of the best programmers in the world) at http://www.recmath.org/contest. Contests are held 2-3 times a year and are free to enter.

Bibliography

[1] Erich Gamma, *et al*, ***Design Patterns***, Addison-Wesley, 1995. The classic book on object-oriented design patterns by the "gang of four".

[2] Judea Pearl, ***Heuristics***, Addison-Wesley, 1985. An indispensable book that gives a mathematical analysis of many of the algorithms in our book, including game trees, A*, DFS, and many others.

[3] William H. Press, *et al*, ***Numerical Recipes in C***, Cambridge University Press, 1988. This book contains a useful chapter on simulated annealing. It was the basis for our SA solution for TSP.

[4] Emile AArts and Jan Karel Lenstra, editors, ***Local Search in Combinatorial Optimization***, Princeton University Press, 2003. This book contains much material on simulated annealing, as well as other local search techniques. It includes a chapter on the TSP as well.

[5] Richard Bronson and G. Naadimuthu, ***Operations Research Second Edition***, Schaum's Outlines, McGraw Hill, 1982. This book contains material on Dynamic Programming and Branch and Bound. It also contains much material on techniques not included in our book. For some problems, these are more suitable than the graph searching techniques we have explored.

[6] Thomas H. Cormen, *et al*, ***Algorithms***, MIT Press, 1990. This book gives a clear mathematical analysis of many of the algorithms in our book. It is very comprehensive and widely used.

[7] T.C. Hu and M.T. Shing, ***Combinatorial Algorithms***, Dover Publications, 2002. This is a fairly comprehensive book at a modest price.

Except for [1], none of the above books is object-oriented. But all are quite useful.